LEARN / BUILD / GROW

Small Steps
Make Big
Developers

JAVASCRIPT

FROM BASICS TO REAL-WORLD PROJECTS

- Syntax & Basics
- DOM Manipulation
- Functions & Objects
- ES6+ Features
- Real-World Projects



DHAMODHARAN

CONTENTS

What is inside

Fifty chapters that start with variables, data types, strings and operators, then build through arrays, objects, functions, classes and the DOM into asynchronous JavaScript, APIs, security, debugging, sixty interview answers and six mini projects.

01	Introduction of JavaScript	01
02	Variable	02
03	Data Types	04
04	String Method	06
05	Operator	10
06	Windows Methods	20
07	Date Object	27
08	Array	31
09	Array Destructuring	40
10	Classes	42
11	Object	48
12	Object Destructuring	55
13	Spread Operator (...)	59
14	Conditional Statements	62
15	Looping Statements	69
16	Set	80
17	map(), filter(), reduce()	85
18	Functions	92
19	Anonymous Function	101
20	Arrow Function	104
21	this	109
22	Scope	114
23	Higher-Order Function	118
24	OOPs	120
25	Access Modifiers	125

CONTENTS

26	Events	129
27	Inheritance	138
28	DOM	148
29	Form	163
30	setInterval() and setTimeout()	171
31	Synchronous vs Asynchronous	177
32	AJAX	183
33	Cookies	195
34	Session Storage & Local Storage	200
35	Error Handling	208
36	Rest Parameters	214
37	Closures	217
38	Microtask vs Macrotask	220
39	Promises	223
40	Shallow vs Deep Copy & Prototypes	228
41	ES6 Modules (import / export)	233
42	REST API Integration	236
43	HTTP Methods and Status Codes	242
44	CORS (Cross-Origin Resource Sharing)	247
45	Debouncing and Throttling	251
46	JWT Authentication	254
47	JavaScript Debugging	259
48	Important Topics	267
49	Interview Questions and Answers	279
50	Mini Projects	290

CHAPTER 01

Introduction of JavaScript

WHAT IS JAVASCRIPT? · COMMENTS · PRACTICAL TASKS

1. JavaScript ulagam muzhuvadhum romba popular-ah irukkura programming language.

2. JavaScript enbadhu Web-oda programming language. Adhaavadhu, websites-la interactive features create panna JavaScript use pannuvom.

3. JavaScript kathukkuradhu easy. Beginners kooda basic concepts-la irundhu easy-ah learn pannalaam.

4. JavaScript oru lightweight, cross-platform, single-threaded, interpreted and JIT-compiled programming language.

- Lightweight: Kammiyaana resources use panni work aagum.
- Cross-platform: Different operating systems-la run aagum.
- Single-threaded: Oru main thread-la oru task-ah execute pannum; aana asynchronous features moolama vera tasks-um handle pannalaam.
- Interpreted / JIT-compiled: JavaScript engine code-ah execute pannum; modern engines performance improve panna JIT compilation-um use pannum.

5. JavaScript webpages-ku use panra scripting language-nu kooda solluvaanga.

6. JavaScript webpages develop panna romba famous. Adhu mattum illaama, browser-ku veliya Node.js maadhiri environments-layum JavaScript use pannuvaanga.

Comments

1. JavaScript comments use panni, JavaScript code eppadi work aagudhunu explain pannalaam. Idhanaala code-ah padikkiravangalukku easy-ah puriyum, code-um readable-ah irukkum.

2. Alternative code-ah test pannumbodhu, andha code execute aagama thadukka JavaScript comments use pannalaam. Appo code-ah delete pannaama temporary-ah disable pannittu test pannalaam.

Comments in both PHP and JavaScript

```
// Single Line Comment

/*
   Multi-line
   comment
*/
```

> **Practical Task 1:** Create a webpage that displays a welcome message using JavaScript.

> **Practical Task 2:** Write a JavaScript program using single-line and multi-line comments to explain the code.

CHAPTER 02

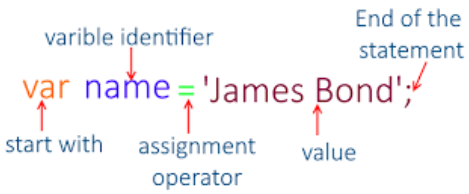
Variable

DECLARATION · VAR · LET · CONST · BLOCK SCOPE

- Variables are containers for Storing Data
- JavaScript Variables can be declared in 4 ways:
- Automatically.
- Using var. →Global / Function Scope
- Using let. →Local / Block Scope
- Using const. →Block Scope

Var Variable

- var variable support redeclared
- var statement oru variable-ah declare panna use aagum.
- Variables na information-ah store panni vachukka use panra containers maadhiri.
- JavaScript-la oru variable create panradha variable declaration nu solluvom.




```
var name = "Dhamodharan";
console.log(name);
```

Explanation:

- var – variable declare panna use pannuvom.
- name – variable name.
- "Dhamodharan" – variable-la store pannirukkura value.
- console.log(name) – variable-la irukkura value-ah output-la display pannum.

let Variable

- let keyword JavaScript-oda ES6 version-la (2015) introduce pannanga.
- let use panni oru variable declare pannita, adhe scope-la adhe variable name-ah marubadiyum declare panna mudiyadhu.
- let variable-ah use panradhukku munnadi, andha variable-ah declare pannanum.
- let variable oru block { } kulla declare pannina, andha block-kulla mattum dhaan access panna mudiyum.

Cannot be Redeclared

- let use panni declare panna variables-ah, adhe scope-la marubadiyum declare panna mudiyadhu.
- Idhanaala theriyaama same variable-ah redeclare panradha avoid pannalaam.

```
let name = "Dhamodharan";
let name = "JavaScript"; // Error
```

Explanation:

- First let name — variable correct-ah declare aagum.
- Second let name — adhe scope-la same variable name-ah marubadiyum declare panradhaala error varum.

Inga let use panni variable-ah oru murai dhaan declare pannirukkom. Adhukku appuram name-oda value-ah change pannirukkom. Idhu allowed.

Remember:

- let → Declare once in the same scope.
- let → Value-ah change pannalaam.
- let → Block scope follow pannum.

Const Variable

- const keyword JavaScript-oda ES6 version-la (2015) introduce pannanga.
- const use panni oru variable declare pannita, adhe scope-la adhe variable name-ah marubadiyum declare panna mudiyadhu.
- const use panni declare panna variable-ku oru value assign pannita, andha variable-ku pudhu value-ah marubadiyum assign panna mudiyadhu.
- const variable-ah { } block-kulla declare pannina, andha block-kulla mattum dhaan access panna mudiyum.

```
const PI = 3.141592653589793;
PI = 3.14;      // Error
PI = PI + 10;   // Error
```

Mela irukkura code-la error varradhukku reason, PI variable-ah const use panni declare pannirukkom.

- First line-la PI-ku 3.141592653589793 value assign pannirukkom.
- Second line-la PI-oda value-ah 3.14-nu change panna try pannirukkom.

- Third line-la PI-oda value-kooda 10 add panni pudhu value assign panna try pannirukkom.

Aana const variable-ku marubadiyum value assign panna mudiyadhu. Adhanaala indha assignment statements error kudukkum.

Remember: const variable-ku value assign pannina piragu, andha variable-ah reassign panna mudiyadhu.

Block Scope

- ES6 (2015) varradhukku munnadi, JavaScript-la Global Scope-um Function Scope-um irundhuchu.
- ES6 version-la let matrum const appadingira rendu mukkiyamaana JavaScript keywords introduce pannanga.
- Indha rendu keywords-um JavaScript-la Block Scope-ah provide pannum.
- { } block-kulla variable declare pannina, andha block-ku veliya andha variable-ah access panna mudiyadhu.

> **Practical Task 1:** Create variables to store your name, age, and city. Display them in the browser console.

> **Practical Task 2:** Create a simple bill calculator using let and const variables.

CHAPTER 03

Data Types

PRIMITIVE & NON-PRIMITIVE · STRING · NUMBER · BOOLEAN · UNDEFINED · NULL · SYMBOL · TYPEOF

1. JavaScript-la Data Types enbadhu, oru variable-la enna type of data store pannirukkom enbadhai define pannum.

2. JavaScript-la rendu vagaiyaana data types irukku:

- Primitive Data Types
- Non-Primitive Data Types

Primitive Data Types

1. Primitive data types enbadhu, oru single value-ah represent panra basic data types aagum.

Example: String, Number, Boolean, Undefined, Null, BigInt, Symbol.

2. Primitive data types immutable-nu sollappadum. Adhaavadhu, oru value create aana piragu andha value-ah change panna mudiyadhu. Aana andha value store pannirukkura variable-ku pudhu value-ah reassign pannalaam.

Non-Primitive Data Type

Non-primitive data type-la Object include aagum.

The Object Data Type

JavaScript-la Object data type moolama different types of data-va store pannalaam.

Object data type-la pinvarum 3 types irukkum:

1. An Object: Oru object-la multiple properties-ah key: value format-la store pannalaam.
2. An Array: Array use panni multiple values-ah ore variable-la store pannalaam.
3. A Date: Date object use panni date matrum time-related information-ah store pannalaam.

String Datatype

String enbadhu text-ah store panna use aagum. JavaScript-la strings-ah quotes kulla write pannuvom.

- Single quotes: 'Hello'
- Double quotes: "Hello"
- Backticks: `Hello`
- Single quotes-um double quotes-um practically ore maadhiri dhaan work aagum. Rendum use panni text-ah store pannalaam.
- Backticks (``) generally oru string-kulla variables illa expressions-ah include panna use aagum.
- Backticks kulla ${} use panni variable illa expression-ah string-kulla include pannalaam. Idhukku Template Literals nu peru.

Number Datatype

Number enbadhu JavaScript-la integer matrum decimal numbers-ah represent panna use aagum.

- Integer: Decimal point illaadha whole numbers.
- Floating-point numbers: Decimal values irukkura numbers.
- Exponentials: Periya numbers-ah illa romba chinna numbers-ah scientific notation-la represent pannalaam.

Boolean Datatype

Indha data type logical values-ah represent pannum. Boolean enbadhu rendu values-la edhaavadhu onna represent pannum:

- true
- false

Undefined Datatype

undefined enbadhu oru variable-ku value assign pannappadala enbadhai represent pannum.

Oru variable-ah declare pannittu, adhukku endha value-um assign pannala na, andha variable-oda value undefined-ah irukkum.

Eg:

```
let name;
console.log(name);
```

null Datatype

JavaScript-la null enbadhu oru special value. Idhu oru variable-la empty value illa unknown value irukkuradhai represent panna use aagum.

Eg:

```
let name = null;
console.log(name);
```

undefined vs null:

- undefined — Variable declare pannirukkom, aana value assign pannala.
- null — Value illai-nu intentional-ah assign pannirukkom.

Symbol Datatype

Symbol datatype JavaScript-oda ES6 version-la (2015) introduce pannanga.

Symbol datatype-la irukkura value-ah Symbol value nu solluvom.

Symbol enbadhu oru primitive datatype. Ovvoru Symbol() call-um unique-ah irukkura symbol value-ah create pannum. Immutable na, create panna symbol value-ah change panna mudiyadhu.

typeof Datatype

Oru variable-la enna type of data irukku-nu kandupidikka JavaScript-la typeof operator use pannuvom.

> **Practical Task 1:** Create variables using String, Number, Boolean, Undefined, and Null data types. Display their types using typeof.

> **Practical Task 2:** Create a student profile using different JavaScript data types.

CHAPTER 04

String Method

LENGTH · CASE CONVERSION · SEARCHING · EXTRACTING · REPLACING · SPLITTING · REPEATING

1. String Length

length – Oru string-la ethana characters irukku-nu count panni return pannum. Spaces-um characters-ah count aagum.

```
let str = "Hello, World!";
console.log(str.length); // Output: 13
```

Explanation: "Hello, World!" string-la letters, comma, space, exclamation mark ellathaiyum serthu total 13 characters irukku.

2. Case Conversion

1. toLowerCase()

String-la irukkura ella uppercase letters-aiyum lowercase-ah maathum.

```
console.log("HELLO".toLowerCase()); // Output: hello
```

2. toUpperCase()

String-la irukkura ella lowercase letters-aiyum uppercase-ah maathum.

```
console.log("hello".toUpperCase()); // Output: HELLO
```

3. Searching and Matching

1. indexOf(substring)

Oru string-kulla specific word illa character enga first time varudhunu index position-ah return pannum. Andha substring illana -1 return pannum.

```
console.log("hello world".indexOf("world")); // Output: 6
```

Explanation: world word index 6-la start aagudhu. JavaScript-la index 0-la irundhu start aagum.

2. lastIndexOf(substring)

Oru substring ethana murai vandhaalum, adhu last-ah enga start aagudhunu index position-ah return pannum. Kidaikkalana -1 return pannum.

```
console.log("hello world world".lastIndexOf("world")); // Output: 12
```

Explanation: Last-ah irukkura world word index 12-la start aagudhu.

3. includes(substring)

Oru string-kulla specific word illa substring irukka illaiya-nu check pannum. Irundha true, illana false return pannum.

```
console.log("hello world".includes("world")); // Output: true
```

4. startsWith(substring)

Oru string specific word illa text-la start aagudha-nu check pannum.

```
console.log("hello world".startsWith("hello")); // Output: true
```

Explanation: String hello-la start aaguradhunaala true return aagum.

5. endsWith(substring)

Oru string specific word illa text-la end aagudha-nu check pannum.

```
console.log("hello world".endsWith("world")); // Output: true
```

Explanation: String world-la mudiyuradhunaala true return aagum.

4. Extracting Substrings

1. slice(start, end)

Oru string-la irundhu namakku thevaiyaana oru portion-ah extract pannum. start index include aagum; end index include aagadhu.

```
console.log("Hello, World!".slice(7, 12)); // Output: World
```

Explanation: Index 7-la start panni, index 12-ku munnadi vara characters-ah extract pannum.

2. substring(start, end)

Idhuvum string-la irundhu oru portion-ah extract pannum. Aana slice() maadhiri negative index-ah support pannaadhu; negative values 0-ah treat pannappadum.

```
console.log("Hello, World!".substring(7, 12)); // Output: World
```

Explanation: Index 7-la irundhu index 12-ku munnadi vara characters return aagum.

5. Replacing and Modifying

1. replace(searchValue, newValue)

String-la irukkura specific text-ah pudhu text-aala replace pannum. String search pannumbodhu, default-ah first matching occurrence mattum replace aagum.

```
console.log("Hello, World!".replace("World", "JavaScript"));
// Output: Hello, JavaScript!
```

2. replaceAll(searchValue, newValue)

String-la irukkura matching text-oda ella occurrences-aiyum replace pannum.

```
console.log("Hello, World! World!".replaceAll("World", "JavaScript"));
// Output: Hello, JavaScript! JavaScript!
```

Explanation: World rendu murai irukkuradhunaala rendu idathulayum JavaScript-nu replace aagum.

3. trim()

String-oda starting-um ending-um irukkura extra whitespace-ah remove pannum. Naduvula irukkura spaces remove aagadhu.

```
console.log("   Hello, World!   ".trim());
// Output: Hello, World!
```

4. trimStart()

String-oda starting-la irukkura whitespace-ah mattum remove pannum.

```
console.log("   Hello!".trimStart());
// Output: Hello!
```

5. trimEnd()

String-oda ending-la irukkura whitespace-ah mattum remove pannum.

```
console.log("Hello!   ".trimEnd());
// Output: Hello!
```

6. Splitting and Joining

1. split(delimiter)

Oru string-ah specific separator base panni split panni, multiple values irukkura array-ah return pannum.

```
console.log("a,b,c".split(","));
// Output: ["a", "b", "c"]
```

Explanation: Comma , irukkura idathula string split aagi, a, b, c nu array values-ah maarum.

2. concat(...strings)

Rendu illa adhukku mela strings-ah onna join panni pudhu string-ah return pannum.

```
console.log("Hello".concat(", ", "World!"));
// Output: Hello, World!
```

Explanation: "Hello", ", ", "World!" moonu strings-um join aagum.

7. Repeating

repeat(count)

Oru string-ah kudukkura count alavukku repeat pannum.

```
console.log("Hi".repeat(3));
// Output: HiHiHi
```

Explanation: "Hi" string 3 times repeat aagum.

8. Other Useful Methods

charAt(index)

Oru string-la specific index position-la irukkura character-ah return pannum.

```
console.log("hello".charAt(1));
// Output: e
```

Explanation: JavaScript-la index 0-la start aagum.

- h → index 0
- e → index 1
- l → index 2
- l → index 3
- o → index 4

Adhanaala charAt(1) output e-nu varum.

> **Practical Task 1:** Create a program that converts a user's name to uppercase and lowercase.

> **Practical Task 2:** Create a program to count the number of characters in a sentence and check whether a particular word exists.

CHAPTER 05

Operator

ASSIGNMENT · ARITHMETIC · COMPARISON · LOGICAL · BITWISE · STRING · TERNARY · OTHER

JavaScript-la Operator enbadhu, operands (values matrum variables) mela operations perform panna use panra oru special symbol.

Types of Operator

- Assignment Operators
- Arithmetic Operators
- Comparison Operators
- Logical Operators
- String Operators
- bitwise Operator
- Ternary operator
- Increment / Decrement Operator
- Other Operators

Assignment Operators

Assignment operator enbadhu, right side-la irukkura value-ah left side-la irukkura variable-ku assign panna use aagum.

2. Simple Assignment Operator (=) operator use panni, right side-la irukkura value-ah left side-la irukkura variable-ku assign pannuvom.

Example:

```
const x = 5;
```

Explanation:

- const – Variable declare panna use pannuvom.
- x – Variable name.
- = – Assignment operator.
- 5 – Variable-ku assign panra value.

Inga = operator use panni, 5 value-ah x variable-ku assign pannirukkom.

Remember: JavaScript-la = enbadhu value assign panna use aagum; rendu values equal-ah irukka-nu check panna use aagadhu.

Arithmetic Operators

Arithmetic operators enbadhu numbers (direct values illa variables) mela mathematical calculations perform panna use aagum. Idhu calculation-oda result-ah oru numerical value-ah return pannum.

Operator	Name	Explanation
+	Addition	Rendu numbers-ah kootta use pannuvom.
-	Subtraction	Oru number-la irundhu innoru number-ah kazhikka use pannuvom.
*	Multiplication	Rendu numbers-ah perukka use pannuvom.
/	Division	Oru number-ah innoru number-aala vagukka use pannuvom.
%	Remainder	Division pannumbodhu meedhi irukkura value-ah return pannum.
++	Increment	Variable-oda value-ah 1 increase pannum.
--	Decrement	Variable-oda value-ah 1 decrease pannum.
**	Exponentiation	Oru number-ah innoru number-oda power-ku calculate pannum.

Comparison Operators

Comparison operators enbadhu rendu values illa variables-ah compare panni, andha comparison correct-ah irundha true-vum, correct-ah illana false-vum return pannum.

Comparison operators-la compare panra operands numbers, strings, Boolean values illa objects-ah irukkalaam.

Strings-ah compare pannumbodhu, characters-oda Unicode values base panni compare pannum. Adhaavadhu, alphabetical order maadhiri character by character compare pannum; aana exact result Unicode values-ah poruthadhu.

Comparison Operators Table

Operator	Name	Explanation
==	Equal to	Rendu values equal-ah irukka-nu check pannum. Data type different-ah irundhaalum type conversion nadakkalaam.
!=	Not equal to	Rendu values equal-ah illana true return pannum.
===	Strict equal to	Rendu values-um equal-ah irukkanum; data type-um same-ah irukkanum.
!==	Strict not equal to	Values different-ah irundhaalum, illa data type different-ah irundhaalum true return pannum.
>	Greater than	Left-side value, right-side value-ah vida perusa irukka-nu check pannum.
>=	Greater than or equal to	Left-side value perusa illa equal-ah irukka-nu check pannum.
<	Less than	Left-side value, right-side value-ah vida chinnadha irukka-nu check pannum.
<=	Less than or equal to	Left-side value chinnadha illa equal-ah irukka-nu check pannum.

Examples

```
let x = 10;
let y = 5;
```

Logical Operators

1. Logical operators enbadhu logical operations perform panni, true illa false nu Boolean value-ah return pannum.

2. Logical operators mostly conditions base panni decisions edukka use aagum. Adhaavadhu, oru condition correct-ah irukka illaiya-nu check panni adhukku etha maadhiri code execute pannalaam.

3. Boolean values-ah handle pannavum, loops eppo stop aaganum-nu termination conditions set pannavum logical operators use pannalaam.

Logical Operators Table

Operator	Name	Explanation
&&	Logical AND	Rendu conditions-um true-ah irundha mattum true return pannum. Illana false return pannum.
||	Logical OR	Rendu conditions-la edhaavadhu onnu true-ah irundhaalum true return pannum. Rendum false-ah irundha mattum false return pannum.
!	Logical NOT	true-ah false-ah maathum; false-ah true-ah maathum.

Example

```
let x = true;
let y = false;
console.log(x && y); // false
console.log(x || y); // true
console.log(!x);     // false
```

Explanation:

- x && y — Rendu values-um true-ah irukkanum. Inga y false, adhanaala result false.
- x || y — Edhaavadhu oru value true-ah irundha podhum. Inga x true, adhanaala result true.
- !x — x-oda value true; ! operator adhai false-ah maathum.

Remember:

- && → Rendum true-ah irukkanum.
- || → Edhaavadhu onnu true-ah irundha podhum.
- ! → Boolean value-ah reverse pannum.

Bitwise Operators

Bitwise operators enbadhu numbers-oda binary representation (0s and 1s) mela operations perform panna use aagum.

Operator	Name	Usage	Explanation
&	Bitwise AND	a & b	Rendu binary bits-um 1-ah irundha mattum result 1 varum.
|	Bitwise OR	a | b	Rendu binary bits-la edhaavadhu onnu 1-ah irundha result 1 varum.

💡 Note: Bitwise operators everyday programming-la romba kammiya use aagum. Aana binary operations purinjukka idhu useful-ah irukkum.

Bitwise AND (&)

A & B-la rendu bits-um 1-ah irundha mattum output 1 varum.

A	B	Output (A & B)
0	0	0
0	1	0
1	0	0
1	1	1

Example:

```
console.log(5 & 3); // Output: 1
```

Explanation:

- 5 → Binary 101
- 3 → Binary 011
- AND operation → 001
- Output → 1

Bitwise OR (|)

A | B-la rendu bits-la edhaavadhu onnu 1-ah irundhaalum output 1 varum.

A	B	Output (A | B)
0	0	0
0	1	1
1	0	1
1	1	1

Example:

```
console.log(5 | 3); // Output: 7
```

Explanation:

- 5 → Binary 101
- 3 → Binary 011
- OR operation → 111
- Output → 7

String Operators –Explanation

1. JavaScript-la strings-ah modify pannavum, strings mela operations perform pannavum String Operators use aagum.

2. JavaScript-la strings-ah handle panna mukkiyamaana rendu operators irukku: + matrum +=.

3. Indha operators use panni oru string-oda innoru string-ah join pannalaam. Idhukku String Concatenation nu peru.

Example 1: Addition (+)

```
let firstName = "Dhamodharan";
let greeting = "Hello " + firstName;
console.log(greeting);
```

Output:

```
Hello Dh amodharan
Inga + operator rendu strings-ah join pannudhu.
```

Example 2: Addition Assignment (+=)

```
let text = "Hello";
text += " World";
console.log(text);
```

Output:

```
Hello World
```

```
Inga += operator, pudhu string-ah existing text variable-oda value-kooda join panni assign pannudhu.
```

Remember:

- + → Rendu strings-ah join pannum.
- += → Existing string-kooda innoru string-ah join panni variable-ku assign pannum.

Ternary Operator

Ternary operator enbadhu oru condition-ah check panni, andha condition-oda result base panni oru expression-ah execute panna use aagum.

Ternary operator test condition-ah evaluate pannum.

1. Condition true-ah irundha: expression1 execute aagum.
2. Condition false-ah irundha: expression2 execute aagum.

Ternary operator-la moonu operands irukkuradhunaala idhukku Ternary Operator nu peru. Idhai Conditional Operator nu kooda solluvaanga.

Syntax

```
condition ? expression1 : expression2;
```

- condition – Check panna vendiya condition.
- ? – Condition true-ah irundha enna execute pannanum-nu separate pannum.
- : – Condition false-ah irundha enna execute pannanum-nu separate pannum.

Example: Pass or Fail

```
// program to check pass or fail
let marks = prompt('Enter your marks :');
// check the condition
let result = (marks >= 40) ? 'pass' : 'fail';
console.log(`You ${result} the exam.`);
```

Explanation

- prompt('Enter your marks :') — User-kitta marks enter panna oru popup display pannum.
- marks >= 40 — Marks 40-ku equal-ah illa adhigama irukka-nu check pannum.
- ? 'pass' — Condition true-ah irundha pass assign aagum.
- : 'fail' — Condition false-ah irundha fail assign aagum.
- console.log() — Final result-ah console-la display pannum.

1. Increment / Decrement Operator

Increment (++) matrum Decrement (--) operators JavaScript-la oru variable-oda value-ah 1 increase illa 1 decrease panna use aagum.

- ++ → Value-ah 1 increase pannum.
- -- → Value-ah 1 decrease pannum.

++ Operator as Prefix and Postfix

1. Prefix (++var)

++ operator-ah prefix-la use pannumbodhu, variable-oda value first 1 increase aagum. Adhukku appuram andha new value return aagum.

2. Postfix (var++)

++ operator-ah postfix-la use pannumbodhu, variable-oda original value first return aagum. Adhukku appuram variable-oda value 1 increase aagum.

💡 -- operator-um idhe maadhiri dhaan work aagum. Aana value-ah 1 decrease pannum.

Example 1: Post-increment (x++)

```
let x = 10;
console.log(x++);
console.log(x);
```

Output:

```
10
11
```

Explanation:

- x++ — First original value 10 print aagum; appuram x value 11 aagum.
- Next console.log(x) — Updated value 11 print aagum.

Example 2: Post-decrement (x--)

```
let x = 10;
console.log(x--);
console.log(x);
```

Output:

```
10
9
```

Explanation:

- x-- — First original value 10 print aagum; appuram x value 9 aagum.
- Next console.log(x) — Updated value 9 print aagum.

Prefix Example

```
let x = 10;
console.log(++x);
console.log(x);
```

Output:

```
11
11
```

Inga ++x use pannirukkom. Adhanaala first value 11-ah increase aagi, appuram print aagum.

2. Comma Operator (,)

Comma operator multiple expressions-ah evaluate pannum. Aana last expression-oda value mattum return pannum.

Idhu loops-la multiple variables update pannumbodhu use aagum.

Example 1

```
let a = (1, 2, 3);
console.log(a);
```

Output:

```
3
```

Explanation:

- First 1 evaluate aagum.
- Next 2 evaluate aagum.
- Last 3 evaluate aagum.
- Last expression 3-oda value mattum a variable-ku assign aagum.

3. delete Operator

delete operator use panni, oru object-la irukkura specific property-ah remove pannalaam.

Property successfully delete aana true return aagum. Property delete panna mudiyalana false return aagalaam.

Example: Object

```
let obj = { x: 42, y: 99 };
console.log(obj);
delete obj.x;
console.log(obj);
```

Output:

```
{ x: 42, y: 99 }
```

{ y: 99 }

Explanation:

- obj object-la x matrum y properties irukku.
- delete obj.x — x property-ah remove pannum.
- Adhukku appuram y property mattum object-la irukkum.

4. typeof Operator

typeof operator use panni, oru value illa operand-oda data type enna-nu kandupidikkalaam. Idhu data type-ah oru string format-la return pannum.

Important Note: typeof null result object-nu varum. Idhu JavaScript-oda historical quirk; null unmaiyila primitive value dhaan.

5. void Operator

void operator oru expression-ah evaluate pannum. Aana andha expression-oda result-ah return pannaama, undefined return pannum.

Example

```
let result = void(0);
console.log(result);
```

Output:

```
undefined
```

Explanation:

- void(0) — 0 expression evaluate aagum.
- Aana void operator result-ah undefined-ah maathum.
- Adhanaala result variable-la undefined store aagum.

HTML Example

```
<a href="javascript:void(0)">Click me</a>
```

Indha example-la javascript:void(0) use pannumbodhu, link click pannina void(0) expression undefined return pannum. Idhu oru traditional technique; modern websites-la link-ku badhila button use pannradhu pala situations-la better.

6. in Operator –

in operator use panni, oru object-la specific property irukka illaiya-nu check pannalaam. Object-oda prototype chain-layum property irundhaal true return pannum.

Example 1: Object

```
let obj = { name: "Alice", age: 25 };
console.log("name" in obj);
console.log("gender" in obj);
```

Output:

```
true
false
```

Explanation:

- "name" in obj — name property object-la irukkuradhunaala true.
- "gender" in obj — gender property object-layum prototype chain-layum illadhadhunaala false.

> **Practical Task 1:** Create a calculator that performs addition, subtraction, multiplication, and division.

> **Practical Task 2:** Create a program to calculate the total price after applying a discount.

CHAPTER 06

Windows Methods

WINDOW OBJECT · ALERT · CONFIRM · PROMPT · OPEN · CLOSE · INPUT & OUTPUT

Window Object

1. window object ella major web browsers-layum support aagum. Idhu browser-oda window-ah represent pannum.

2. Global scope-la declare panra JavaScript objects, functions matrum var variables browser-la window object-oda members-ah aagum.

3. Global scope-la declare panna var variables, browser-la window object-oda properties-ah irukkum.

4. Global scope-la declare panna functions, browser-la window object-oda methods-ah irukkum.

Window Object Methods

Method	Explanation
alert()	Message-oda oru alert box-ah, OK button-oda display pannum.
confirm()	Message-oda confirm dialog box-ah OK matrum Cancel buttons-oda display pannum.
prompt()	User-kitta input vaanga oru dialog box-ah display pannum.
open()	Pudhu browser window illa tab-ah open panna use aagum.
close()	Current window-ah close panna try pannum; browser restrictions irukkalaam.

alert()

1 JavaScript-la already built-in-ah irukkura useful function dhaan alert().

2. alert() function use pannina, screen-la oru popup dialog box display aagum. Adhula namma kudukkura message show aagum.

3. Indha statement-la showAlert() function-ah first call pannanum-nu sollirukku. Aana idhu ella situations-layum correct illa. Direct-ah alert() function-ah call pannalaam.

Example:

```
alert("Hello Dhamodharan!");
```

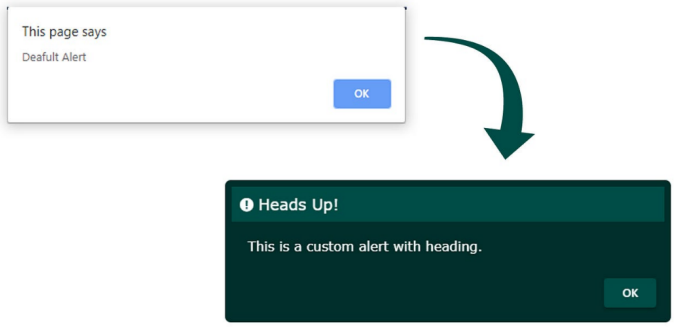
Indha code execute aagumbodhu, Hello Dhamodharan! message-oda popup box display aagum.

showAlert() use panni call panna example:

```
function showAlert() {
    alert("Welcome to JavaScript!");
}
showAlert();
```

Explanation:

- function showAlert() — showAlert-nu oru function create pannrom.
- alert() — Popup box-la message display pannum.



showAlert() — Function-ah call pannrom; appodhu function-kulla irukkura code execute aagum.

confirm()

1. confirm() method use pannina, screen-la oru dialog box display aagum. Andha box-la oru message-um, OK button-um, Cancel button-um irukkum.

2. User OK button click pannina true return aagum. Cancel button click pannina false return aagum.

Example

```
<!DOCTYPE html>
<html>
<body>
<h1>The Window Object</h1>
<h2>The confirm() Method</h2>
<p>Click the button to display a confirm box.</p>
<script>
    confirm("Press a button!");
</script>
</body>
</html>
```

Explanation

- confirm("Press a button!") — Browser-la confirm dialog box display pannum.
- User OK click pannina — true return aagum.
- User Cancel click pannina — false return aagum.

JavaScript prompt() Method

1. prompt() na enna?

prompt() enbadhu JavaScript-la irukkura oru built-in function. Idhu user-kitta text input vaanga oru dialog box-a display pannum.

Example: User name, age, city pondra details-a user-kitta ketka use pannalaam.

2. Eppadi work aagum?

- Web page open aagumbodhu prompt() dialog box display aagum.
- User text box-la value enter pannalaam.
- OK click pannina, enter panna value return aagum.
- Cancel click pannina, null return aagum.
- User OK illa Cancel click panra varaikkum JavaScript execution wait pannum.

3. Example

```
<!DOCTYPE html>
<html>
<body>
<h1>The Window Object</h1>
<h2>The prompt() Method</h2>
<p>Click the button to demonstrate the prompt box.</p>
<script>
  let user = prompt("Enter the User name:");
  console.log(user);
</script>
</body>
</html>
```

5. Output Example



Open()

1. Idhu pudhu window-la content-a display pannum.
2. open() method enbadhu JavaScript-la pudhu window-a open pannavum, already irukkura windows-a manipulate pannavum use aagura oru powerful method.
3. Idha use panni pudhu browser windows-a open pannalaam. Adhe maadhiri, already irukkura browser window-la pudhu tabs-aiyum open pannalaam.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>The Window Object</h1>
<h2>The open() Method</h2>
<p>Pudhu browser window-a open panna button-a click pannunga.</p>
<button onclick="myFunction()">Try it</button>
<script>
function myFunction() {
  window.open("https://www.w3schools.com");
}
</script>
</body>
</html>
```

Close()

1. close() method enbadhu current window-ai alladhu indha method call pannappatta window-ai close pannum.
2. Indha method-a use panni, JavaScript script moolama window.open() use seidhu open panna windows-ai mattum close panna mudiyum. Browser-la user manually open panna ella windows-aiyum close panna mudiyadhu.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>The Window Object</h1>
<h2>The open() and close() Methods</h2>
<button onclick="openWin()">Open "myWindow"</button>
<button onclick="closeWin()">Close "myWindow"</button>
<script>
let myWindow;
function openWin() {
    myWindow = window.open("", "", "width=200,height=100");
}
function closeWin() {
    myWindow.close();
}
</script>
</body>
</html>
```

> **Practical Task 1:** Display a welcome message using the alert() method.

> **Practical Task 2:** Use confirm() to ask users whether they want to continue and display the result.

Input and Output Statement

Input Statement

Input statement enbadhu, keyboard-la irundhu user oru response kudukkura varaikkum program execution-ah temporary-ah stop pannum. User kudukkura response-ah oru specific variable-la store pannum.

1. Idhu rendu vagaiyaana input instructions provide pannum:
2. Text Input: Text data-va read pannuradhukku.
3. Numeric Input: Number data-va read pannuradhukku.
4. Text input statement-oda general format:

```
variable = prompt(message, defaultValue);
```

Idhula:

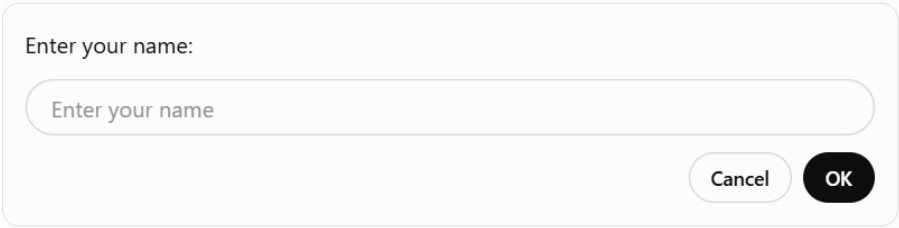
1. variable – User kudukkura input-ah store panna use panra variable.
2. prompt() – User-kitta input vaanguradhukku use pannuvom.
3. message – User-kku display aagura message.
4. defaultValue – Input box-la default-ah irukkura value.
5. Indha statement-la variable enbadhu, program-la already declare pannirukkura actual variable-ah irukkum. Indha variable-la text value store pannuvom.

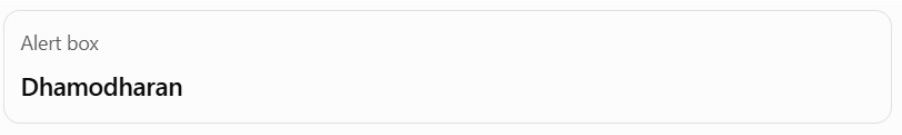
Input Value from the User Using prompt()

1. JavaScript-la user-kitta input vaanguradhukku prompt() method-ah use pannalaam.
2. Indha method-la user-kku display aaganum-nu ninaikkura message-ah kudukkalaam.
3. User oru value type panni OK button-ah click pannumbodhu, andha input value return aagum.
4. Indha value-ah direct-ah oru variable-la store pannalaam. Illana, variable-la store pannuradhukku munnadi thevaiyaana data type-kku convert pannalaam.

Example

```
let name = prompt("Enter your name:");
alert(name);
```



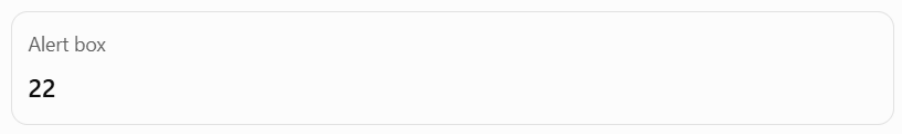


Idhu eppadi work aagum?

- prompt() user-kitta name-ah enter panna sollum.
- User enter panna name name variable-la store aagum.
- alert(name) user enter panna name-ah display pannum.

Important: prompt() moolama vaangura input default-ah String (text)-ah irukkum. Number-ah use pannanumna Number() use panni convert pannalaam.

```
let age = Number(prompt("Enter your age:"));
alert(age);
```



Add Two Float Numbers

Indha JavaScript program-la rendu decimal numbers-ah add panni, result-ah rounding pannama eppadi display panrathu, rounding panni eppadi display panrathu nu paakalaam.

Code Explanation

```
var num1 = 4.12;
var num2 = 9.87;
var result = 0;
// Adding float values
result = parseFloat(num1) + parseFloat(num2);
// Without rounding the result
console.log("Without rounding: " + result);
// Rounding the result to 2 digits after decimal
result = result.toFixed(2);
console.log("With rounding: " + result);
```

Line-by-line Explanation

```
var num1 = 4.12;
var num2 = 9.87;
var result = 0;
result = parseFloat(num1) + parseFloat(num2);
console.log("Without rounding: " + result);
result = result.toFixed(2);
console.log("With rounding: " + result);
```

Line-by-line explanation:

1. num1 = 4.12, num2 = 9.87 — Rendu float numbers-ah store pannrom.
2. result = 0 — Result store panna variable create pannrom.
3. parseFloat(num1) + parseFloat(num2) — Rendu values-aiyum float numbers-ah convert panni add pannum. Inga already numbers-ah irukkurathala parseFloat() thevai illa.
4. console.log() — Rounding pannama result-ah display pannum. JavaScript floating-point calculation nala 13.989999999999998 nu varalaam.
5. result.toFixed(2) — Decimal point-ku apram 2 digits mattum display pannum. Result 13.99 nu varum.

Output:

```
Without rounding: 13.989999999999998
With rounding: 13.99
```

Important: toFixed(2) result-ah String type-la return pannum. Decimal point-ku apram 2 digits venumna idha use pannalaam.

> **Practical Task 1:** Get a user's name using prompt() and display a personalized greeting.

> **Practical Task 2:** Get two numbers from the user and display their sum.

CHAPTER 07

Date Object

DATE OBJECT · CONSTRUCTOR · GETTERS · HUMAN-READABLE FORMAT · QUICK REVISION

JavaScript Date Object

1. JavaScript-la Date Object enbathu oru particular time-ah represent pannum. Ithu platform-independent format-la irukkum.
2. Date object current date and time-ah represent panna use aagum.
3. Date object-la January 1, 1970, UTC midnight-la irunthu calculate panna milliseconds value store aagum. Intha starting point-ku Epoch nu peru.
4. Note: TC39 team, JavaScript-la date and time handle pannurathukku Temporal nu oru pudhu API-ah develop pannitu irukanga.
5. Date object use panni date and time related values-ah get pannalaam, set pannalaam. Example: Day, Month, Year, Hour, Minute and Seconds.

Constructor

Date object create pannurathukku 4 types of Date constructors use pannalaam.

1. Date() – Current date and time-ah return pannum.
2. Date(milliseconds) – January 1, 1970-lirunthu kudukkappatta milliseconds-ah base panni date create pannum.
3. Date(dateString) – Date string-ah base panni date create pannum.
4. Date(year, month, day, hours, minutes, seconds, milliseconds) – Namakku thevaiyana year, month, day and time values kuduthu date create pannalaam.

Getting Full Year

```
const now = new Date();
console.log(now.getFullYear()); // 2020
```

Explanation:

- new Date() – Current date and time-ah get pannum.
- getFullYear() – Current year-ah return pannum.

Output: 2020

Getting Month

```
const now = new Date();
console.log(now.getMonth()); // 0
```

Explanation:

- getMonth() – Current month-ah number format-la return pannum.
- Month counting 0-la irunthu start aagum.
- January = 0, February = 1 nu December = 11 varaikkum irukkum.

Note: Month values 0–11 range-la irukkum.

Getting Date

```
const now = new Date();
console.log(now.getDate()); // 4
```

Explanation:

- getDate() – Current month-la innaikku enna date-nu return pannum.
- Date values 1–31 varaikkum irukkum.

Example: Innaikku 4th date-na output 4 varum.

Getting Day

```
const now = new Date();
console.log(now.getDay()); // 6
```

Explanation:

- getDay() – Week-la innaikku enna day-nu number format-la return pannum.
- Sunday = 0
- Monday = 1
- Tuesday = 2
- Wednesday = 3
- Thursday = 4
- Friday = 5
- Saturday = 6

Note: Weekday values 0–6 range-la irukkum.

Getting Hours

```
const now = new Date();
console.log(now.getHours()); // 0
```

Explanation:

- getHours() – Current hour-ah return pannum.
- Hours values 0–23 range-la irukkum.
- Example: Time 00:56:41 na output 0 varum.

Getting Minutes

```
const now = new Date();
console.log(now.getMinutes()); // 56
```

Explanation:

- getMinutes() – Current minutes-ah return pannum.
- Minutes values 0–59 range-la irukkum.
- Example: Time 00:56:41 na output 56 varum.

Getting Seconds

```
const now = new Date();
console.log(now.getSeconds()); // 41
```

Explanation:

- getSeconds() – Current seconds-ah return pannum.
- Seconds values 0–59 range-la irukkum.
- Example: Time 00:56:41 na output 41 varum.

Getting Time

Intha method January 1, 1970-lirunthu current time varaikkum evvalavu milliseconds pass aagirukku nu return pannum. Ithu Unix Timestamp-nu sollappadum.

getTime() use panni intha milliseconds value-ah get pannalaam.

1. Using getTime()

```
const now = new Date();
console.log(now.getTime());
// 1578092201341
```

Explanation:

- getTime() – January 1, 1970-lirunthu current date and time varaikkum pass aana total milliseconds-ah return pannum.
- Example output 1578092201341 enbathu January 4, 2020, 00:56:41 varaikkum pass aana milliseconds.

Note: getTime() milliseconds-la value return pannum, seconds-la illa.

Date and Time-ah Human Readable Format-la Display Pannurathu

Date object methods use panni date and time values-ah separate-ah get panni, namakku puriyura format-la display pannalaam.

Example:

```
const now = new Date();
const year = now.getFullYear();  // Year return pannum
const month = now.getMonth() + 1; // Month (0-11)
const date = now.getDate();      // Date (1-31)
const hours = now.getHours();    // Hours (0-23)
const minutes = now.getMinutes();// Minutes (0-59)
console.log(`${date}/${month}/${year} ${hours}:${minutes}`);
// Example Output: 4/1/2020 0:56
```

Explanation:

- getFullYear() – Current year-ah get pannum.
- getMonth() + 1 – Current month-ah get pannum. Month 0-la start aagurathala +1 add pannirukkom.
- getDate() – Current date-ah get pannum.
- getHours() – Current hour-ah get pannum.
- getMinutes() – Current minutes-ah get pannum.
- ${} – Template literal use panni variable values-ah string-kulla insert pannalaam.

Output:

```
4/1/2020 0:56
```

Important Date Methods – Quick Revision

Method	Use
getFullYear()	Year get panna
getMonth()	Month get panna (0–11)
getDate()	Date get panna (1–31)
getDay()	Weekday get panna (0–6)
getHours()	Hours get panna (0–23)
getMinutes()	Minutes get panna (0–59)
getSeconds()	Seconds get panna (0–59)
getTime()	Milliseconds get panna

Important Difference:

- getDate() → Month-la irukkura date-ah return pannum.
- getDay() → Week-la irukkura day-ah return pannum.

Example: Innaikku Saturday, date 31-na getDate() output 31; getDay() output 6.

> **Practical Task 1:** Display the current date and time on a webpage.

> **Practical Task 2:** Create an age calculator using the user's date of birth.

CHAPTER 08

Array

CREATE · ACCESS · ADD / CHANGE / REMOVE · LENGTH · METHODS · SLICE VS SPLICE

1. JavaScript-la Array enbathu data-va store pannurathukku use aagura oru global object type.
2. Array-la zero illa athukku mela values-ah oru ordered collection-ah store pannalaam. Different data types-um store pannalaam. Ovvoru element-ah access pannurathukku 0-la irunthu start aagura numbered index use pannuvom.
3. Idhu primitive data type-ah dhaan irukkanumnu avasiyam illa. Endha oru object-ah irundhaalum store panni vachikkalaam.
4. Endha data type-ah irundhaalum ore array-la store pannikka mudiyum.

Example:

```
const words = ['hello', 'world', 'welcome'];
```

Inga words enbathu oru array. Intha array-la motham 3 values store aagirukku.

Create an Array in Two Ways

1. Using an Array Literal

Array create pannurathukku easiest method [] (square brackets) use pannurathu.

Example:

```
const cars = ["Saab", "Volvo", "BMW"];
```

Spaces and line breaks mukkiyam illa. Array declaration-ah multiple lines-layum write pannalaam.

```
const cars = [
  "Saab",
  "Volvo",
  "BMW"
];
```

Rendu examples-um same output-thaan kudukkum.

2. Using the JavaScript Keyword new

new Array() use panniyum array create pannalaam.

```
const cars = new Array("Saab", "Volvo", "BMW");
```

Access Elements of an Array

Array-la irukkura elements-ah access pannurathukku Index (0, 1, 2...) use pannuvom.

Example:

```
const myArray = ['h', 'e', 'l', 'l', 'o'];
// First element
console.log(myArray[0]); // "h"
// Second element
console.log(myArray[1]); // "e"
```

Explanation:

- myArray[0] – First element-ah access pannum.
- myArray[1] – Second element-ah access pannum.

Output:

```
h
e
```



Array indexing in JavaScript

Important Note: Array-oda index 1-la start aagaathu. Eppothum 0-la irunthuthaan start aagum.

Add an Element to an Array

Array-la pudhu elements add pannurathukku JavaScript-la built-in methods push() and unshift() use pannalaam.

Push()

1. push() method array-oda last-la pudhu elements-ah add pannum.
2. push() method original array-oda length-ah change pannum.
3. push() method updated array-oda new length-ah return pannum.

Example:

```
let dailyActivities = ['eat', 'sleep'];
// Add an element at the end
dailyActivities.push('exercise');
console.log(dailyActivities);
```

Explanation:

- dailyActivities.push('exercise') – Array-oda last-la exercise add pannum.

Output:

```
['eat', 'sleep', 'exercise']
```

Unshift()

1. unshift() method array-oda starting-la pudhu element-ah add pannum.

Example:

```
let dailyActivities = ['eat', 'sleep'];
// Add an element at the start
dailyActivities.unshift('work');
console.log(dailyActivities);
```

Explanation:

- unshift('work') – Array-oda first-la work add pannum.

Output:

```
['work', 'eat', 'sleep']
```

Change the Elements of an Array

Array-oda index value-ah access panni, already irukkura element-ah change pannalaam. Pudhu element-ah add pannavum pannalaam.

Example:

```
let dailyActivities = ['eat', 'sleep'];
// Index 2-la new element add pannum
dailyActivities[2] = 'exercise';
console.log(dailyActivities);
```

Explanation:

- dailyActivities[2] – Array-la index 2-la irukkura element-ah target pannum.
- Antha index-la element illathathala exercise nu pudhu value add aagum.

Output:

```
['eat', 'sleep', 'exercise']
```

Index 3-la element add panna enna aagum?

Oru array-la already 2 elements irukku. Ippo index 3-la pudhu element add panna, index 2 empty-ah irukkum.

Example:

```
let dailyActivities = ['eat', 'sleep'];
// Index 3-la new element add pannum
dailyActivities[3] = 'exercise';
console.log(dailyActivities);
```

Output:

```
['eat', 'sleep', <1 empty item>, 'exercise']
```

Explanation:

- Index 0 – eat
- Index 1 – sleep
- Index 2 – Empty
- Index 3 – exercise

Basically, Array-la irukkura elements-ah vida high index-la pudhu element add panna, idaiyila irukkura indexes empty-ah irukkum. Ithu undefined value maathiri behave aagum; console-la <1 empty item> nu display aagalaam.

Remove an Element from an Array

Pop()

1. pop() method array-oda last element-ah remove pannum.
2. pop() method remove panna element-ah return pannum.

Example:

```
let dailyActivities = ['work', 'eat', 'sleep', 'exercise'];
// Remove the last element
dailyActivities.pop();
console.log(dailyActivities);
// Remove the last element again
const removedElement = dailyActivities.pop();
// Get removed element
console.log(removedElement);
console.log(dailyActivities);
```

Explanation:

- First pop() – exercise remove aagum.
- Second pop() – sleep remove aagum.
- removedElement variable-la remove panna sleep value store aagum.

Output:

```
['work', 'eat', 'sleep']
```

sleep

['work', 'eat']

Shift()

First element-ah remove pannanumna shift() method use pannalaam.

shift() method array-oda first element-ah remove pannum. Remove panna element-ah return-um pannum.

Example:

```
let dailyActivities = ['work', 'eat', 'sleep'];
// Remove the first element
dailyActivities.shift();
console.log(dailyActivities);
```

Explanation:

- shift() – Array-oda first element work-ah remove pannum.

Output:

```
['eat', 'sleep']
```

Array Length

Array-la motham ethana elements irukku nu therinjukka length property use pannuvom.

Example:

```
const dailyActivities = ['eat', 'sleep'];
// Get the total number of elements
console.log(dailyActivities.length);
```

Explanation:

- dailyActivities.length – Array-la irukkura total elements count-ah return pannum.

Output:

```
2
```

Array Methods

JavaScript-la array values-ah add, remove, search, sort, copy and modify pannurathukku pala built-in array methods irukku.

Important Array Methods – Description

Method	Explanation
concat()	Rendu illa athukku mela arrays-ah join panni new array return pannum.
indexOf()	Oru element array-la entha index-la irukku nu return pannum.
find()	Condition satisfy aagura first element-ah return pannum.
findIndex()	Condition satisfy aagura first element-oda index-ah return pannum.
forEach()	Array-la irukkura ovvoru element-kum function-ah execute pannum.
includes()	Array-la particular element irukka illaiya nu check pannum.
push()	Array-oda last-la new element add panni, new length-ah return pannum.
unshift()	Array-oda beginning-la new element add panni, new length-ah return pannum.
pop()	Last element-ah remove panni, removed element-ah return pannum.
shift()	First element-ah remove panni, removed element-ah return pannum.
sort()	Array elements-ah sort pannum.
slice()	Array-la oru particular portion-ah select panni new array return pannum. Original array-ah change pannaathu.
splice()	Existing elements-ah remove or replace pannalaam; pudhu elements-ah add pannavum pannalaam. Original array-ah change pannum.

1. concat()

concat() method rendu arrays-ah join panni oru new array-ah return pannum.

```
let arr1 = [1, 2];
let arr2 = [3, 4];
let result = arr1.concat(arr2);
console.log(result);
```

Output:

```
[1, 2, 3, 4]
```

Explanation: arr1 and arr2 rendu arrays-um join aagi result array create aagum.

2. indexOf()

indexOf() method oru particular element array-la entha index-la irukku nu return pannum.

```
let fruits = ["apple", "banana", "cherry"];
console.log(fruits.indexOf("banana"));
```

Output:

```
1
```

Explanation: banana array-la index 1-la irukkurathala output 1 varum.

Oru element array-la illana -1 return pannum.

3. find()

find() method kudukkappatta condition satisfy aagura first element-ah return pannum.

```
let numbers = [5, 12, 8, 130, 44];
let found = numbers.find(num => num > 10);
console.log(found);
```

Output:

```
12
```

Explanation:

- num => num > 10 – Number 10-vida perusa irukka nu check pannum.
- 5 condition satisfy aagala.
- 12 condition satisfy aaguthu.
- So, 12 return aagum. Athukku appuram remaining elements check pannaathu.

4. findIndex()

findIndex() method condition satisfy aagura first element-oda index-ah return pannum.

```
let numbers = [5, 12, 8, 130, 44];
let index = numbers.findIndex(num => num > 10);
console.log(index);
```

Output:

```
1
```

Explanation: 10-vida perusa irukkura first number 12. Athu index 1-la irukkurathala output 1 varum.

5. forEach()

forEach() method array-la irukkura ovvoru element-kum oru function-ah execute pannum.

```
let colors = ["red", "green", "blue"];
colors.forEach(color => console.log(color));
```

Output:

```
red
green
blue
```

Explanation: Array-la irukkura ovvoru color-um one by one console.log() moolama display aagum.

6. includes()

includes() method array-la particular element irukka illaiya nu check pannum.

```
let animals = ["dog", "cat", "lion"];
console.log(animals.includes("cat"));
console.log(animals.includes("tiger"));
```

Output:

```
true
false
```

Explanation:

- includes("cat") – cat array-la irukkurathala true return pannum.
- includes("tiger") – tiger array-la illathathala false return pannum.

7. sort()

sort() method array-la irukkura elements-ah sort pannum. Strings-ah alphabetical order-la arrange pannum.

```
let fruits = ["banana", "apple", "cherry"];
fruits.sort();
console.log(fruits);
```

Output:

```
["apple", "banana", "cherry"]
```

Explanation: Fruits alphabetical order-la arrange aagum.

8. slice()

slice() method array-la irunthu particular portion-ah select panni new array-ah return pannum. Original array change aagaathu.

```
let arr = [1, 2, 3, 4, 5];
let sliced = arr.slice(1, 3);
console.log(sliced);
```

Output:

```
[2, 3]
```

Explanation:

- slice(1, 3) – Index 1-la irunthu start aagum.
- Index 3-ku munnaadi stop aagum.
- So, index 1 and 2 values mattum return aagum.

Important: Ending index include aagaathu.

9. splice()

splice() method use panni array-la irukkura existing elements-ah remove pannalaam, replace pannalaam, pudhu elements-ah add pannalaam.

```
let arr = [1, 2, 3, 4, 5];
arr.splice(2, 2, 99, 100);
console.log(arr);
```

Output:

```
[1, 2, 99, 100, 5]
```

Explanation:

```
splice(2, 2, 99, 100)
```

- 2 – Index 2-la irunthu operation start aagum.
- 2 – Rendu elements remove aagum (3, 4).
- 99, 100 – Antha edathula pudhu values add aagum.

Important Difference: slice() vs splice()

slice()	splice()
Array-la irunthu portion-ah select pannum.	Elements-ah add, remove, replace pannum.
New array return pannum.	Removed elements-ah array format-la return pannum.
Original array change aagaathu.	Original array change aagum.

Note: push(), pop(), shift(), unshift() and splice() original array-ah modify pannum. slice() and concat() original array-ah modify pannaathu.

CHAPTER 09

Array Destructuring

BASIC · SKIP VALUES · DEFAULT VALUES · REST OPERATOR · IMPORTANT POINTS

Array Destructuring na array-la irukkura values-ah separate variables-ku easy-ah assign pannurathu.

Example:

```
let fruits = ["Apple", "Mango", "Orange"];
let [a, b, c] = fruits;
console.log(a);
console.log(b);
console.log(c);
```

Output:

```
Apple
Mango
Orange
```

Explanation:

- a → Apple
- b → Mango
- c → Orange

2. Skip Values

Thevai illadha values-ah comma use panni skip pannalam.

```
let numbers = [10, 20, 30];
let [a, , c] = numbers;
console.log(a);
console.log(c);
```

Output:

```
10
30
```

3. Default Values

Array-la value illana default value assign pannalam.

```
let fruits = ["Apple"];
let [a, b = "Mango"] = fruits;
console.log(a);
console.log(b);
```

Output:

```
Apple
Mango
```

4. Rest Operator (...)

Remaining values-ah oru new array-la store pannalam.

```
let numbers = [10, 20, 30, 40];
let [a, ...rest] = numbers;
console.log(a);
console.log(rest);
```

Output:

```
10
[20, 30, 40]
```

Important Points

1. Array destructuring ES6 feature.
2. Values position based-ah assign aagum.
3. Values skip panna comma use pannalam.
4. Default value set pannalam.
5. ... rest operator remaining values-ah collect pannum.

Arrays

Practical Task 1: Create an array of five subject names. Display its length and access individual elements using their indexes.

Practical Task 2: Create a shopping list that allows the user to add, update and remove items using push(), unshift(), pop() and shift().

Array Methods

> **Practical Task 1:** Given an array of numbers, use map() to create a new array containing the square of every number.

> **Practical Task 2:** Given an array of student marks, use filter() to find passing students and reduce() to calculate the total and average marks.

Array Destructuring

> **Practical Task 1:** Create an array containing a student's name, age and department. Extract the values into separate variables using array destructuring.

> **Practical Task 2:** Use array destructuring to skip selected values, assign default values and collect remaining values with the rest operator.

CHAPTER 10

Classes

CLASS · CONSTRUCTOR · METHODS · INHERITANCE · STATIC METHODS

JavaScript-la Class na objects create panna use panra oru blueprint (template) madhiri.

Simple-ah sollanumna, oru class use panni multiple objects create pannalam.

Real-time Example:

Oru Student class create panrom. Athula:

- Student name
- Age
- Department
- Student details display panra method

Indha class-ah use panni multiple students create pannalam.

1. What is a Class in JavaScript?

1. Class is a blueprint for creating objects.
2. Class-la properties and methods define pannalam.
3. class keyword use panni class create pannuvom.
4. Class-la irundhu objects create panna new keyword use pannuvom.

Syntax:

```
class ClassName {
    // Properties
    // Methods
}
```

2. Simple Class Example

```
class Student {
    name = "Dhamodharan";
    age = 22;
    display() {
        console.log(this.name);
        console.log(this.age);
    }
}
let s1 = new Student();
s1.display();
```

Screen Output:

```
Dhamodharan
22
```

Explanation:

- class Student – Student nu oru class create panrom.
- name – Property.
- age – Property.
- display() – Method.
- new Student() – Student class-la irundhu object create panrom.
- s1.display() – Display method-ah call panrom.
- this – Current object-ah refer pannum.

3. Constructor in JavaScript

Constructor na class-la object create aagumbodhu automatically execute aagura special method.

- constructor() nu define pannanum.
- Object create pannumbodhu automatically call aagum.
- Object-ku initial values assign panna use aagum.

Example:

```
class Student {
    constructor() {
        console.log("Student Created");
    }
}
let s1 = new Student();
```

Screen Output:

```
Student Created
```

Inga new Student() execute aagumbodhu constructor automatically call aagum.

4. Constructor with Parameters

Constructor-kulla parameters pass panni object-ku different values assign pannalam.

Example:

```
class Student {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    display() {
        console.log("Name:", this.name);
        console.log("Age:", this.age);
    }
}
let s1 = new Student("Dhamodharan", 22);
let s2 = new Student("Arun", 21);
s1.display();
s2.display();
```

Screen Output:

```
Name: Dhamodharan
Age: 22
Name: Arun
Age: 21
```

Explanation:

- name, age – Constructor parameters.
- this.name = name – Object-oda name property-ku value assign panrom.
- this.age = age – Object-oda age property-ku value assign panrom.
- s1, s2 – Rendu different objects.

5. Methods in JavaScript Class

Class-kulla create panra functions-ah methods nu solluvom.

Example:

```
class Calculator {
    add(a, b) {
        console.log(a + b);
    }
    sub(a, b) {
        console.log(a - b);
    }
    mul(a, b) {
        console.log(a * b);
    }
}
let c1 = new Calculator();
c1.add(10, 5);
c1.sub(10, 5);
c1.mul(10, 5);
```

Screen Output:

```
15
5
50
Inga add(), sub(), mul() ellame methods.
```

6. Multiple Objects in Class

Oru class use panni multiple objects create pannalam.

Example:

```
class Car {
    constructor(brand, price) {
        this.brand = brand;
        this.price = price;
    }
    display() {
        console.log(this.brand, this.price);
    }
}
let car1 = new Car("BMW", 5000000);
let car2 = new Car("Audi", 4000000);
let car3 = new Car("Benz", 6000000);
car1.display();
car2.display();
car3.display();
```

Screen Output:

```
BMW 5000000
Audi 4000000
Benz 6000000
```

One class-la irundhu three different objects create pannirukkom.

7. Inheritance in JavaScript

Inheritance na oru class-oda properties and methods-ah another class use panradhu.

- extends keyword use pannuvom.
- Parent class-la irukkura properties and methods child class-ku available aagum.
- super() use panni parent class constructor-ah call pannalam.

Example:

```
class Animal {
    eat() {
        console.log("Animal is eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Dog is barking");
    }
}
let d1 = new Dog();
d1.eat();
d1.bark();
```

Screen Output:

```
Animal is eating
Dog is barking
```

Explanation:

- Animal – Parent class.
- Dog – Child class.
- extends Animal – Animal class-ah inherit panrom.
- d1.eat() – Parent class method.
- d1.bark() – Child class method.

8. Static Methods in JavaScript

Static method na object create pannama direct-ah class name use panni call panra method.

- static keyword use pannuvom.
- Static method-ah class name moolama call pannanum.
- Normal-ah object use panni static method call panna mudiyadhu.

Example:

```
class Demo {
    static show() {
        console.log("Welcome to JavaScript");
    }
}
Demo.show();
```

Screen Output:

```
Welcome to JavaScript
```

9. Important Class Keywords

Keyword	Purpose
class	Create a class
constructor	Initialize object values
new	Create an object
this	Refer to current object
extends	Inherit another class
super	Call parent constructor or methods
static	Create static methods

10. Real-Time Example – Employee Management

```
class Employee {
    constructor(name, role, salary) {
        this.name = name;
        this.role = role;
        this.salary = salary;
    }
    display() {
        console.log("Employee Name:", this.name);
        console.log("Role:", this.role);
        console.log("Salary:", this.salary);
    }
}
let emp1 = new Employee("Dhamodharan", "Java Developer", 25000);
emp1.display();
```

Screen Output:

```
Employee Name: Dhamodharan
Role: Java Developer
Salary: 25000
```

Practical Task 1: Create a Student class with a constructor that accepts name, roll number and department. Create and display two student objects.

Practical Task 2: Create an Employee class with a constructor and a method that calculates and displays an employee's annual salary.

CHAPTER 11

Object

DECLARATION · ACCESS · NESTED OBJECTS · METHODS · BUILT-IN METHODS

1. JavaScript-la Object enbathu pala properties-ah onna collect panni store pannura oru data structure.
2. Oru property enbathu oru name (illa key) and athukku assign pannirukkura value-oda combination-ah irukkum.
3. Oru property's value function-ah iruntha, antha property-ku Method nu peru.

Simple Example:

```
const person = {
    name: "John",
    age: 20
};
```

Inga:

- name – Key
- "John" – Value
- age – Key
- 20 – Value

💡 Note: Neenga already vera programming languages therinjirunthaalum, JavaScript objects konjam different-ah irukkum. JavaScript-la object create pannurathukku kandippa class create panna vendiya avasiyam illa.

JavaScript Object Declaration

Syntax:

```
const object_name = {
    key1: value1,
    key2: value2
};
```

Explanation:

Inga object_name nu oru object create pannirukkom. Ovvoru object member-um key: value pair-ah irukkum.

- Ovvoru key and value-kum naduvula : use pannuvom.
- Ovvoru key-value pair-kum naduvula comma , use pannuvom.
- Object values ellam curly braces {} kulla irukkum.

Example:

```
// Object creation
const person = {
    name: 'John',
    age: 20
};
console.log(typeof person); // object
```

Explanation:

Inga person enbathu oru object. Athula string (John) and number (20) maathiri different data types store pannirukkom.

Object-ah single line-layum create pannalaam.

```
const person = { name: 'John', age: 20 };
```

Intha example-la name and age keys; John and 20 athoda values.

Accessing Object Properties

Object-la irukkura property-oda value-ah access pannurathukku athoda key-ah use pannuvom.

1. Using Dot Notation

Syntax:

```
objectName.key
```

Example:

```
const person = {
    name: 'John',
    age: 20,
};
// Accessing property
console.log(person.name); // John
```

Explanation:

- person.name – person object-la irukkura name property-ah access pannum.
- Output-la John display aagum.

2. Using Bracket Notation

Syntax:

```
objectName["propertyName"]
```

Example:

```
const person = {
    name: 'John',
    age: 20,
};
// Accessing property
console.log(person["name"]); // John
```

Explanation:

- person["name"] – Bracket notation use panni name property-ah access pannirukkom.
- Output-la John display aagum.

JavaScript Nested Objects

Oru object-kulla innoru object-ah store pannalaam. Ippadi oru object-kulla innoru object iruntha athukku Nested Object nu peru.

Example:

```
// Nested object
const student = {
    name: 'John',
    age: 20,
    marks: {
        science: 70,
        math: 75
    }
};
// Accessing student object property
console.log(student.marks);
// { science: 70, math: 75 }
// Accessing marks object property
console.log(student.marks.science); // 70
```

Explanation:

- student – Main object.
- marks – student object-kulla irukkura nested object.
- science and math – marks object-oda properties.
- student.marks – Full marks object-ah access pannum.
- student.marks.science – Science mark-ah mattum access pannum.

JavaScript Object Methods

Oru object-oda property value-ah function-ah define pannina, athukku Object Method nu peru.

Example:

```
const person = {
    name: 'Sam',
    age: 30,
    // Using function as a value
    greet: function() {
        console.log('hello');
    }
};
person.greet(); // hello
```

Explanation:

- greet – Object-oda method name.
- function() – Function-ah define pannirukkom.
- person.greet() – Object method-ah call pannurathukku use pannuvom.
- Output-la hello display aagum.

Add Key/Value Pair to an Object

1. Using Dot Notation

Dot notation use panni object-kulla pudhu key-value pair add pannalaam.

Example:

```
// Program to add a key/value pair to an object
const person = {
    name: 'Monica',
    age: 22,
    gender: 'female'
};
// Add a key/value pair
person.height = 5.4;
console.log(person);
```

Explanation:

- person.height = 5.4 – person object-kulla height nu pudhu property create panni 5.4 value assign pannirukkom.

Output:

```
{
    name: "Monica",
    age: 22,
    gender: "female",
    height: 5.4
}
```

2. Using Square Bracket Notation

Square bracket notation use panniyum object-kulla pudhu key-value pair add pannalaam.

Example:

```
// Program to add a key/value pair to an object
const person = {
    name: 'Monica',
    age: 22,
    gender: 'female'
};
// Add a key/value pair
person['height'] = 5.4;
console.log(person);
```

Explanation:

- person['height'] = 5.4 – Bracket notation use panni height property-ah add pannirukkom.

Output:

```
{
    name: "Monica",
    age: 22,
    gender: "female",
    height: 5.4
}
```

Object Methods

JavaScript-la object-oda keys, values and key-value pairs-ah access pannurathukku sila important built-in methods irukku.

1. Object.keys()

1. Object.keys() method oru object-la irukkura ella property keys-ah array format-la return pannum.
2. Intha method original object-ah change pannaathu.

Syntax:

```
Object.keys(object);
```

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>The Object.keys() Method</h2>
<p>Object.keys() returns an array containing the keys of an object:</p>
<p id="demo"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
const keys = Object.keys(fruits);
document.getElementById("demo").innerHTML = keys;
</script>
</body>
</html>
```

Explanation:

- Object.keys(fruits) – fruits array-oda indexes-ah keys-ah return pannum.
- Array-la indexes 0, 1, 2, 3 nu irukkum.
- document.getElementById("demo").innerHTML – Result-ah HTML page-la display pannum.

Output:

```
0,1,2,3
```

Another Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>The Object.keys() Method</h2>
<p id="demo"></p>
<script>
const person = {
    firstName: "John",
    lastName: "Doe",
    age: 50,
    eyeColor: "blue"
};
const keys = Object.keys(person);
document.getElementById("demo").innerHTML = keys;
</script>
</body>
</html>
```

Explanation:

- Object.keys(person) – person object-la irukkura ella keys-ah array format-la return pannum.

Output:

```
firstName,lastName,age,eyeColor
```

2. Object.values()

Object.values() method object-la irukkura ella property values-ah oru array format-la return pannum.

Syntax:

```
Object.values(obj);
```

Example:

```
// Returning property values
let object = {
    0: '23',
    1: 'geeksforgeeks',
    2: 'true'
};
console.log(Object.values(object));
```

Explanation:

- Object.values(object) – Object-la irukkura values-ah mattum array format-la return pannum.
- Keys-ah return pannaathu.

Output:

```
["23", "geeksforgeeks", "true"]
```

3. Object.entries()

Object.entries() method object-la irukkura ovvoru key and value pair-ah oru separate array-ah return pannum. Ithu mothama oru array-kulla store aagum.

Syntax:

```
Object.entries(obj);
```

Example:

```
const obj = {
    name: "Adam",
    age: 20,
    location: "Nepal"
};
// Returns properties in key-value format
console.log(Object.entries(obj));
```

Explanation:

- Object.entries(obj) – Object-la irukkura ovvoru key and value pair-ah array format-la return pannum.
- Ovvoru pair-um [key, value] format-la irukkum.

Output:

```
[
    ["name", "Adam"],
    ["age", 20],
    ["location", "Nepal"]
]
```

4. Object.hasOwnProperty()

hasOwnProperty() method oru particular property antha object-kulla direct-ah irukka illaiya nu check pannum.

Property value null illa undefined-ah irunthaalum, antha property object-kulla iruntha true return pannum.

Syntax:

```
obj.hasOwnProperty(prop);
```

Example:

```
const obj = {};
obj.id = 42;
// Check if id is present in obj or not
console.log(obj.hasOwnProperty("id"));
```

Explanation:

- obj.id = 42 – obj object-kulla id property add pannirukkom.
- obj.hasOwnProperty("id") – id property object-kulla irukka nu check pannum.

Output:

```
true
```

Important Object Methods – Quick Revision

Method	Use
Object.keys()	Object-oda keys-ah array-ah return pannum.
Object.values()	Object-oda values-ah array-ah return pannum.
Object.entries()	Key-value pairs-ah nested array-ah return pannum.
hasOwnProperty()	Particular property object-kulla direct-ah irukka nu check pannum.

Easy-ah nyabagam vachukonga:

```
const person = {
    name: "John",
    age: 20
};
console.log(Object.keys(person));
// ["name", "age"]
console.log(Object.values(person));
// ["John", 20]
console.log(Object.entries(person));
// [["name", "John"], ["age", 20]]
console.log(person.hasOwnProperty("name"));
// true
```

Important Correction: Object.keys() method Array Iterator return pannaathu; keys irukkura normal array-ah return pannum.

CHAPTER 12

Object Destructuring in JavaScript

BASIC · RENAME · DEFAULT VALUES · NESTED · REST · FUNCTIONS

Object Destructuring na object-la irukkura properties-ah separate variables-ku easy-ah extract pannurathu.

1. Basic Example

```
let person = {
    name: "Arun",
    age: 22,
    city: "Chennai"
};
```

let { name, age, city } = person;

```
console.log(name);
console.log(age);
console.log(city);
```

Output:

```
Arun
22
Chennai
```

Explanation:

- name → Arun
- age → 22
- city → Chennai

2. Rename Variables

Property name-ah different variable name-ku change pannalam.

```
let person = {
    name: "Arun",
    age: 22
};
```

let { name: userName, age: userAge } = person;

```
console.log(userName);
console.log(userAge);
```

Output:

```
Arun
22
```

3. Default Values

Object-la property illana default value kudukkalam.

```
let person = {
    name: "Arun"
};
```

let { name, city = "Chennai" } = person;

```
console.log(name);
console.log(city);
```

Output:

```
Arun
Chennai
```

4. Nested Object Destructuring

Object-kulla innoru object irundha, adhulayum destructuring pannalam.

```
let student = {
    name: "Kumar",
    marks: {
        tamil: 80,
        english: 90
    }
};
```

let { name, marks: { tamil, english } } = student;

```
console.log(name);
console.log(tamil);
console.log(english);
```

Output:

```
Kumar
80
90
```

5. Rest Operator (...)

Thevaiyana property eduthuttu remaining properties-ah separate object-la store pannalam.

```
let person = {
    name: "Arun",
    age: 22,
    city: "Chennai",
    course: "Java"
};
```

let { name, ...rest } = person;

```
console.log(name);
console.log(rest);
```

Output:

```
Arun
```

```
{
    age: 22,
    city: "Chennai",
    course: "Java"
}
```

6. Function-la Object Destructuring

Function parameter-laye destructuring use pannalam.

```
let student = {
    name: "Raja",
    age: 21
};
function display({ name, age }) {
    console.log(name);
    console.log(age);
}
display(student);
```

Output:

```
Raja
21
```

Array vs Object Destructuring

Array Destructuring	Object Destructuring
Position based	Property name based
let [a, b] = arr	let {name, age} = obj
Order important	Property order important illa

Easy-ah remember pannunga:

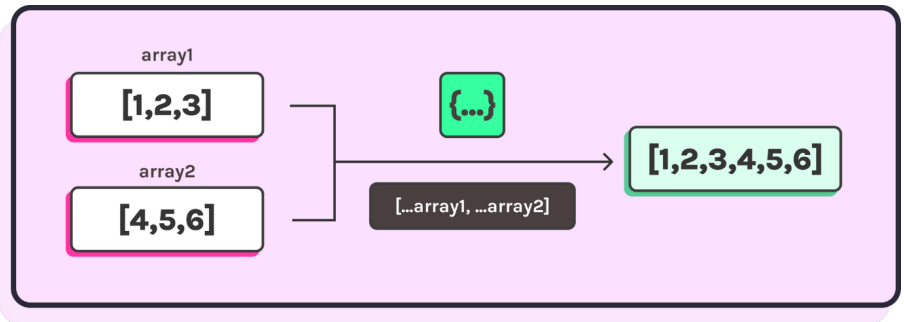
- Array Destructuring → [] use pannuvom.
- Object Destructuring → {} use pannuvom.
- Object-la property name match aaganum.

CHAPTER 13

Spread Operator (...)

ARRAYS · OBJECTS · COPYING · UPDATING · SHALLOW COPY

JavaScript-la Spread Operator (...) use panni, already irukkura Array illa Object-la irukkura ella values-ah-vum illa specific values-ah-vum quickly copy panni, innoru Array illa Object-la add pannikkalaam.



Example 1: Array-la Spread Operator Use Pannurathu

```
<!DOCTYPE html>
<html>
<body>
<script>
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];
document.write(numbersCombined);
</script>
</body>
</html>
```

Explanation:

- numbersOne-la [1, 2, 3] values irukku.
- numbersTwo-la [4, 5, 6] values irukku.
- ...numbersOne and ...numbersTwo use panni rendu Array values-ahvum numbersCombined-la add pannirukkom.

Output:

```
1,2,3,4,5,6
```

Example 2: Object-la Spread Operator Use Pannurathu

Array mattum illaama, Object-layum Spread Operator use pannalaam.

```
<!DOCTYPE html>
<html>
<body>
<script>
const myVehicle = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red'
};
const updateMyVehicle = {
  type: 'car',
  year: 2021,
  color: 'yellow'
};
const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle};
// Check the result object in the console:
console.log(myUpdatedVehicle);
</script>
<p>Press F12 and see the result object in the console view.</p>
</body>
</html>
```

Explanation:

- myVehicle Object-la brand, model, color properties irukku.
- updateMyVehicle Object-la type, year, color properties irukku.
- ...myVehicle and ...updateMyVehicle use panni rendu Object properties-ahvum myUpdatedVehicle-la copy pannirukkom.

Important: Rendu Object-layum same property name (color) irukku. So, last-la varra Object-oda value (yellow) dhaan final Object-la irukkum.

Output:

```
{
  brand: 'Ford',
  model: 'Mustang',
  color: 'yellow',
  type: 'car',
  year: 2021
}
```

💡 Note: Spread Operator ES6 (ECMAScript 2015)-la introduce pannanga. Romba old browsers-la Spread Syntax support illama irukkalaam.

Example 3: Updating Array (Without Spread Operator)

```
const numbers = [1, 2, 3];
// Add new value
numbers.push(4);
console.log(numbers); // [1, 2, 3, 4]
```

Explanation:

- numbers-nu oru Array create pannirukkom.
- push(4) use panni Array-oda last-la 4 add pannirukkom.

Problem:

1. push() original Array-ah directly change (mutate) pannidum.
2. React maathiri frameworks-la State Array-ah direct-ah change pannurathu bugs-ku reason aagalaam (State Mutation Issue).

Example 4: Updating Array (With Spread Operator)

```
const numbers = [1, 2, 3];
const newNumbers = [...numbers, 4];
console.log(newNumbers); // [1, 2, 3, 4]
```

Explanation:

- numbers original Array-la [1, 2, 3] values irukku.
- ...numbers use panni existing Array values-ah copy pannirukkom.
- Adhoda last-la 4 add pannirukkom.
- Ippadi pannumbodhu newNumbers-nu oru new Array create aagum.
- Original numbers Array change aagaathu.

Output:

```
[1, 2, 3, 4]
```

Difference Between Without Spread and With Spread

Without Spread	With Spread
push() use pannuvom	... use pannuvom
Original Array change aagum	New Array create aagum
Mutation nadakkum	Original Array unchanged-ah irukkum
React State update-ku direct mutation avoid pannanum	React State update-ku useful

Remember: Spread Operator use pannumbodhu original Array/Object change aagaathu; adhanoda values copy panni new Array/Object create pannalaam. Aana idhu shallow copy dhaan. Nested Objects irundha, avai deep copy aagaathu.

Objects

> **Practical Task 1:** Create a product object with name, price, quantity and category. Display its properties using both dot notation and bracket notation.

> **Practical Task 2:** Create a nested employee object and add, update and delete properties. Display the final object in the console.

Object Methods

> **Practical Task 1:** Create an object containing product details and use Object.keys(), Object.values() and Object.entries() to display its data.

> **Practical Task 2:** Create an object with several properties and use hasOwnProperty() to check whether selected properties belong to that object.

Object Destructuring and Spread Operator

> **Practical Task 1:** Extract name, age and address from a nested student object using object destructuring, including default values and renamed variables.

> **Practical Task 2:** Merge two objects and update an array using the spread operator without modifying the original array or object.

CHAPTER 14

Conditional Statements

IF · IF...ELSE · ELSE IF · SWITCH · TERNARY OPERATOR

JavaScript-la Conditional Statements use panni, oru condition true-ah irukka false-ah irukka-nu check panni, adhukku etha maathiri code execute pannalaam.

Simple-ah sollanumna, condition-ah check panni code-oda execution-ah control pannurathukku Conditional Statements use aagum.

JavaScript-la different types of Conditional Statements irukku:

1. If Statement: Oru condition true-ah irundha, adhukulla irukkura code execute aagum.
2. Else Statement: if condition false-ah irundha, else-kulla irukkura code execute aagum.
3. Else If Statement: First if condition false-ah irundha, next condition-ah check pannurathukku else if use pannuvom.

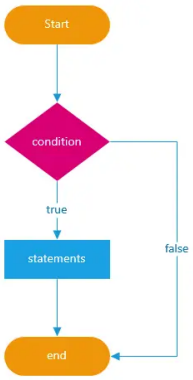
If Statement

if statement oru expression-ah evaluate pannum. Andha expression true-ah irundha mattum, if block-kulla irukkura code execute aagum.

Condition false-ah irundha, if block-kulla irukkura code execute aagaathu.

Syntax:

```
if (expression) {
    // Condition true-ah irundha execute aagura code
}
```



Example:

```
let age = 20;
if (age >= 18) {
    document.write("You are eligible to vote");
}
```

Output:

```
You are eligible to vote
```

Explanation:

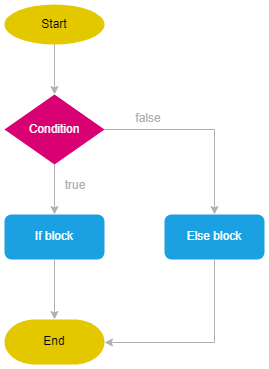
- age = 20 nu value assign pannirukkom.
- age >= 18 condition check aagum.
- 20 is greater than 18. So condition true.
- Adhanala if block-kulla irukkura statement execute aagum.

If...Else Statement

1. if...else statement-la, kudukkura condition true (truthy)-ah irundha, if block-kulla irukkura code execute aagum.
2. Condition false-ah irundha, else block-kulla irukkura code execute aagum.
3. if condition pass aagala-na, vera oru block of code execute aaganum-na else statement use pannuvom.

Syntax:

```
if (expression) {
    // Condition true-ah irundha execute aagura code
}
else {
    // Condition false-ah irundha execute aagura code
}
```



Example:

```
let age = 16;
if (age >= 18) {
    document.write("You are eligible to vote");
}
else {
    document.write("You are not eligible to vote");
}
```

Output:

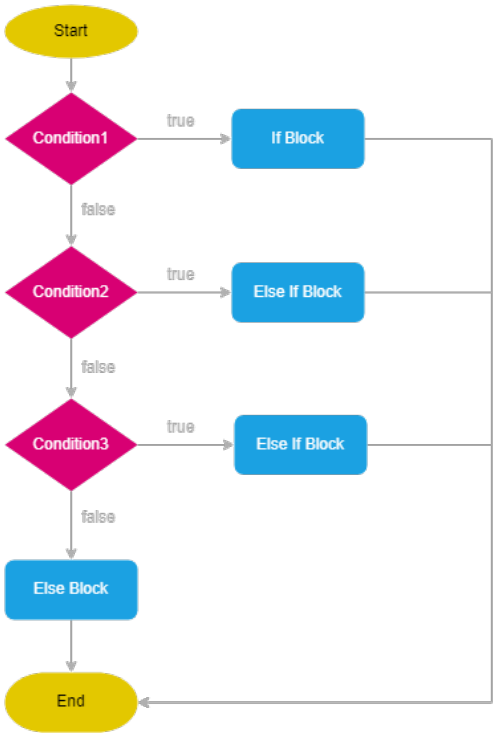
```
You are not eligible to vote
```

Explanation:

- age = 16 nu value assign pannirukkom.
- age >= 18 condition check aagum.
- 16 is less than 18. So condition false.
- Adhanala if block execute aagaathu.
- else block-kulla irukkura statement execute aagum.

Else If Statement

1. if...else statement-la, kudukkura condition true-ah irundha, if block-kulla irukkura code execute aagum.
2. First condition false-ah irundha, next condition-ah check pannurathukku else if use pannuvom.
3. Oru condition-um true-ah illana, last-la irukkura else block execute aagum.



Thevai pattal evvalavu else if statements venumnaalum use pannikkalaam.

Syntax:

```
if (expression1) {
    // expression1 true-ah irundha execute aagum
}
else if (expression2) {
    // expression2 true-ah irundha execute aagum
}
else if (expression3) {
    // expression3 true-ah irundha execute aagum
}
else {
    // Endha expression-um true illana execute aagum
}
```

Example:

```
let mark = 75;
if (mark >= 90) {
    document.write("Grade A");
}
else if (mark >= 70) {
    document.write("Grade B");
}
else if (mark >= 50) {
    document.write("Grade C");
}
else {
    document.write("Fail");
}
```

Output:

```
Grade B
```

Explanation:

- mark = 75 nu value assign pannirukkom.
- First if condition (75 >= 90) check aagum. Idhu false.
- Adhukku next else if condition (75 >= 70) check aagum. Idhu true.
- So, Grade B print aagum.
- Condition true aagitta, adhukku keezha irukkura remaining else if and else blocks check aagaathu.

Remember: if condition-la irundhu start aagi, ovvoru else if condition-ahvum order-ah check pannum. First true condition kidaichadhum andha block mattum execute aagum.

Switch Statement

1. switch statement-um JavaScript-oda Conditional Statements-la oru type. Different conditions-ku etha maathiri different actions perform pannurathukku use aagum.
2. Neraya options irukkumbodhu, adhula oru particular value-ku match aagura code block-ah select panni execute pannurathukku switch use pannuvom.
3. Romba lengthy-ah irukkura nested if...else statements-ku badhila switch statement use pannalaam. Idhu code-ah simple-ahவும் readable-ahவும் maintain panna help pannum.
4. **Jump Statement:** Program execute aagumbodhu, normal flow-ah interrupt panni, oru particular place-ku jump aagurathukku Jump Statement use aaguthu.

Syntax:

```
switch (expression) {
    case value1:
        // value1 match aana execute aagum
        break;
    case value2:
        // value2 match aana execute aagum
        break;
    default:
        // Endha case-um match aagala-na execute aagum
}
```

Example: Display Today's Day

```
<!DOCTYPE html>
<html>
<body>
<p>Click the button to display what day it is today.</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
    var day;
    switch (new Date().getDay()) {
        case 0:
```

```
            day = "Sunday";
            break;
        case 1:
            day = "Monday";
            break;
        case 2:
            day = "Tuesday";
            break;
        case 3:
            day = "Wednesday";
            break;
        case 4:
            day = "Thursday";
            break;
        case 5:
            day = "Friday";
            break;
        case 6:
            day = "Saturday";
            break;
        default:
            day = "Unknown Day";
    }
    document.getElementById("demo").innerHTML = "Today is " + day;
}
</script>
</body>
</html>
```

Explanation:

- myFunction() function button click pannumbodhu execute aagum.
- new Date().getDay() use panni current day-oda number-ah get pannuvom.
- switch andha number-ah ovvoru case value-oda compare pannum.
- Match aagura case-kulla irukkura code execute aagum.
- break use pannurathoda purpose, andha case execute aanadhum switch-la irundhu veliya varanum. Illana next case-um execute aagum (fall-through).
- Endha case-um match aagala-na default block execute aagum.

getDay() Return Values

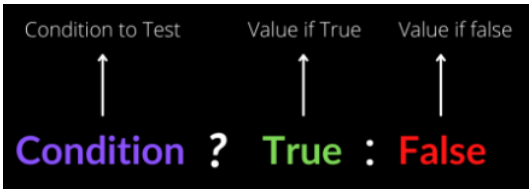
Value	Day
0	Sunday
1	Monday
2	Tuesday
3	Wednesday
4	Thursday
5	Friday
6	Saturday

Remember:

- case → Oru particular value-ah check pannum.
- break → Switch statement-la irundhu veliya varum.
- default → Endha case-um match aagala-na execute aagum.
- switch → Multiple conditions/options-ah handle panna useful-ah irukkum.

Ternary Operator (? :)

1. Ternary Operator enbadhu JavaScript-la irukkura oru Conditional Operator.
2. Idhu rendu expressions-la edhu execute aaganum-nu decide pannum.
3. Condition true-ah irundha, True Expression execute aagum.
4. Condition false-ah irundha, False Expression execute aagum.
5. Neenga kudukkura condition-ah check panni, adhoda result-ku etha maathiri rendu expressions-la onnu select panni execute pannum.



Syntax:

```
condition ? expression1 : expression2
```

Example:

```
let age = 20;
let result = age >= 18 ? "Eligible" : "Not Eligible";
document.write(result);
```

Output:

```
Eligible
```

Explanation:

- age = 20 nu value assign pannirukkom.
- age >= 18 condition check aagum.
- Condition true-ah irukkuradhala "Eligible" return aagum.
- Condition false-ah irundha "Not Eligible" return aagum.

Remember: Ternary Operator enbadhu simple if...else condition-ah ore line-la write pannurathukku useful-ah irukkum.

> **Practical Task 1:** Create a program that checks whether a number is positive, negative or zero using if...else.

> **Practical Task 2:** Create a menu-driven calculator using switch and a student grade calculator using if...else if...else or the ternary operator.

CHAPTER 15

Looping Statements

FOR · WHILE · DO...WHILE · FOR...IN · FOR...OF · BREAK & CONTINUE

1. JavaScript-la Loops use panni, oru task-ah repeated-ah perform pannalaam.
2. Loop-la kudukkura conditions usually true அல்லது false nu result return pannum.
3. Defined condition false aagura varaikkum loop repeated-ah execute aagite irukkum.

Different Kinds of Loops

JavaScript-la 5 types of Loops irukku:

1. for: Oru block of code-ah specified number of times repeat pannurathukku use aagum.
2. for...in: Object-la irukkura properties-ah iterate pannurathukku use aagum.
3. for...of: Iterable Object-la irukkura values-ah iterate pannurathukku use aagum.
4. while: Kudukkura condition true-ah irukkura varaikkum code block-ah repeat pannum.

5. do...while: Idhuvum condition true-ah irukkura varaikkum code block-ah repeat pannum. Aana first time condition check pannurathukku munnaadiye code execute aagum.

Entry Controlled Loops

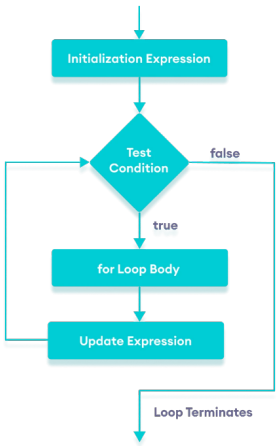
1. Indha type of Loop-la, Loop body-kulla enter aagurathukku munnaadiye test condition check pannappadum.
2. for Loop and while Loop renduume Entry Controlled Loops aagum.
3. Condition first-la false-ah irundha, Loop body oru murai kooda execute aagaathu.

Exit Controlled Loops

1. Indha type of Loop-la, Loop body execute aanadhukku appuram test condition check pannappadum.
2. Adhanala condition true-ah irundhaalum false-ah irundhaalum, Loop body minimum oru murai kandippa execute aagum.
3. do...while Loop oru Exit Controlled Loop aagum.

For Loop

1. for Loop use panni, oru condition false aagura varaikkum code-ah repeated-ah execute pannalaam.
2. JavaScript for Loop-oda structure, Java and C programming languages-la irukkura for Loop maathiriye irukkum.
3. for Loop-la mainly 3 parts irukkum:



Syntax:

```
for (initialization; condition; afterthought) {
    // Code to be executed
}
```

Three Parts Explanation:

Part	Explanation
Initialization	Starting value-ah initialize pannuvom.
Condition	Ovvoru iteration-kum condition check aagum.
Afterthought	Loop body execute aanadhukku appuram value update aagum.

Example: JavaScript For Loop

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript For Loop</h2>
<p id="demo"></p>
<script>
let text = "";
for (let i = 0; i < 5; i++)
{
  text = text + "The number is " + i + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```

Explanation:

- let text = ""; → Empty String-ah text variable-la assign pannirukkom.
- let i = 0 → Loop 0-la irundhu start aagum.
- i < 5 → i value 5-ku kammiya irukkura varaikkum Loop execute aagum.
- i++ → Ovvoru iteration mudinjadhum i value 1 increase aagum.
- text = text + "The number is " + i + "
"; → Ovvoru iteration-layum current i value-ah text-oda add pannuvom.
 use panni next line-ku move aagum.
- document.getElementById("demo").innerHTML = text; → Final-ah text variable-la irukkura content-ah HTML page-la display pannum.

Output:

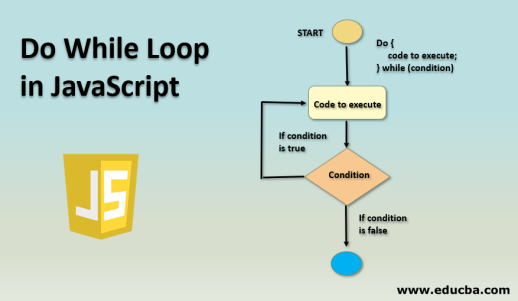
```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
```

Loop Execution:

Remember: i < 5 condition use pannirukkaradhala 0 to 4 varaikkum total 5 times Loop execute aagum. 5 vandhadhum condition false aagi Loop stop aagidum.

Do...While Loop

1. JavaScript-la do...while statement use panni, oru condition false aagura varaikkum code block-ah repeated-ah execute pannalaam.
2. Indha Loop-la condition-ah check pannurathukku munnaadi do block-kulla irukkura code first execute aagum.
3. Adhanala condition first-la false-ah irundhaalum, do...while Loop minimum oru murai kandippa execute aagum.
4. do...while statement use panni, oru code block-ah first one time execute pannitu, condition true-ah irukkura varaikkum repeated-ah execute pannalaam.
5. do...while Loop-ah use pannurathu eppona, oru code block minimum oru muraiyaavadhu execute aaganum-nu ninaikkumbodhu use pannuvom.
6. Remember: do...while Loop-la first code execute aagum; adhukku appuram dhaan condition check aagum.



Syntax:

```
do {
    // Code to be executed
} while (condition);
```

Example:

```
let i = 1;
do {
    document.write(i + "<br>");
    i++;
} while (i <= 5);
```

Output:

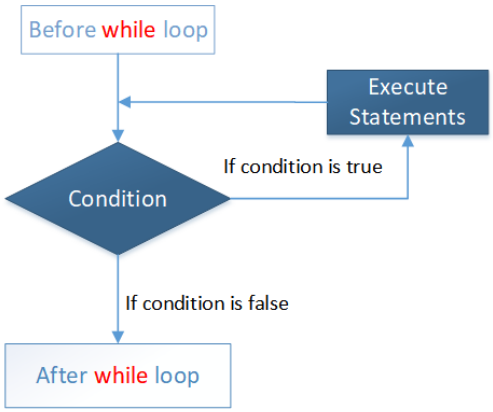
```
1
2
3
4
5
```

Remember: do...while Loop-la first code execute aagum; adhukku appuram dhaan condition check aagum.

Condition false-ah irundhaalum minimum one time execute aagum.

While Loop

1. JavaScript-la while statement use panni, kudukkura test condition true-ah irukkura varaikkum specified code block-ah repeated-ah execute pannalaam.
2. while Loop-la code execute aagurathukku munnaadiye condition check pannappadum.
3. Condition true-ah irundha code execute aagum. Condition false-ah irundha Loop execute aagaathu.



Syntax:

```
while (condition) {
    // Code to be executed
}
```

Example:

```
let i = 1;
while (i <= 5) {
    document.write(i + "<br>");
    i++;
}
```

Output:

```
1
2
3
4
5
```

Explanation:

- let i = 1 → Starting value 1-la irundhu start aagum.
- i <= 5 → Condition first check aagum.
- Condition true-ah irundha number display aagum.
- i++ → Ovvoru iteration-layum i value 1 increase aagum.
- i = 6 aagumbodhu condition false aagum. So Loop stop aagidum.

While vs Do...While Loop

While Loop	Do...While Loop
First condition check aagum.	First code execute aagum.
Condition true-ah irundha code execute aagum.	Code execute aanadhukku appuram condition check aagum.
Condition first-la false-ah irundha, oru murai kooda execute aagaathu.	Condition first-la false-ah irundhaalum minimum one time execute aagum.
Entry Controlled Loop.	Exit Controlled Loop.

For...In Loop

1. JavaScript-la for...in Loop use panni, oru Object-la irukkura properties (keys)-ah one by one iterate pannalaam.
2. for...in Loop, enumerable property true-ah set pannirukkura Object keys-ah mattum iterate pannum.
3. JavaScript Object properties-ku 4 attributes irukku:
4. Value: Property-la store pannirukkura value.
5. Writable: Andha property-oda value-ah change panna mudiyuma-nu decide pannum.
6. Enumerable: Andha property-ah Loop-la iterate panna mudiyuma-nu decide pannum.
7. Configurable: Andha property-ah delete panna mudiyuma அல்லது attributes-ah change panna mudiyuma-nu decide pannum.

Important Points

1. for...in Loop-ah mostly Array illaadha Objects-la irukkura properties-ah iterate pannurathukku use pannuvom. Array-ku for...in use panna mudiyum. Aana generally recommend pannappadaathu. Array values-ah loop panna for, for...of அல்லது forEach() use pannurathu better.
2. for...in Loop use pannumbodhu, Object-oda own enumerable properties mattum illaama, adhan Prototype Chain-la irukkura enumerable properties-um include aagalaam.
3. for...in Loop-la properties iterate aagura order, Object-la properties define pannirukkura order-oda eppovum match aaganum-nu avasiyam illa.

For...In Loop Syntax

```
for (let i in obj1) {
    // obj1-la irukkura ella keys-ahvum
    // Console-la print pannum
    console.log(i);
}
```

Explanation:

- obj1 → Iterate panna vendiya Object.
- let i → Ovvoru iteration-layum Object-oda property name (key) i variable-la store aagum.
- console.log(i) → Property names-ah Console-la display pannum.

Example 1: Object-la for...in Loop

```
const person = {
    name: "Alice",
    age: 30,
    city: "New York"
};
for (const property in person) {
    console.log(`${property}: ${person[property]}`);
}
```

Explanation:

- person-nu oru Object create pannirukkom.
- for...in use panni Object-oda properties-ah one by one access pannuvom.
- property variable-la name, age, city nu keys store aagum.
- person[property] use panni andha key-oda value-ah access pannuvom.
- Template Literals (`) use panni key and value-ah display pannirukkom.

Output:

```
name: Alice
age: 30
city: New York
```

Example 2: Array-la for...in Loop

```
const numbers = [1, 4, 7, 10];
for (const index in numbers) {
    console.log(`Element at index ${index}: ${numbers[index]}`);
}
```

Explanation:

- numbers-nu oru Array create pannirukkom.
- for...in use panni Array-oda indexes-ah one by one access pannuvom.
- index variable-la 0, 1, 2, 3 nu index values store aagum.
- numbers[index] use panni andha index-la irukkura Array value-ah access pannuvom.

Output:

```
Element at index 0: 1
Element at index 1: 4
Element at index 2: 7
Element at index 3: 10
```

Remember:

- Object-la for...in use pannumbodhu → Property keys kidaikkum.
- Array-la for...in use pannumbodhu → Index values kidaikkum.
- - for...in Array-kum work aagum. Aana Array values-ah iterate pannurathukku for...of illa forEach() use pannurathu generally better.

For...Of Loop

1. JavaScript-la for...of Loop use panni, oru iterable-la irukkura values-ah one by one sequential order-la access pannalaam.
2. for...of Loop use panni Arrays, Sets, Maps, Strings pondra iterable Objects-la irukkura values-ah iterate pannalaam.
3. Loop-la ovvoru value-ah access pannuradhum oru Iteration nu solluvom.
4. Ovvoru iteration-layum current value element variable-la store aagum. Andha value-ah use panni statements execute pannalaam.

Syntax

```
for (element of iterable) {
    // for...of Loop-oda code block
}
```

Here:

1. iterable → Iterate panna vendiya iterable Object (Array, Set, String pondravai).
2. element → Iterable-la irukkura ovvoru value-um indha variable-la store aagum.

For...Of with Arrays

```
// Array
const students = ['John', 'Sara', 'Jack'];
// using for...of
for (let element of students) {
    // Display the values
    console.log(element);
}
```

Explanation:

- students-nu oru Array create pannirukkom.
- for...of use panni Array-la irukkura values-ah one by one access pannuvom.
- Ovvoru iteration-layum element variable-la current value store aagum.

Output:

```
John
Sara
Jack
```

For...Of with Strings

```
// String
const string = 'code';
// using for...of loop
for (let i of string) {
    console.log(i);
}
```

Explanation:

- string variable-la 'code' nu oru String store pannirukkom.
- for...of use panni String-la irukkura ovvoru character-ahvum one by one access pannuvom.
- i variable-la ovvoru character-um store aagum.

Output:

```
c
o
d
e
```

For...Of with Sets

```
// Define Set
const set = new Set([1, 2, 3]);
// Looping through Set
for (let i of set) {
    console.log(i);
}
```

Explanation:

- new Set([1, 2, 3]) use panni oru Set create pannirukkom.
- Set-la duplicate values allow aagaathu.
- for...of use panni Set-la irukkura values-ah one by one access pannuvom.
- Ovvoru iteration-layum i variable-la current value store aagum.

Output:

```
1
2
3
```

Remember: for...in vs for...of

for...in	for...of
Object properties (keys)-ah iterate pannum.	Iterable values-ah iterate pannum.
Array-la indexes kidaikkum.	Array-la values kidaikkum.
Object-ku useful.	Array, String, Set, Map-ku useful.

Easy-ah remember pannunga:

- for...in → Index / Keys.
- for...of → Values.

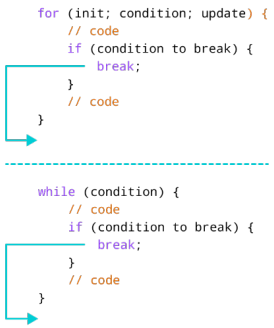
Break Statement

JavaScript-la break statement use panni, Loop execute aagittu irukkumbodhu, break statement encounter aana udane Loop-ah stop pannalaam.

Adhaavathu, condition true-ah irundhaalum break execute aana Loop immediately terminate aagidum.

Syntax:

```
break;
```



Example 1: Break with For Loop

```
// i-oda value-ah print pannurathukku
for (let i = 1; i <= 5; i++) {
    // Break condition
    if (i == 3) {
        break;
    }
    console.log(i);
}
```

Output:

```
1
2
```

Explanation:

- for Loop 1-la irundhu start aagum.
- i <= 5 condition true-ah irukkura varaikkum Loop execute aagum.
- if (i == 3) condition use panni i value 3-ah irukka-nu check pannuvom.
- i value 3 aagumbodhu break execute aagi Loop stop aagidum.
- Adhanala 1, 2 mattum Console-la print aagum.

💡 Note: break statement almost always Decision-Making Statements (if, if...else) kooda use pannappadum. Condition satisfy aagumbodhu Loop-ah stop pannurathukku idhu useful-ah irukkum.

Continue Statement

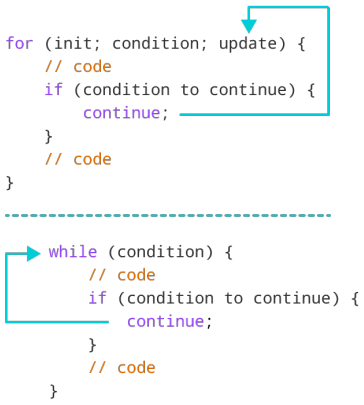
JavaScript-la continue statement use panni, Loop-la current iteration-ah skip pannitu, next iteration-ku move aagalaam.

Adhaavathu, continue statement encounter aana, current iteration-la adhukku keezha irukkura remaining code execute aagaathu. Direct-ah next iteration-ku poidum.

Syntax:

```
continue;
```

Working of JavaScript continue Statement



Continue with For Loop

for Loop-la continue statement use pannumbodhu, current iteration skip aagum. Adhukku appuram control flow direct-ah updateExpression-ku (increment/decrement part) move aagum.

Adhaavathu, for Loop-la current iteration skip aanadhukku appuram i++ execute aagi, next iteration start aagum.

Example 1: Print the Value of i

```
// i-oda value-ah print pannurathukku
for (let i = 1; i <= 5; i++) {
    // Continue condition
    if (i == 3) {
        continue;
    }
    console.log(i);
}
```

Output:

```
1
2
4
5
```

Explanation:

- Loop 1-la irundhu start aagum.
- i == 3 condition check aagum.
- i value 3 aagumbodhu continue execute aagum.
- Adhanala console.log(i) statement current iteration-la skip aagum.
- Adhukku appuram i++ execute aagi, next iteration continue aagum.
- So, 3 mattum print aagaathu. Maththa values print aagum.

💡 Note 1: continue statement almost always Decision-Making Statements (if, if...else) kooda use pannappadum.

💡 Note 2: Break vs Continue

- break → Loop-ah complete-ah terminate pannidum.
- continue → Current iteration-ah mattum skip pannitu next iteration-ku move aagum.

> **Practical Task 1:** Print numbers from 1 to 100 and generate a multiplication table using for, while and do...while loops.

> **Practical Task 2:** Create a program that loops through an object and an array using for...in and for...of. Demonstrate break and continue.

CHAPTER 16

Set

CREATE · ADD · DELETE · HAS · SIZE · ITERATE · DUPLICATES

1. What is a Set?

Definition: A Set is a collection of unique values. It does not allow duplicate values.

Simple-ah sollanumna, Set-la same value-ah multiple times add pannalum, oru value mattum dhaan store aagum.

Example:

Normal Array:

```
let numbers = [10, 20, 10, 30, 20];
console.log(numbers);
```

Screen Output:

```
[10, 20, 10, 30, 20]
```

Set:

```
let numbers = new Set([10, 20, 10, 30, 20]);
console.log(numbers);
```

Screen Output:

```
Set(3) {10, 20, 30}
```

Inga duplicate values automatically remove aagidum.

Important Features:

1. Set allows only unique values.
2. Duplicate values automatically remove aagum.
3. Insertion order preserve aagum.
4. Set is iterable, so loops use panni access pannalam.
5. Array-la irukkura madhiri index-based access Set-la kedaiyadhu.

2. Create a Set

Set create panna new Set() use pannuvom.

Syntax:

```
const mySet = new Set();
```

Example:

```
const mySet = new Set();
console.log(mySet);
```

Screen Output:

```
Set(0) {}
Inga empty Set create pannirukkom.
```

3. Add Items to a Set – add()

add() method use panni Set-kulla new values add pannalam.

Example:

```
const mySet = new Set();
mySet.add(10);
mySet.add(20);
mySet.add(30);
mySet.add(10);
console.log(mySet);
```

Screen Output:

```
Set(3) {10, 20, 30}
```

Explanation:

- add(10) – 10 add aagum.
- add(20) – 20 add aagum.
- add(30) – 30 add aagum.
- add(10) – Already 10 irukkaradhunaala duplicate add aagadhu.

4. Delete Items – delete()

delete() method use panni particular value-ah remove pannalam.

Example:

```
const mySet = new Set([10, 20, 30, 40]);
mySet.delete(20);
console.log(mySet);
```

Screen Output:

```
Set(3) {10, 30, 40}
```

Explanation:

- delete(20) – Set-la irukkura 20 remove aagum.
- Remaining values mattum irukkum.

5. Check Value – has()

has() method use panni oru value Set-la irukka illaiya nu check pannalam.

It returns either true or false.

Example:

```
const mySet = new Set([10, 20, 30]);
console.log(mySet.has(20));
console.log(mySet.has(50));
```

Screen Output:

```
true
false
```

Explanation:

- has(20) – 20 irukku, so true.
- has(50) – 50 illa, so false.

6. Get Set Size – size

size property use panni Set-la ethana unique values irukku nu find pannalam.

Example:

```
const mySet = new Set([10, 20, 30, 10, 20]);
console.log(mySet.size);
```

Screen Output:

```
3
```

Inga 5 values kuduthirukkom. Aana duplicates remove aaguradhunaala 3 unique values mattum irukku.

7. Clear All Items – clear()

clear() method use panni Set-la irukkura ella values-ah remove pannalam.

Example:

```
const mySet = new Set([10, 20, 30]);
mySet.clear();
console.log(mySet);
```

Screen Output:

```
Set(0) {}
Inga complete Set empty aagidum.
```

8. Iterating Over a Set

Set-la irukkura values-ah loop use panni access pannalam.

Two common methods:

1. for...of
2. forEach()

A. Using for...of

```
const mySet = new Set([10, 20, 30]);
for (let value of mySet) {
    console.log(value);
}
```

Screen Output:

```
10
20
30
```

Explanation:

- for...of – Set-la irukkura each value-ah one by one access pannum.
- value – Current value-ah store pannum.

B. Using forEach()

```
const mySet = new Set([10, 20, 30]);
```

mySet.forEach(value => {

```
    console.log(value);
});
```

Screen Output:

```
10
20
30
```

Inga forEach() each value-ku callback function execute pannum.

9. Removing Duplicates from an Array

Set-oda most useful application idhu dhaan.

Array-la duplicate values irundha, Set use panni remove pannalam.

Example:

```
const numbers = [10, 20, 20, 30, 40, 40, 50];
const uniqueNumbers = [...new Set(numbers)];
console.log(uniqueNumbers);
```

Screen Output:

```
[10, 20, 30, 40, 50]
```

Explanation:

Step 1: Original array:

[10, 20, 20, 30, 40, 40, 50]

Step 2: Set create panrom:

new Set(numbers);

Output:

```
Set(5) {10, 20, 30, 40, 50}
```

Step 3: Spread operator ... use panni again Array-ah convert panrom.

[...new Set(numbers)]

Final Output:

```
[10, 20, 30, 40, 50]
```

10. Difference Between Set and Array

Features	Set	Array
Duplicates	Not allowed	Allowed
Ordering	Insertion order	Insertion order
Index access	No	Yes
Size	size	length
Add item	add()	push()
Remove item	delete()	pop(), splice()
Loop	Yes	Yes

Practical Task 1: Create a Set from an array containing duplicate numbers and display only the unique values.

Practical Task 2: Create a Set manager that allows users to add, delete, check and clear values. Display the Set size after each operation.

CHAPTER 17

map(), filter(), reduce()

MAP · FILTER · REDUCE · DIFFERENCES · REAL-TIME EXAMPLE

Indha three methods-um JavaScript-la Array Higher-Order Methods nu solluvanga.

Simple-ah sonna, array-la irukkura values-ah process panna use pannuvom.

1. map() – Ovvoru element-ah modify panni new array create pannum.
2. filter() – Condition satisfy pannura elements-ah mattum select pannum.
3. reduce() – Array values-ah combine panni single result-ah return pannum.

1. map() Method

Definition: map() method array-la irukkura ovvoru element-kum oru operation perform panni, new array return pannum.

Original array change aagadhu.

Syntax:

```
array.map(function(element, index, array) {
    return value;
});
```

Example 1: Numbers Double

```
let numbers = [10, 20, 30, 40, 50];
let result = numbers.map(function(num) {
    return num * 2;
});
console.log(result);
```

Output:

```
[20, 40, 60, 80, 100]
```

Tanglish Explanation:

- numbers – Original array.
- map() – Ovvoru element-ah edukkum.
- num – Current element.
- num * 2 – Ovvoru number-um 2-ah multiply pannum.
- result – New array store aagum.

Step-by-step:

Original	Operation	Result
10	10 × 2	20
20	20 × 2	40
30	30 × 2	60
40	40 × 2	80
50	50 × 2	100

Example 2: Using Arrow Function

```
let numbers = [1, 2, 3, 4, 5];
let result = numbers.map(num => num * num);
console.log(result);
```

Output:

```
[1, 4, 9, 16, 25]
```

Inga num * num use panni ovvoru number-oda square calculate pannirukkom.

Example 3: Convert Names to Uppercase

```
let names = ["arun", "kumar", "raja"];
let result = names.map(name => name.toUpperCase());
console.log(result);
```

Output:

```
["ARUN", "KUMAR", "RAJA"]
```

2. filter() Method

Definition: filter() method array-la irukkura elements-la condition satisfy pannura elements-ah mattum select panni new array return pannum.

Syntax:

```
array.filter(function(element) {
    return condition;
});
```

Example 1: Find Even Numbers

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8];
let result = numbers.filter(function(num) {
    return num % 2 == 0;
});
console.log(result);
```

Output:

```
[2, 4, 6, 8]
```

Tanglish Explanation:

- filter() – Ovvoru number-ah check pannum.
- num % 2 == 0 – Number even-ah irukka nu check pannum.
- Condition true na array-la include pannum.
- Condition false na skip pannidum.

Number	Condition	Result
1	False	Skip
2	True	Include
3	False	Skip
4	True	Include
5	False	Skip
6	True	Include

Example 2: Find Numbers Greater Than 50

```
let numbers = [10, 25, 50, 65, 80, 100];
let result = numbers.filter(num => num > 50);
console.log(result);
```

Output:

```
[65, 80, 100]
```

Example 3: Filter Students

```
let students = [
    { name: "Arun", mark: 85 },
    { name: "Kumar", mark: 45 },
    { name: "Raja", mark: 90 },
    { name: "Mani", mark: 35 }
];
let result = students.filter(student => student.mark >= 50);
console.log(result);
```

Output:

```
[
  { name: "Arun", mark: 85 },
  { name: "Raja", mark: 90 }
]
```

Inga 50 or above mark edutha students mattum new array-la varuvanga.

3. reduce() Method

Definition: reduce() method array-la irukkura ella elements-ah process panni single value-ah return pannum.

Example: Total, average, multiplication result calculate panna use pannalam.

Syntax:

```
array.reduce(function(accumulator, currentValue) {
    return accumulator + currentValue;
}, initialValue);
```

Important terms:

- accumulator – Previous calculation result-ah store pannum.
- currentValue – Current element.
- initialValue – Accumulator-ku starting value.

Example 1: Find Sum of Array

```
let numbers = [10, 20, 30, 40, 50];
let result = numbers.reduce(function(total, num) {
    return total + num;
}, 0);
console.log(result);
```

Output:

```
150
```

Tanglish Explanation:

Starting value 0.

Step	total	num	Calculation	Result
1	0	10	0 + 10	10
2	10	20	10 + 20	30
3	30	30	30 + 30	60
4	60	40	60 + 40	100
5	100	50	100 + 50	150

Final result 150.

Example 2: Using Arrow Function

```
let numbers = [5, 10, 15, 20];
let result = numbers.reduce((total, num) => total + num, 0);
console.log(result);
```

Output:

```
50
```

Example 3: Find Product of Numbers

```
let numbers = [2, 3, 4, 5];
let result = numbers.reduce((total, num) => total * num, 1);
console.log(result);
```

Output:

```
120
```

Calculation:

1 × 2 = 2

2 × 3 = 6

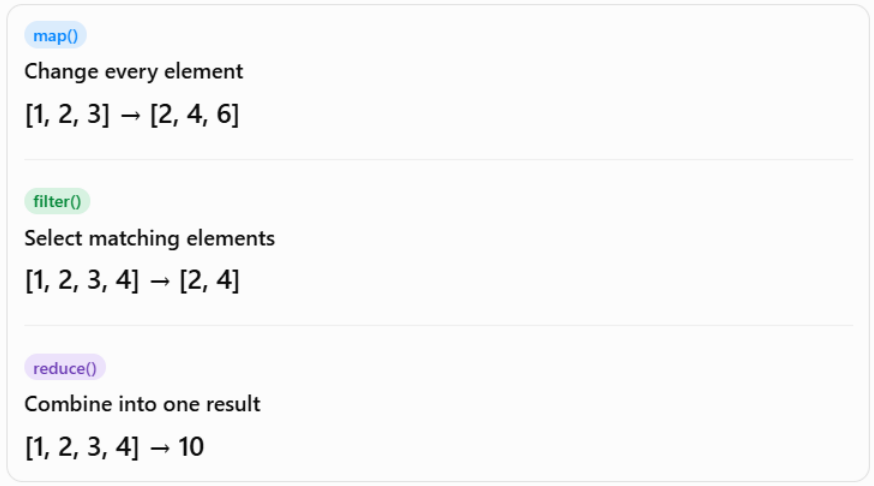
6 × 4 = 24

24 × 5 = 120

4. Difference Between map(), filter(), reduce()

Feature	map()	filter()	reduce()
Purpose	Modify elements	Select elements	Combine elements
Return	New array	New array	Single value (usually)
Iteration	Every element	Every element	Every element
Main use	Transformation	Condition checking	Calculation

Easy way to remember:



5. Real-Time Example – IT Company Employee Data

```
let employees = [
    { name: "Arun", salary: 20000 },
    { name: "Kumar", salary: 30000 },
    { name: "Raja", salary: 40000 },
    { name: "Mani", salary: 15000 }
];
// map()
let names = employees.map(emp => emp.name);
console.log(names);
// filter()
let highSalary = employees.filter(emp => emp.salary >= 30000);
console.log(highSalary);
// reduce()
let totalSalary = employees.reduce((total, emp) => total + emp.salary, 0);
console.log(totalSalary);
```

Output:

```
["Arun", "Kumar", "Raja", "Mani"]
```

[

{ name: "Kumar", salary: 30000 },

{ name: "Raja", salary: 40000 }

```
]
```

105000

Explanation:

- map() – Employees-oda names mattum extract pannuthu.
- filter() – Salary 30,000 or above irukkura employees-ah select pannuthu.
- reduce() – Ella employees-oda salary-ah add pannuthu.

> **Practical Task 1:** Use map() to convert an array of names to uppercase and to double an array of numbers.

> **Practical Task 2:** Use filter() to find employees with salaries above a given amount and reduce() to calculate the total salary.

CHAPTER 18

Functions

DECLARE · CALL · PARAMETERS · RETURN · REST · NOTES

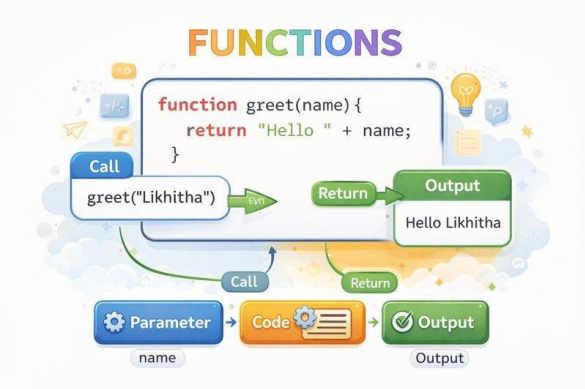
JavaScript-la Function na oru specific task perform panna use panra reusable block of code.

Simple-ah sollanumna, oru task-ku code write panni, adha thevaipadumbodhu multiple times call panni use pannalam. Idhudhaan Function.

Real-time Example:

Oru calculator-la Addition function create panrom.

- Input: 10, 20
- Process: 10 + 20
- Output: 30



Why Functions are Important?

1. Code Reusability – Same code-ah multiple times use pannalam.

2. Code Reduction – Repeated code write panna thevai illa.
3. Easy Maintenance – Code-ah easy-ah manage pannalam.
4. Modularity – Big program-ah small functions-ah divide pannalam.

1. Declaring a Function

Function create panradhukku function keyword use pannuvom.

Syntax:

```
function functionName() {
    // Function body
}
```

Rules:

1. function keyword use pannanum.
2. Function-ku oru meaningful name kudukkanum.
3. Function name-ku apram () use pannanum.
4. Function body { } kulla write pannanum.
5. Function declare pannina mattum execute aagadhu; call pannanum.

Example:

```
function greet() {
    console.log("Hello there");
}
```

Screen Output:

```
No output
```

Yen output varala? Function declare mattum pannirukkom. Innum call pannala.

2. Calling a Function

Function-ah execute panna function name use panni call pannanum.

Syntax:

```
functionName();
```

Example:

```
function greet() {
    console.log("Hello there!");
}
greet();
```

Screen Output:

```
Hello there!
```

Explanation:

- function greet() – Function create panrom.
- console.log() – Message display panrom.
- greet() – Function-ah call panrom.
- Function call aagumbodhu dhaan function body execute aagum.

Important: Oru function-ah multiple times call pannalam.

```
function greet() {
    console.log("Welcome!");
}
greet();
greet();
greet();
```

Screen Output:

```
Welcome!
Welcome!
Welcome!
```

3. Function Parameters

Definition: A parameter is a variable that receives a value when a function is called.

Simple-ah sollanumna, function-kulla outside-la irundhu values receive panna parameters use pannuvom.

Example:

```
function greet(name) {
    console.log("Hello " + name + ":)");
}
greet("Simon");
```

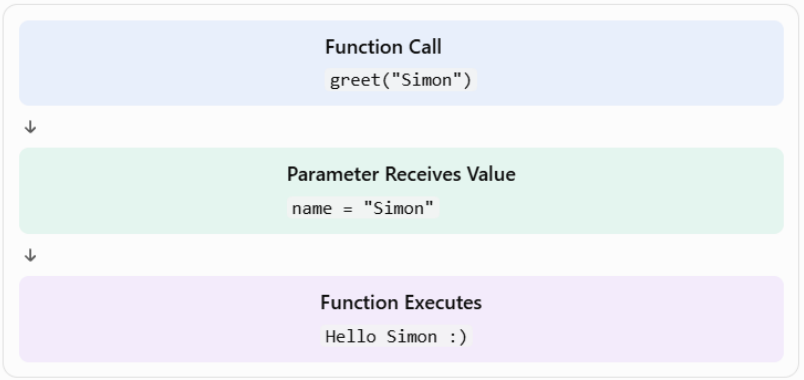
Screen Output:

```
Hello Simon :)
```

Explanation:

- name – Parameter.
- "Simon" – Argument.
- greet("Simon") – Function call.
- Function call pannumbodhu Simon value name parameter-ku pass aagum.

Step-by-step:



4. Function with Two Parameters – Add Two Numbers

Oru function-la multiple parameters use pannalam.

Example:

```
function add(a, b) {
    console.log(a + b);
}
add(3, 4);
add(2, 9);
```

Screen Output:

```
7
11
```

Explanation:

First function call:

```
add(3, 4);
```

- a = 3
- b = 4
- a + b = 7

Second function call:

```
add(2, 9);
```

- a = 2
- b = 9
- a + b = 11

Remember:

- Parameters = Function declaration-la receive panra variables (a, b).
- Arguments = Function call pannumbodhu pass panra actual values (3, 4).

5. Function Return

Definition: The return statement is used to send a value back from a function to the place where the function was called.

Simple-ah sollanumna, function-kulla calculate panna result-ah veliya return panna return use pannuvom.

Example:

```
function add(a, b) {
    return a + b;
}
let result = add(10, 20);
console.log(result);
```

Screen Output:

```
30
```

Explanation:

1. add(10, 20) function call aagum.
2. a = 10, b = 20.
3. return a + b → 30 return pannum.
4. result variable-la 30 store aagum.
5. console.log(result) output display pannum.

Difference between console.log() and return

Using console.log():

```
function add(a, b) {
    console.log(a + b);
}
let result = add(10, 20);
console.log(result);
```

Screen Output:

```
30
undefined
```

Using return:

```
function add(a, b) {
    return a + b;
}
let result = add(10, 20);
console.log(result);
```

Screen Output:

```
30
Why undefined?
```

First example-la function result-ah console-la display mattum pannudhu. Value-ah return pannala. So result-ku undefined assign aagum.

Important Rules of Return

1. return function execution-ah stop pannum.
2. Return panna value function call-ku thirumba pogum.
3. Return statement-ku apram irukkura code execute aagadhu.
4. Return use pannala na function default-ah undefined return pannum.

Example:

```
function demo() {
    console.log("Hello");
    return 100;
    console.log("Welcome");
}
demo();
```

Screen Output:

```
Hello
```

```
return 100 execute aana udane function stop aagidum. So Welcome display aagadhu.
```

6. Function with User Input

User-kitta input receive panni function-ku pass pannalam.

Example: Sum of Two Numbers

```
function add(a, b) {
    return a + b;
}
let number1 = Number(prompt("Enter first number:"));
let number2 = Number(prompt("Enter second number:"));
let result = add(number1, number2);
console.log("The sum is " + result);
```

Screen Output (Browser):

PROMPT 1

Enter first number:

3.4

PROMPT 2

Enter second number:

4

Console Output

```
The sum is 7.4
```

Explanation:

- prompt() – User-kitta input receive pannum.
- Number() – String input-ah number-ah convert pannum.
- add(number1, number2) – Function-ku values pass pannum.
- return – Addition result-ah return pannum.
- console.log() – Final result display pannum.

7. Function with Many Parameters

JavaScript function-la multiple parameters pass pannalam.

Example: Three numbers addition.

```
function add(a, b, c) {
    return a + b + c;
}
console.log(add(10, 20, 30));
```

Screen Output:

```
60
```

Inga:

- a = 10
- b = 20
- c = 30

Total = 60.

Arguments Object

JavaScript normal function declaration-la arguments nu oru array-like object available-ah irukkum.

Idha use panni function-ku pass panna arguments-ah access pannalam.

Example:

```
function add() {
    console.log(arguments[0]);
    console.log(arguments[1]);
    console.log(arguments[2]);
}
add(10, 20, 30);
```

Screen Output:

```
10
20
30
```

Explanation:

- arguments[0] – First argument.
- arguments[1] – Second argument.
- arguments[2] – Third argument.

Important: arguments is an array-like object, not a real Array. Arrow functions-ku own arguments object kedaiyadhu.

8. Rest Parameter (...)

Definition: Rest parameter allows a function to accept an indefinite number of arguments and collect them into an array.

Simple-ah sollanumna, function-ku evlo arguments pass pannalum, ellathayum oru array-la collect panna Rest Parameter use pannuvom.

... symbol use pannuvom.

Syntax:

```
function functionName(...args) {
    // Function body
}
```

Example: Sum of Multiple Numbers

```
function sum(...args) {
    let total = 0;
    for (let arg of args) {
        total = total + arg;
    }
    return total;
}
let result = sum(4, 9, 16, 25, 29, 100, 66, 77);
console.log(result);
```

Screen Output:

```
326
```

Explanation:

- ...args – All arguments-ah array-la collect pannum.
- total = 0 – Initial value.
- for...of – Array-la irukkura values-ah one by one access pannum.
- total = total + arg – Each value-ah total-kooda add pannum.
- return total – Final result return pannum.

Step-by-step calculation:

Value	Total
4	4
9	13
16	29
25	54
29	83
100	183
66	249
77	326

Final Answer = 326

Important Rules:

1. Rest parameter ... use pannanum.
2. Rest parameter arguments-ah array-ah collect pannum.
3. Oru function-la only one rest parameter irukkanum.
4. Rest parameter always last parameter-ah irukkanum.

Another Example:

```
function student(name, ...marks) {
    console.log("Name:", name);
    console.log("Marks:", marks);
}
student("Dhamodharan", 80, 90, 85);
```

Screen Output:

```
Name: Dhamodharan
Marks: [80, 90, 85]
```

Inga first argument name-ku pogum. Remaining arguments ellam marks array-la store aagum.

9. Difference Between Parameters, Arguments, Return and Rest

Concept	Meaning	Example
Parameter	Input receive panra variable	function add(a,b)
Argument	Function-ku pass panra value	add(10,20)
Return	Result-ah back send pannum	return a+b
Rest	Multiple arguments-ah array-ah collect pannum	...args
Arguments object	Passed arguments-ah access pannum	arguments[0]

10. Important Function Notes

- Function declaration pannina mattum execute aagadhu; call pannanum.
- Function-ah multiple times call pannalam.
- Parameters optional; function-ku input thevai illainalum create pannalam.
- Return optional; function return illama task perform pannalam.
- return execute aana function stop aagum.
- Rest parameter multiple arguments-ah array-ah collect pannum.

> **Practical Task 1:** Create a function that accepts two numbers as parameters and returns their sum. Call it with different values.

> **Practical Task 2:** Create a bill calculator function that accepts product prices, calculates the total and returns the result to the caller.

CHAPTER 19

Anonymous Function

SYNTAX · PARAMETERS · RETURN · USES · COMPARISON

Anonymous Function na name illama create pannura function. Usually, function-ku name kudukama create pannuvom.

Simple-ah sonna, oru function-ku name irukkadhu. Andha function-ah oru variable-la store panni use pannalam.

1. What is an Anonymous Function?

Normal function:

```
function greet() {
    console.log("Hello!");
}
greet();
```

Output:

```
Hello!
```

Anonymous function:

```
var greet = function() {
    console.log("Hello!");
};
greet();
```

Output:

```
Hello!
```

Explanation:

- function() - Function-ku name kudukkala.
- greet - Function-ah store pannura variable.
- greet() - Function-ah call pannurathu.

Inga greet function name illa; adhu function-ah store pannura variable.

2. Anonymous Function Syntax

```
function() {
    // Function Body
}
```

Indha function-ku name illa. Usually, idhai oru variable-la assign panni use pannuvom.

3. Anonymous Function with Parameters

Anonymous function-kum parameters pass pannalam.

```
var add = function(a, b) {
    console.log(a + b);
};
add(10, 20);
```

Output:

```
30
```

Explanation:

1. a, b - Parameters.
2. 10, 20 - Arguments.
3. a + b - Addition operation.
4. add(10, 20) - Function call.

4. Anonymous Function with Return

```
var multiply = function(a, b) {
    return a * b;
};
var result = multiply(5, 4);
console.log(result);
```

Output:

```
20
```

```
return use panni function result-ah outside-ku send pannalam.
```

5. Uses of Anonymous Function

1. Function as a Value

Anonymous function-ah variable-la store pannalam.

```
var greet = function() {
    console.log("Welcome!");
};
greet();
```

Output:

```
Welcome!
2. Callback Function
```

Oru function-kulla innoru function-ah argument-ah pass pannurathukku anonymous function use pannalam.

```
setTimeout(function() {
    console.log("Hello JavaScript!");
}, 2000);
```

Output (after 2 seconds):

Hello JavaScript!

3. IIFE (Immediately Invoked Function Expression)

IIFE na function create pannadhum immediately execute aaguradhu.

(function() {

```
    console.log("This runs immediately!");
})();
```

Output:

```
This runs immediately!
```

Explanation:

- (function() { }) - Function-ah expression-ah wrap pannirukkom.
- () - Function-ah immediately call pannuthu.
- So, separate-ah function call panna thevai illa.

6. Normal Function vs Anonymous Function

Normal Function	Anonymous Function
Function name irukkum	Function name irukkadhu
function greet()	function()
Name use panni call pannalam	Usually variable moolama call pannuvom
Can be called by its name	Can be stored and passed as a value

7. Important Points

1. Anonymous function-ku name irukkadhu.
2. Anonymous function-ah variable-la store pannalam.
3. Parameters and return use pannalam.
4. Callback functions-la use pannalam.
5. IIFE create panna use pannalam.
6. Function expression-ah assign pannina, variable assign aagurathukku munnaadi call panna mudiyadhu.

CHAPTER 20

Arrow Function

SYNTAX · NO ARGUMENT · ONE ARGUMENT · RETURN · RULES

Arrow Function na JavaScript-la function-ah short-ah write pannura oru method.

- ES6 (2015) version-la introduce pannanga.

- Idhu normal function-oda shorter syntax.
- Arrow function-ku separate name irukkadhu; usually variable-la assign pannuvom.
- Idhai Lambda Function-num solluvanga.

1. Arrow Function Syntax

```
let myFunction = (arg1, arg2) => {
    // Function Body
};
```

Explanation:

- myFunction – Function-ah store pannura variable.
- arg1, arg2 – Function parameters.
- => – Arrow symbol.
- { } – Function body.

2. Normal Function vs Arrow Function

Before Arrow Function:

```
let hello = function() {
    return "Hello World!";
};
console.log(hello());
```

Output:

```
Hello World!
```

After Arrow Function:

```
let hello = () => {
    return "Hello World!";
};
console.log(hello());
```

Output:

```
Hello World!
```

Explanation:

Normal function-la function keyword use pannuvom.

Arrow function-la function keyword-ku badhila => use pannuvom.

3. Arrow Function with No Argument

Function-ku parameters thevai illana empty parentheses () use pannanum.

```
let greet = () => {
    console.log("Hello");
};
greet();
```

Output:

```
Hello
```

Explanation:

- () – No parameters.
- => – Arrow function.
- console.log() – Message print pannum.
- greet() – Function call.

4. Arrow Function with One Argument

Oru parameter mattum irundha parentheses illamalum write pannalam.

```
let greet = (x) => {
    console.log(x);
};
greet("Hello");
```

Output:

```
Hello
```

Short Syntax:

```
let greet = x => console.log(x);
greet("Welcome");
```

Output:

```
Welcome
```

Note: One parameter irundha (x) or x rendu method-layum write pannalam.

5. Arrow Function with Multiple Arguments

Multiple parameters irundha parentheses compulsory.

```
let add = (a, b) => {
    console.log(a + b);
};
add(10, 20);
```

Output:

```
30
```

Explanation:

- a, b – Parameters.
- 10, 20 – Arguments.
- a + b – Addition.
- add(10, 20) – Function call.

6. Arrow Function with Return

Arrow function-la return use panni result return pannalam.

```
let multiply = (a, b) => {
    return a * b;
};
let result = multiply(5, 4);
console.log(result);
```

Output:

```
20
```

7. Arrow Function with Single Expression

Function body-la single expression mattum irundha {} and return remove pannalam.

Normal Arrow Function:

```
let add = (a, b) => {
    return a + b;
};
console.log(add(10, 20));
```

Short Arrow Function:

```
let add = (a, b) => a + b;
console.log(add(10, 20));
```

Output:

```
30
```

Important: {} illama single expression write pannina JavaScript automatically result-ah return pannum. Idhai Implicit Return nu solluvanga.

8. Arrow Function as an Expression

Condition based-ah different arrow functions assign pannalam.

```
let age = 5;
let welcome = (age < 18)
```

? () => console.log("Baby")

: () => console.log("Adult");

```
welcome();
```

Output:

```
Baby
```

Explanation:

1. age = 5 – Age value 5.
2. age < 18 – Condition check pannum.
3. Condition true na first arrow function select aagum.
4. Condition false na second arrow function select aagum.
5. welcome() – Selected function execute aagum.

9. Arrow Function with Even or Odd

```
let check = (num) => {
    if (num % 2 == 0) {
        return "Even";
    }
    else {
        return "Odd";
    }
};
console.log(check(10));
console.log(check(7));
```

Output:

```
Even
Odd
```

10. Important Rules of Arrow Function

1. Arrow functions ES6-la introduce pannappattadhu.
2. => symbol arrow function-ah represent pannum.
3. No parameters na () use pannanum.
4. One parameter na parentheses optional.
5. Multiple parameters na () compulsory.
6. Single expression na implicit return use pannalam.
7. Multiple statements irundha {} use panni, result return panna return write pannanum.
8. Arrow function-ku its own this binding kedaiyadhu; surrounding scope-la irundhu this value edukkum.
9. Arrow functions-ah new use panni constructor-ah call panna mudiyadhu.

11. Normal Function vs Arrow Function

Normal Function	Arrow Function
function keyword use pannum	=> use pannum
Syntax konjam lengthy	Short syntax
Own this irukkum	Own this kedaiyadhu
Constructor-ah use pannalam	Constructor-ah use panna mudiyadhu
arguments object available	Own arguments object kedaiyadhu

> **Practical Task 1:** Create an anonymous function and assign it to a variable. Use it to calculate the area of a rectangle.

> **Practical Task 2:** Rewrite a normal function as an arrow function and use arrow functions to find whether a number is even or odd.

CHAPTER 21

this

GLOBAL · OBJECT METHOD · NORMAL FUNCTION · ARROW · CLASS

JavaScript-la this keyword na current execution context-la irukkura object-ah refer pannum.

Simple-ah sonna, this oda value namma function-ah eppadi call panrom enbadhai poruthu change aagum.

Important: this function-ah refer pannaadhu. Function call aagumbodhu endha object-oda context-la execute aagutho, andha object-ah refer pannum.

1. this in Global Context

Global scope-la this use pannumbodhu browser and Node.js-la different behaviour irukkum.

Browser Example:

```
console.log(this);
```

Browser Output:

Window

Explanation:

Browser-la global context-la this என்பது window object-ah refer pannum.

Node.js Example:

```
console.log(this);
```

Node.js CommonJS Output:

{}

Node.js CommonJS module top-level-la this empty object-ah irukkum. ES Modules-la top-level this undefined.

2. this Inside an Object Method

Object-oda method-kulla this use pannina, andha method-ah call pannura object-ah refer pannum.

Example:

```
const person = {
    name: "Alice",
    greet() {
        console.log(`Hi, I'm ${this.name}`);
    }
};
person.greet();
```

Output:

```
Hi, I'm Alice
```

Explanation:

- person - Object name.
- name - Object property.
- greet() - Object method.
- this.name - Current object-oda name property.

person.greet() nu call pannumbodhu, this என்பது person object-ah refer pannum.

Another Example:

```
const student = {
    name: "Dhamodharan",
    course: "Java Full Stack",
    details() {
        console.log(this.name);
        console.log(this.course);
    }
};
student.details();
```

Output:

```
Dhamodharan
Java Full Stack
```

3. this Inside Normal Function

Normal function-la this value function eppadi call pannapaduthu enbadhai poruthu change aagum.

Example:

```
function display() {
    console.log(this);
}
display();
```

Browser non-strict mode output:

Window

Strict mode output:

undefined

Explanation:

Normal function-ah direct-ah call pannumbodhu:

- Browser non-strict mode-la this → window.
- Strict mode-la this → undefined.

4. this Inside Arrow Functions

Important concept: Arrow function-ku own this kedaiyadhu.

Adhu thanoda outer scope-la irundhu this value-ah inherit pannikkum. Idhai Lexical this nu solluvanga.

Example:

```
let person = {
    name: "Busra",
    show: function() {
        let display = () => {
            console.log(this.name);
        };
        display();
    }
};
person.show();
```

Output:

```
Busra
```

Explanation:

1. person object create pannirukkom.
2. show() normal function.
3. display() arrow function.
4. Arrow function-ku own this illa.
5. Adhu show() function-la irukkura this-ah inherit pannum.
6. person.show() call pannadhala this → person.
7. So this.name → "Busra".

5. Normal Function vs Arrow Function – this

Normal Function:

```
const person = {
    name: "Alex",
    greet: function() {
        console.log(this.name);
    }
};
person.greet();
```

Output:

```
Alex
```

Arrow Function:

```
const person = {
    name: "Alex",
    greet: () => {
        console.log(this.name);
    }
};
person.greet();
```

Output:

```
undefined
```

Reason: Arrow function object-oda this-ah own-ah create pannaadhu. Outer scope-la irukkura this-ah inherit pannum. So object method-ah arrow function use panni define pannuradhu usually correct choice illa.

6. this Inside Class

Class methods-la this current instance object-ah refer pannum.

```
class Student {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    display() {
        console.log(this.name);
        console.log(this.age);
    }
}
let s1 = new Student("Arun", 22);
s1.display();
```

Output:

```
Arun
22
```

Explanation:

- this.name → Current student object's name.
- this.age → Current student object's age.
- s1 → Student class-la create pannina object.

7. this Using Multiple Objects

```
const person1 = {
    name: "Arun",
    greet() {
        console.log(this.name);
    }
};
const person2 = {
    name: "Kumar",
    greet() {
        console.log(this.name);
    }
};
person1.greet();
person2.greet();
```

Output:

```
Arun
Kumar
```

Ovvoru object-um thanoda method call pannumbodhu, this andha object-ah refer pannum.

8. Important Rules – Quick Revision

Where this is used	Meaning
Browser global scope	window
Node.js CommonJS top-level	{}
Object method	Calling object
Normal function	Depends on how function is called
Arrow function	Inherits outer this
Class instance method	Current instance
Class constructor	Newly created object

> **Practical Task 1:** Create a car object with brand and price properties and a method that uses this to display the car details.

> **Practical Task 2:** Create an object with both normal and arrow function methods and demonstrate how the value of this differs.

CHAPTER 22

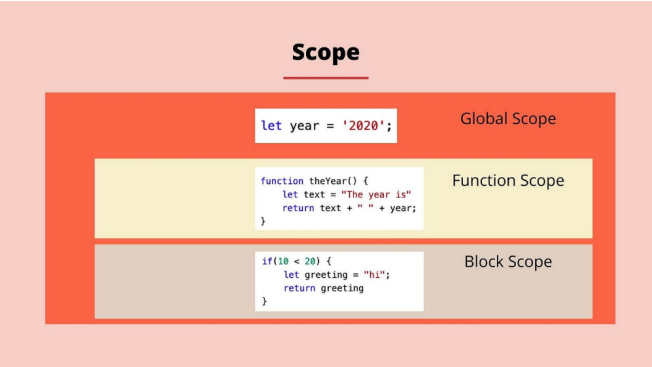
Scope

GLOBAL SCOPE · LOCAL SCOPE · BLOCK SCOPE · SCOPE COMPARISON

Scope na, JavaScript-la variables and functions code-oda endha part-la access panna mudiyum (use panna mudiyum) enbadhai kurikkum.

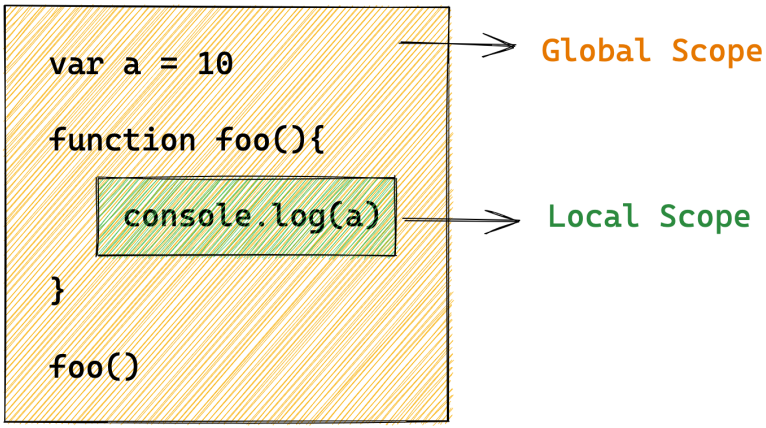
JavaScript-la Variables-ku mainly 2 types of Scope irukku:

1. Global Scope
2. Local Scope



1. Global Scope

1. Oru variable-ah program-oda top-la அல்லது function-ku veliya declare pannina, adhai Global Scope Variable nu solluvom.
2. Programmer define panna specific scope (function/block) kulla illaama irukkura variables, objects and references global scope-la irukkalaam.
3. Global Scope na, JavaScript-oda entire execution environment-ah kurikkum. Global scope-la declare panna variables-ah different parts of the program-la access pannalaam.



Example:

```
let name = "Dhamodharan";
function display() {
    console.log(name);
}
display();
console.log(name);
```

Output:

```
Dhamodharan
Dhamodharan
```

Explanation:

- name variable function-ku veliya declare pannirukkom.
- Adhanala function-kullayum veliya yum access panna mudiyum.
- Idhudhaan Global Scope.

2. Local Scope

1. Oru variable-ah function-kulla declare pannina, adhai Local Scope Variable nu solluvom.
2. Local Variable-ah andha function-kulla mattum dhaan access panna mudiyum.
3. Function-ku veliya அல்லது vera function-la irundhu andha variable-ah access panna try pannina, ReferenceError varum.

Example:

```
function display() {
    let age = 25;
    console.log(age);
}
display();
console.log(age); // Error
```

Output:

```
25
ReferenceError: age is not defined
```

Explanation:

- age variable display() function-kulla declare pannirukkom.
- Function-kulla console.log(age) use pannumbodhu value print aagum.
- Function-ku veliya age-ah access panna try pannumbodhu error varum.

Remember: Local Scope variable-ah declare panna function-kulla mattum dhaan access panna mudiyum.

Difference Between Global Scope and Local Scope

Global Scope	Local Scope
Function-ku veliya declare pannuvom	Function-kulla declare pannuvom
Different parts-la access pannalaam	Declare panna function-kulla mattum access pannalaam
Program-wide access irukkum	Limited access irukkum

Remember: let and const use panni declare panna variables block scope-um follow pannum. var function scope follow pannum. Idhai next topic-la detailed-ah paarkkalaam.

let is Block Scoped

JavaScript-la let keyword use panni declare pannura variable Block Scope follow pannum.

Block Scope na, oru variable-ah { } curly braces-kulla declare pannina, andha block-kulla mattum dhaan access panna mudiyum.

Example: Block-Scoped Variable

```
// Global variable
let a = 'Hello';
function greet() {
    // Local variable
    let b = 'World';
    console.log(a + ' ' + b);
    if (b == 'World') {
        // Block-scoped variable
        let c = 'hello';
        console.log(a + ' ' + b + ' ' + c);
    }
    // Variable c cannot be accessed here
    console.log(a + ' ' + b + ' ' + c);
}
greet();
```

Output

```
Hello World
Hello World hello
Uncaught ReferenceError: c is not defined
```

Explanation

1. a – Global Variable: a oru Global Variable. Idhai program-oda different parts-la access pannalaam.
2. b – Local Variable: b oru Local Variable. Idhai greet() function-kulla mattum access panna mudiyum.
3. c – Block-Scoped Variable: c oru Block-Scoped Variable. Idhai if statement-oda { } block-kulla mattum access panna mudiyum.

Why Error Occurs?

- if block-kulla c variable declare pannirukkom.
- if block-kulla c-ah access pannumbodhu error varaathu.
- if block-ku veliya c-ah access panna try pannumbodhu ReferenceError varum.

Remember:

Variable	Scope	Access
a	Global Scope	Program-wide
b	Function Scope	greet() function-kulla
c	Block Scope	if block-kulla

💡 Important: let and const rendu keywords-um Block Scoped. var mattum block scope follow pannaathu.

CHAPTER 23

Higher-Order Function

FIRST-ORDER FUNCTIONS · RETURNING FUNCTIONS · PASSING FUNCTIONS · DIFFERENCES

1. JavaScript-la Functions-ah strings அல்லது arrays maathiri variables-kullayum assign pannalaam.
2. Functions-ah another function-ku parameters-ah pass pannalaam அல்லது another function-la irundhu return pannavum mudiyum.
3. Higher-Order Function na, oru function-ah argument-ah accept pannura function அல்லது another function-ah return pannura function aagum.

Example 1: First-Order Function

```
function getNum(number) {
    return number;
}
```

Explanation:

1. Inga getNum function use panni oru number-ah receive pannitu, adhe number-ah return pannirukkom.
2. getNum oru normal function. Idhai First-Order Function nu sollalaam.

Example 2: Higher-Order Function (Returning a Function)

```
function twiceNum(number) {
    return function () {
        return number * 2;
    }
}
let twiceTwo = twiceNum(2);
console.log(twiceTwo());
```

Output:

```
4
```

Explanation:

1. twiceNum(2) call pannumbodhu number-oda value 2-ah irukkum.
2. twiceNum function innoru function-ah return pannudhu.
3. Return aana function number * 2 calculate pannum.
4. twiceTwo() call pannumbodhu 2 * 2 = 4 nu result kidaikkum.
5. Oru function innoru function-ah return pannuradhala twiceNum oru Higher-Order Function aagum.

Example 3: Passing a Function as an Argument

```
function twiceNum(number) {
    number();
    console.log("Hi this is twiceNum function");
}
function dummy() {
    console.log("Hi I am outside function");
}
twiceNum(dummy);
```

Output:

```
Hi I am outside function
Hi this is twiceNum function
```

Explanation:

- dummy() oru normal function.
- twiceNum(dummy) nu call pannumbodhu dummy function-ah argument-ah pass pannirukkom.
- twiceNum-la number() nu call pannumbodhu dummy() function execute aagum.
- First dummy()-oda message print aagum.
- Adhukku appuram twiceNum-oda message print aagum.

Important Difference

First-Order Function	Higher-Order Function
Normal function	Another function-ah accept or return pannum
Function-ah argument-ah accept panna vendiya avasiyam illa	Function-ah argument-ah accept pannalaam
Example: getNum()	Example: twiceNum()

Remember: Oru function innoru function-ah argument-ah accept panninaalum, innoru function-ah return panninaalum adhai Higher-Order Function nu solluvom.

Real-time examples: map(), filter(), reduce(), forEach() ellame JavaScript-la Higher-Order Functions.

Scope and Higher-Order Functions

> **Practical Task 1:** Create global, function-local and block-scoped variables using var, let and const. Test where each variable can be accessed.

> **Practical Task 2:** Create a higher-order function that accepts another function as an argument and applies it to two numbers.

CHAPTER 24

OOPs

ENCAPSULATION · ABSTRACTION · INHERITANCE · POLYMORPHISM · EXAMPLE

OOPs stands for Object-Oriented Programming.

JavaScript-la OOPs na, program-ah objects and classes use panni organize panra programming concept.

Simple-ah sollanumna, real-world things-ah programming-la objects madhiri represent panradhu dhaan OOPs.

Real-time Example:

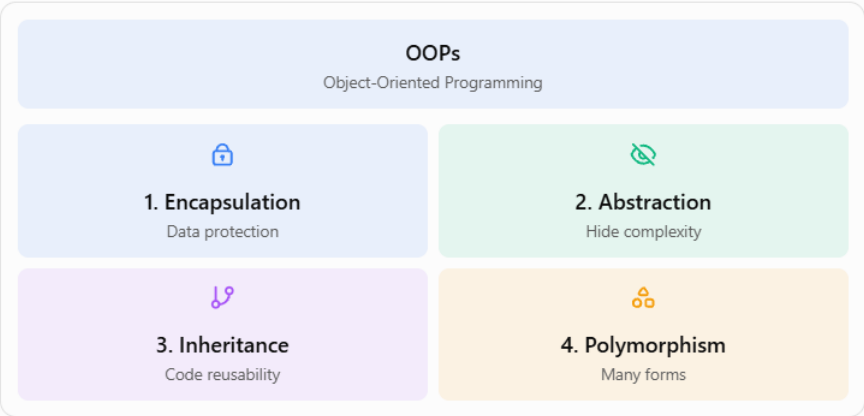
Oru Car-ah consider pannunga.

- Properties: Brand, Color, Price, Model.
- Methods: Start(), Stop(), Drive().

Inga Car is an object. Idha create panna Class use pannalam.

1. Four Main Concepts of OOPs

JavaScript-la OOPs-oda 4 main concepts irukku:



2. Encapsulation

Definition: Encapsulation means wrapping data and methods together into a single unit (class).

Simple-ah sollanumna, data and functions-ah oru class-kulla combine panradhu Encapsulation.

Real-time Example: ATM Machine

ATM-la namma account balance check pannalam. Aana direct-ah internal bank data-ah access panna mudiyadhu.

JavaScript Example:

```
class BankAccount {
    #balance = 5000;
    getBalance() {
        console.log(this.#balance);
    }
}
let account = new BankAccount();
account.getBalance();
```

Screen Output:

```
5000
```

Explanation:

- BankAccount – Class.
- #balance – Private property.
- getBalance() – Balance access panna use panra method.
- # symbol – Private field define panna use aagum.

Private property-ah class-ku veliya direct-ah access panna mudiyadhu.

```
console.log(account.#balance);
```

Output: SyntaxError (private field access outside the class).

3. Abstraction

Definition: Abstraction means hiding internal implementation details and showing only essential features to the user.

Simple-ah sollanumna, eppadi work aagudhu nu theriyama, adha eppadi use panradhu mattum therinja podhum.

Real-time Example: Car

Car drive pannumbodhu:

- Accelerator press panrom.
- Brake press panrom.
- Steering use panrom.

But engine inside epdi work aagudhu nu namakku theriyanum avasiyam illa.

JavaScript Example:

```
class Car {
    start() {
        this.#startEngine();
        console.log("Car Started");
    }
    #startEngine() {
        console.log("Engine Started");
    }
}
let c1 = new Car();
c1.start();
```

Screen Output:

```
Engine Started
Car Started
```

Explanation:

- start() – User access panra public method.
- #startEngine() – Internal implementation hide pannirukkom.
- User start() mattum call panna podhum.

JavaScript-la private methods use panni implementation details hide pannalam.

4. Inheritance

Definition: Inheritance means one class acquiring the properties and methods of another class.

Simple-ah sollanumna, Parent class-la irukkura methods and properties-ah Child class reuse panradhu.

Real-time Example:

- Parent: Animal
- Child: Dog

Dog is an Animal. So Animal-oda common features-ah Dog inherit pannalam.

JavaScript Example:

```
class Animal {
    eat() {
        console.log("Animal is eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Dog is barking");
    }
}
let d1 = new Dog();
d1.eat();
d1.bark();
```

Screen Output:

```
Animal is eating
Dog is barking
```

Explanation:

- Animal – Parent class.
- Dog – Child class.
- extends – Inheritance implement panna use panrom.
- d1.eat() – Parent method.
- d1.bark() – Child method.

Important: Inheritance code reusability-ku help pannum.

5. Polymorphism

Definition: Polymorphism means one thing having many forms.

Simple-ah sollanumna, same method name different classes-la different behavior perform panradhu.

Real-time Example:

Sound nu eduthukonga.

- Dog → Barking
- Cat → Meowing
- Cow → Mooing

Same sound() method, but different outputs.

JavaScript Example:

```
class Animal {
    sound() {
        console.log("Animal makes sound");
    }
}
class Dog extends Animal {
    sound() {
        console.log("Dog barks");
    }
}
class Cat extends Animal {
    sound() {
        console.log("Cat meows");
    }
}
let a1 = new Dog();
let a2 = new Cat();
a1.sound();
a2.sound();
```

Screen Output:

```
Dog barks
Cat meows
```

Explanation:

- Animal – Parent class.
- Dog and Cat – Child classes.
- Rendu class-layum sound() method irukku.
- Aana output different-ah varudhu.

Idhudhaan Method Overriding.

6. Difference Between Four OOPs Concepts

Concept	Meaning	Main Purpose
Encapsulation	Wrapping data and methods	Data protection
Abstraction	Hiding implementation details	Reduce complexity
Inheritance	Acquiring parent features	Code reusability
Polymorphism	One method, many forms	Flexibility

7. Real-time OOPs Example – Employee Management

Indha example-la class, constructor, encapsulation, inheritance and polymorphism concepts use pannirukkom.

```
class Employee {
    constructor(name) {
        this.name = name;
    }
    work() {
        console.log(this.name, "is working");
    }
}
class Developer extends Employee {
    work() {
        console.log(this.name, "is developing software");
    }
}
class Tester extends Employee {
    work() {
        console.log(this.name, "is testing software");
    }
}
let emp1 = new Developer("Dhamodharan");
let emp2 = new Tester("Arun");
emp1.work();
emp2.work();
```

Screen Output:

```
Dhamodharan is developing software
Arun is testing software
```

CHAPTER 25

Access Modifiers

PUBLIC · PRIVATE · PROTECTED · COMPARISON · BANK ACCOUNT

Access Modifiers na class-la irukkura properties and methods-ah enga irundhu access panna mudiyum nu control panra concept.

Simple-ah sollanumna, oru class-la irukkura data-ah yaar access pannalam, yaar access panna mudiyadhu nu control panradhu Access Modifiers.

JavaScript-la mainly 3 access levels pathi padippom:

1. Public
2. Private
3. Protected (JavaScript-la direct built-in protected keyword kedaiyadhu)

1. Public Access Modifier

Definition: Public properties and methods can be accessed from anywhere.

Simple-ah sollanumna, Public members-ah class-kullayum class-ku veliya irundhum access pannalam.

Example:

```
class Student {
    name = "Dhamodharan";
    display() {
        console.log(this.name);
    }
}
let s1 = new Student();
console.log(s1.name);
s1.display();
```

Screen Output:

```
Dhamodharan
Dhamodharan
```

Explanation:

- name – Public property.
- display() – Public method.
- s1.name – Class-ku veliya irundhu access panrom.
- s1.display() – Method-ah veliya irundhu call panrom.

Remember: Public members can be accessed anywhere.

2. Private Access Modifier

Definition: Private properties and methods can only be accessed inside the class where they are declared.

Simple-ah sollanumna, Private members-ah class-kulla mattum dhaan access panna mudiyum. Class-ku veliya direct-ah access panna mudiyadhu.

JavaScript-la # symbol use pannuvom.

Example:

```
class Bank {
    #balance = 5000;
    display() {
        console.log(this.#balance);
    }
}
let b1 = new Bank();
b1.display();
```

Screen Output:

```
5000
```

Outside access try pannina:

```
console.log(b1.#balance);
```

Screen Output:

```
SyntaxError
```

Explanation:

- #balance – Private property.
- display() – Private property-ah class-kulla access panra public method.
- b1.#balance – Outside access panna mudiyadhu.

Remember: Private members are accessible only inside their declaring class.

3. Protected Access Modifier

Definition: Protected members are generally accessible within the same class and its child classes, but not directly outside the class.

Simple-ah sollanumna, Parent class and Child class use pannalam. Aana outside-la direct-ah access panna mudiyadhu.

Important: JavaScript-la protected nu direct keyword kedaiyadhu. Convention-ah _ prefix use pannuvanga. Aana _ use pannina mattum actual protection kedaiyadhu.

Example:

```
class Employee {
    _salary = 25000;
}
class Developer extends Employee {
    display() {
        console.log(this._salary);
    }
}
let d1 = new Developer();
d1.display();
```

Screen Output:

```
25000
```

Outside access:

```
console.log(d1._salary);
```

Screen Output:

```
25000
```

Inga outside-la kooda access aagudhu. Yenna _salary is only a naming convention, actual protected access modifier illa.

4. Difference Between Public, Private and Protected

Feature	Public	Private	Protected
Class inside	Yes	Yes	Yes (convention)
Outside class	Yes	No	Yes (convention)
Child class	Yes	No direct access	Yes (convention)
Syntax	name	#name	_name convention

5. Real-Time Example – Bank Account

```
class BankAccount {
    accountHolder = "Dhamodharan";
    #balance = 10000;
    display() {
        console.log("Name:", this.accountHolder);
        console.log("Balance:", this.#balance);
    }
}
let account = new BankAccount();
account.display();
console.log(account.accountHolder);
```

Screen Output:

```
Name: Dhamodharan
Balance: 10000
Dhamodharan
```

Inga:

- accountHolder – Public.
- #balance – Private.
- display() – Public method.

OOP Concepts and Access Modifiers

> **Practical Task 1:** Create a BankAccount class that demonstrates encapsulation by keeping the balance private and providing deposit and withdraw methods.

> **Practical Task 2:** Create an employee example that demonstrates abstraction and polymorphism, and compare public and private class fields.

CHAPTER 26

Events

EVENT SYNTAX · ONCHANGE · ONCLICK · ONMOUSEOVER · ONKEYDOWN · DELEGATION

Event na, user website-la oru action perform pannumbodhu, adhukku response-ah JavaScript execute aaguradhu.

Simple-ah sollanumna, user oru button click pannuradhu, keyboard key press pannuradhu, mouse-ah move pannuradhu, input value change pannuradhu ellame Events.

Real-time Examples:

- Button click pannina message display aaguradhu.
- Input box-la value change pannina alert varuradhu.
- Mouse-ah text mela kondu pona text color change aaguradhu.
- Keyboard key press pannina background color change aaguradhu.

Event Syntax

HTML element-kulla event attribute use panni JavaScript function call pannalaam.

Single Quotes:

```
<element event='some JavaScript'>
```

Double Quotes:

```
<element event="some JavaScript">
```

Example:

```
<button onclick="alert('Hello')">Click Me</button>
```

Button click pannumbodhu Hello nu alert message varum.

Common HTML Events

Event	Explanation
onchange	HTML element-oda value change aagumbodhu
onclick	HTML element-ah click pannumbodhu
onmouseover	Mouse pointer element mela pogumbodhu
onmouseout	Mouse pointer element-ah vittu veliya pogumbodhu
onkeydown	Keyboard key press pannumbodhu

1. onchange Event

onchange event na, HTML element-oda value change aagi, andha change commit aagumbodhu event trigger aagum.

Example:

```
<!DOCTYPE html>
<html>
<body>
<p>Enter some text:</p>
<input type="text" value="Hello"
onchange="myFunction(this.value)">
<script>
function myFunction(val) {
    alert("New value is: " + val);
}
</script>
</body>
</html>
```

Explanation:

- Input box-la already Hello nu irukkum.
- Adhai edit panni Welcome nu type pannunga.
- Input box-la irundhu veliya click pannunga.
- onchange event trigger aagum.
- this.value use panni current input value-ah function-ku pass pannuvom.
- Alert-la New value is: Welcome nu display aagum.

Output:

```
New value is: Welcome
```

Simple Example: Name input box-la name change pannumbodhu alert message show aaguradhu.

2. onclick Event

onclick event na, user oru HTML element-ah click pannumbodhu JavaScript code execute aagum.

1. Button click pannumbodhu function execute pannalaam.
2. Form submit pannumbodhu action perform pannalaam.
3. Webpage content-ah change pannalaam.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript HTML Events</h1>
<button onclick="document.getElementById('demo').innerHTML=Date()">
    The time is?
</button>
<p id="demo"></p>
</body>
</html>
```

Explanation:

- Button click pannumbodhu onclick event trigger aagum.
- document.getElementById('demo') use panni demo ID irukkura element-ah select pannuvom.
- .innerHTML = Date() use panni current date and time-ah display pannuvom.

Output:

Button click pannina current date and time display aagum.

Simple Example:

```
<button onclick="alert('Button Clicked!')">
```

Click Me

```
</button>
```

Button click pannina Button Clicked! nu alert varum.

3. onmouseover Event

onmouseover event na, user-oda mouse pointer oru HTML element mela enter aagumbodhu event trigger aagum.

Idhai onmouseout event-oda serthu use pannalaam.

- onmouseover → Mouse element mela varumbodhu.
- onmouseout → Mouse element-ah vittu veliya pogumbodhu.

Example:

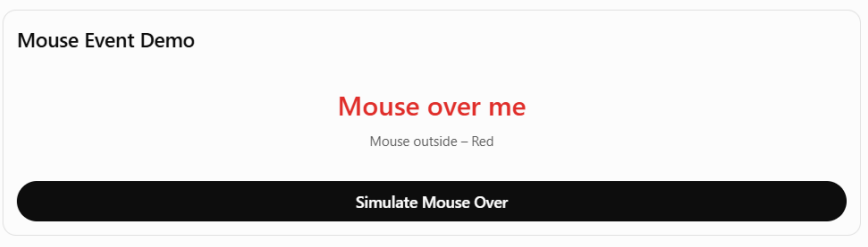
```
<!DOCTYPE html>
<html>
<body>
<h2>HTML DOM EVENTS</h2>
<h1 id="h1"
onmouseover="MouseOver()"
onmouseout="MouseOut()">
    Mouse over me
</h1>
<script>
let h1 = document.getElementById('h1');
function MouseOver() {
    h1.style.color = "green";
}
function MouseOut() {
    h1.style.color = "red";
}
</script>
</body>
</html>
```

Explanation:

- getElementById('h1') use panni heading-ah select pannirukkom.
- Mouse heading mela pogumbodhu MouseOver() function execute aagum.
- h1.style.color = "green" use panni text green color-ah change aagum.
- Mouse heading-ah vittu veliya pogumbodhu MouseOut() function execute aagum.
- Text red color-ah change aagum.

Output:

Mouse Event Demo



4. onkeydown Event

onkeydown event na, user keyboard-la oru key-ah press pannumbodhu event trigger aagum.

Keyboard events-la important-ah 3 types irukku:

1. onkeydown → Keyboard key press pannumbodhu.
2. onkeypress → Key press-related event; idhu deprecated, use panna recommend pannamaattanga.
3. onkeyup → Press panna key-ah release pannumbodhu.

Example:

```
<!DOCTYPE html>
<html>
<body>
<p>
Press and hold a key inside the input field
to set red background color.
Release the key to set green background color.
</p>
<input type="text"
id="demo"
onkeydown="keydownFunction()"
onkeyup="keyupFunction()">
<script>
function keydownFunction() {
    document.getElementById("demo").style.backgroundColor = "red";
}
function keyupFunction() {
    document.getElementById("demo").style.backgroundColor = "green";
}
</script>
</body>
</html>
```

Explanation:

- Input box-la keyboard key press pannumbodhu onkeydown trigger aagum.
- keydownFunction() execute aagum.
- Input box background color red-ah change aagum.
- Key release pannumbodhu onkeyup trigger aagum.
- keyupFunction() execute aagi background color green-ah change aagum.

Output:

- Key press → Red background.
- Key release → Green background.

Important: Event Summary

Event	Eppo execute aagum?	Example
onchange	Value change aagumbodhu	Name input change
onclick	Click pannumbodhu	Button click
onmouseover	Mouse mela varumbodhu	Text color change
onmouseout	Mouse veliya pogumbodhu	Text color change
onkeydown	Key press pannumbodhu	Input background red
onkeyup	Key release pannumbodhu	Input background green

1. Event Delegation

Event Delegation na enna?

JavaScript-la parent element-ku event listener add panni, adhan child elements-la nadakkura events-a handle pannurathuthaan Event Delegation.

Idhu multiple child elements irukkumbothu useful-ah irukkum.

Example

```
<ul id="fruits">
    <li>Apple</li>
    <li>Banana</li>
    <li>Orange</li>
</ul>
<script>
document.getElementById("fruits").addEventListener("click", function(event) {
    if (event.target.tagName === "LI") {
        console.log(event.target.textContent);
    }
});
</script>
```

Screen Output (Browser):



Explanation:

1. ul parent element-ku event listener add pannirukkom.
2. li child element click pannumbothu event parent-ku reach aagum.
3. event.target click panna exact element-a identify pannum.
4. tagName === "LI" check panni fruit name print pannuvom.

Advantages

- Multiple child elements-ku separate event listeners thevai illa.
- Memory usage reduce aagum.
- Dynamically create panna child elements-ukkum work aagum.

2. Event Bubbling and Capturing

JavaScript-la oru HTML element-la event nadakkumbothu, andha event DOM tree-la travel pannum.

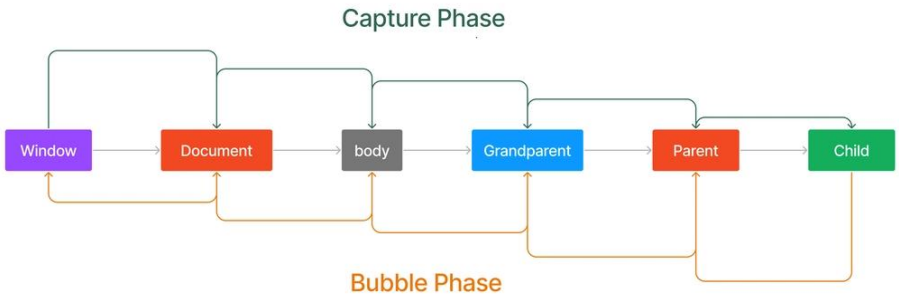
Event travel aagura two main phases:

1. Event Capturing
2. Event Bubbling

A. Event Bubbling

Event Bubbling na enna?

Child element-la event nadakkumbothu, event child-lendhu parent-ku travel aagurathuthaan Event Bubbling.



Example:

```
<div id="parent">
```

Parent

```
    <button id="child">Click Me</button>
</div>
<script>
document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent Clicked");
});
document.getElementById("child").addEventListener("click", function() {
    console.log("Child Clicked");
});
</script>
```

Screen Output (Browser Console):

Child Clicked

Parent Clicked

Explanation:

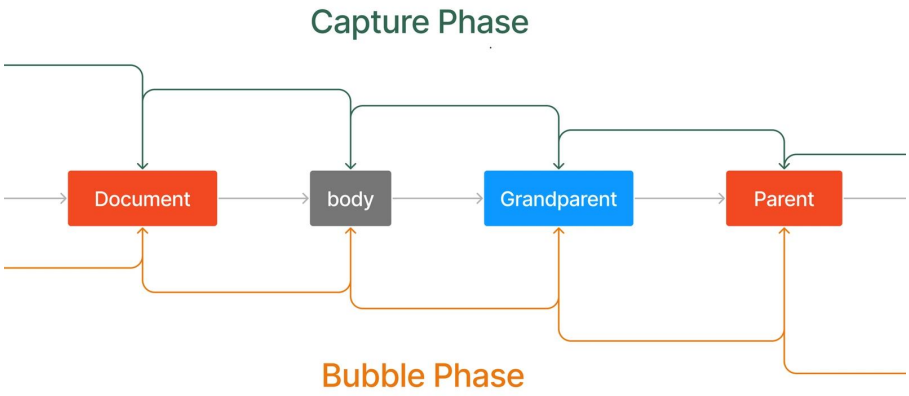
1. First button (child) click aagum.
2. Event parent element-ku travel aagum.
3. So first Child, next Parent print aagum.

Default-ah addEventListener() Event Bubbling phase-la work aagum.

B. Event Capturing

Event Capturing na enna?

Parent element-lendhu child element-ku event travel aagurathuthaan Event Capturing.



Example:

```
<div id="parent">
```

Parent

```
    <button id="child">Click Me</button>
</div>
<script>
document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent Clicked");
}, true);
document.getElementById("child").addEventListener("click", function() {
    console.log("Child Clicked");
}, true);
</script>
```

Screen Output (Browser Console):

Parent Clicked

Child Clicked

Explanation:

1. First parent element-la event execute aagum.
2. Next child element-la event execute aagum.
3. Inga true use pannirukkom. Adhu capturing phase-nu indicate pannum.

3. Event Bubbling vs Event Capturing

Event Bubbling	Event Capturing
Child to Parent	Parent to Child
Default behavior	Explicit-ah enable pannanum
Third argument false	Third argument true
Inner element first	Outer element first

4. Event Propagation Flow



5. stopPropagation()

Event parent-ku or child-ku further travel aagurathai stop panna stopPropagation() use pannuvom.

Example:

```
<div id="parent">
    <button id="child">Click Me</button>
</div>
<script>
document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent Clicked");
});
document.getElementById("child").addEventListener("click", function(event) {
    event.stopPropagation();
    console.log("Child Clicked");
});
</script>
```

Output:

```
Child Clicked
```

Parent event execute aagadhu. Reason: stopPropagation() event bubbling-a stop pannidum.

Hint: Button click pannumbothu first child event listeners execute aagum. Adhukku piragu parent event execute aagum.

> **Practical Task 1:** Create a form with input and button elements. Demonstrate click, change, mouseover and keydown events.

> **Practical Task 2:** Create a parent element with multiple buttons and use event delegation. Demonstrate event bubbling, capturing and stopPropagation().

CHAPTER 27

Inheritance

HOW IT WORKS · SYNTAX · SUPER · TYPES · OVERRIDING

Inheritance is one of the four main concepts of OOPs (Object-Oriented Programming).

Definition: Inheritance means one class acquiring the properties and methods of another class.

Simple-ah sollanumna, Parent class-la irukkura properties and methods-ah Child class use panradhu dhaan Inheritance.

Main Purpose: Code Reusability (Same code-ah multiple times write panna thevai illa).

1. How Inheritance Works

Real-time Example:

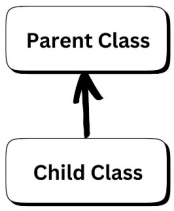
Oru family-ah consider pannunga.

- Parent → Father
- Child → Son

Father-kitta irukkura common properties and methods-ah Son inherit pannalam.

JavaScript-la:

- Parent Class – Base class.
- Child Class – Derived class.
- extends – Parent class-ah inherit panna use pannuvom.



Single Inheritance

2. Inheritance Syntax

```
class Parent {
    // Properties and methods
}
class Child extends Parent {
    // Child properties and methods
}
```

Explanation:

- class Parent – Parent class create panrom.
- class Child – Child class create panrom.
- extends Parent – Parent class-ah Child class inherit panrom.

3. Simple Inheritance Example

```
class Animal {
    eat() {
        console.log("Animal is eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Dog is barking");
    }
}
let d1 = new Dog();
d1.eat();
d1.bark();
```

Screen Output:

```
Animal is eating
Dog is barking
```

Explanation:

1. Animal – Parent class.
2. Dog – Child class.
3. extends Animal – Dog class Animal-ah inherit panrom.
4. eat() – Parent class method.
5. bark() – Child class method.
6. d1.eat() – Parent method-ah call panrom.
7. d1.bark() – Child method-ah call panrom.

Important: Child class object use panni parent class methods-ah access pannalam.

4. Constructor in Inheritance

Parent class and Child class rendu perulayum constructor use pannalam.

Example:

```
class Animal {
    constructor(name) {
        this.name = name;
    }
}
class Dog extends Animal {
    display() {
        console.log("Animal Name:", this.name);
    }
}
let d1 = new Dog("Tommy");
d1.display();
```

Screen Output:

```
Animal Name: Tommy
```

Explanation:

- Parent class-la constructor(name) create pannirukkom.
- Child class extends Animal use panni inherit pannudhu.
- new Dog("Tommy") moolama parent constructor automatically call aagudhu.
- this.name use panni value store panrom.

5. Super Keyword

super keyword use panni Child class-la irundhu Parent class constructor and methods-ah access pannalam.

Example:

```
class Animal {
    constructor(name) {
        this.name = name;
    }
    eat() {
        console.log(this.name, "is eating");
    }
}
class Dog extends Animal {
    constructor(name, color) {
        super(name);
        this.color = color;
    }
    display() {
        console.log("Name:", this.name);
        console.log("Color:", this.color);
    }
}
let d1 = new Dog("Tommy", "Brown");
d1.eat();
d1.display();
```

Screen Output:

```
Tommy is eating
Name: Tommy
Color: Brown
```

Explanation:

- super(name) – Parent class constructor-ah call pannum.
- this.color = color – Child class property.
- d1.eat() – Parent class method.
- d1.display() – Child class method.

Important

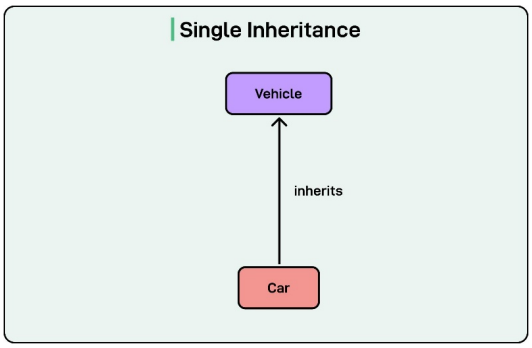
Child class-la constructor define pannina, this use panradhukku munnadi super() call pannanum.

6. Types of Inheritance

OOPs-la commonly 5 types of inheritance pathi padippom.

1. Single Inheritance

One Parent class and One Child class.



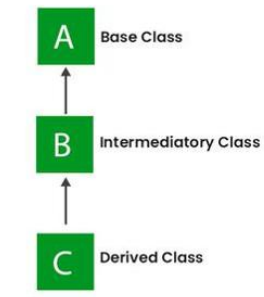

```
class Animal {
    eat() {
        console.log("Eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Barking");
    }
}
let d1 = new Dog();
d1.eat();
d1.bark();
```

Output:

```
Eating
Barking
```

2. Multilevel Inheritance

One class inherits another class, and a third class inherits the second class.



Multilevel Inheritance

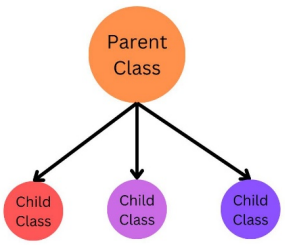
```
class Animal {
    eat() {
        console.log("Eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Barking");
    }
}
class Puppy extends Dog {
    weep() {
        console.log("Puppy is crying");
    }
}
let p1 = new Puppy();
p1.eat();
p1.bark();
p1.weep();
```

Output:

```
Eating
Barking
Puppy is crying
```

3. Hierarchical Inheritance

One Parent class and Multiple Child classes.



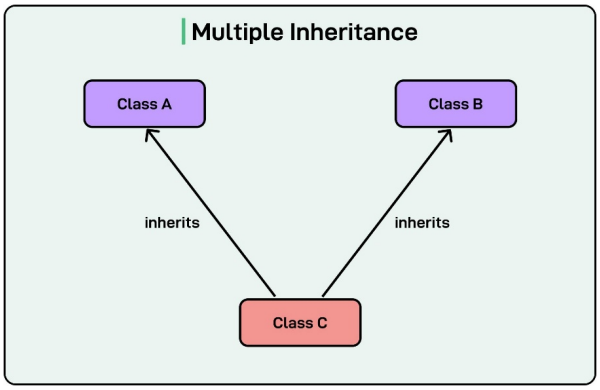
```
class Animal {
    eat() {
        console.log("Eating");
    }
}
class Dog extends Animal {
    bark() {
        console.log("Barking");
    }
}
class Cat extends Animal {
    meow() {
        console.log("Meowing");
    }
}
let d1 = new Dog();
let c1 = new Cat();
d1.eat();
d1.bark();
c1.eat();
c1.meow();
```

Output:

```
Eating
Barking
Eating
Meowing
```

4. Multiple Inheritance

One Child class inherits from multiple Parent classes.



JavaScript class extends directly multiple parent classes-ah support pannaadhu.

```
class Father {
    skills() {
        console.log("Driving");
    }
}
class Mother {
    hobby() {
        console.log("Cooking");
    }
}
class Child extends Father {
    study() {
        console.log("Studying");
    }
}
let c1 = new Child();
c1.skills();
c1.study();
```

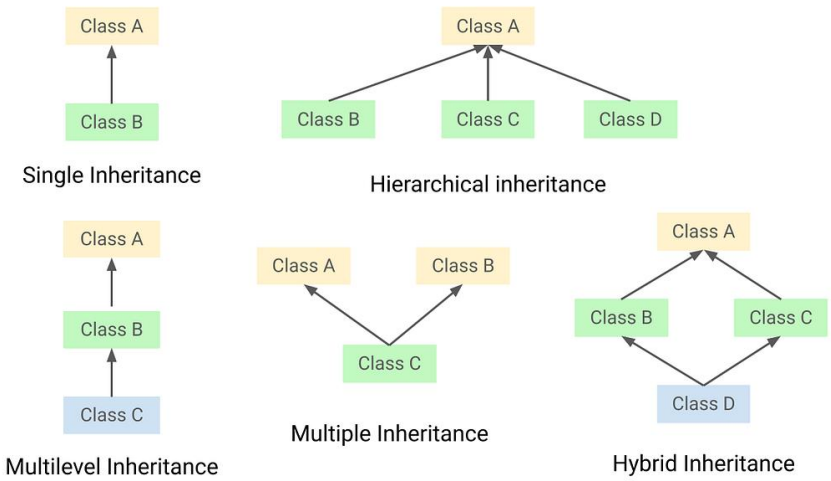
Output:

```
Driving
Studying
```

Inga Child class Father-ah mattum inherit pannudhu. Multiple inheritance-ku JavaScript-la mixins madhiri alternative techniques use pannalam.

5. Hybrid Inheritance

Combination of two or more inheritance types.



JavaScript-la class-based hybrid inheritance direct-ah implement panna limitations irukku. Mixins and composition use panni implement pannalam.

Hybrid Inheritance in JavaScript

Hybrid Inheritance means combining two or more types of inheritance in a single program.

For example:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

JavaScript does not directly support multiple inheritance using classes, but we can achieve hybrid inheritance using extends and other techniques.

Example: Hybrid Inheritance

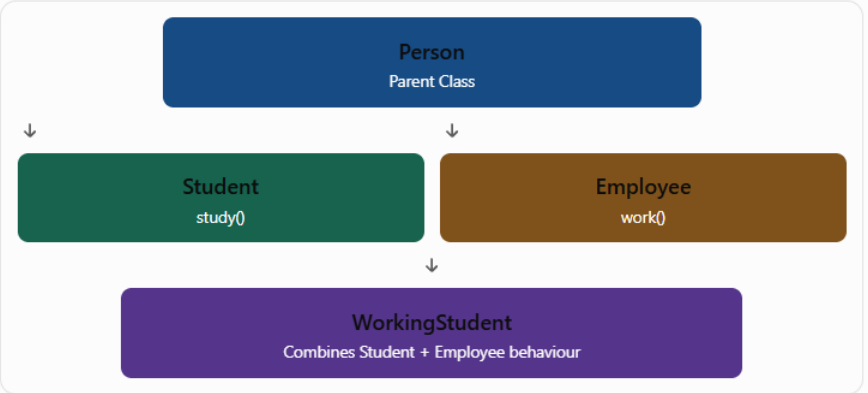
Real-time example: A Person can have two types of roles: Student and Employee. A WorkingStudent can combine both roles.

```
class Person {
    constructor(name) {
        this.name = name;
    }
    displayName() {
        console.log("Name:", this.name);
    }
}
// Single Inheritance
class Student extends Person {
    study() {
        console.log(this.name, "is studying");
    }
}
// Another class derived from Person
class Employee extends Person {
    work() {
        console.log(this.name, "is working");
    }
}
// Combining both behaviours
class WorkingStudent {
    constructor(name) {
        this.student = new Student(name);
        this.employee = new Employee(name);
    }
    showDetails() {
        this.student.displayName();
        this.student.study();
        this.employee.work();
    }
}
let obj = new WorkingStudent("Dhamodharan");
obj.showDetails();
```

Output

```
Name: Dhamodharan
Dhamodharan is studying
Dhamodharan is working
```

Inheritance Structure



Important points:

1. Hybrid inheritance combines multiple inheritance patterns.
2. JavaScript classes support single inheritance through extends.
3. Multiple parent classes cannot be directly extended by one child class.
4. Composition is one way to combine behaviours from multiple classes.

7. Method Overriding

Parent class-la irukkura method-ah Child class-la same name use panni different implementation provide panradhu Method Overriding.

Example:

```
class Animal {
    sound() {
        console.log("Animal makes sound");
    }
}
class Dog extends Animal {
    sound() {
        console.log("Dog barks");
    }
}
let d1 = new Dog();
d1.sound();
```

Screen Output:

```
Dog barks
```

Inga Child class-oda sound() method Parent class-oda sound() method-ah override pannudhu.

8. Advantages of Inheritance

1. Code Reusability – Existing code-ah reuse pannalam.
2. Less Code – Repeated code avoid pannalam.
3. Easy Maintenance – Code manage panradhu easy.
4. Extensibility – Existing class-ah extend panni new features add pannalam.

> **Practical Task 1:** Create a Person class and extend it into a Student class. Use super() to pass values to the parent constructor.

> **Practical Task 2:** Create a multilevel inheritance example with Person, Employee and Manager classes. Override a method in the child class.

CHAPTER 28

DOM

SELECTING ELEMENTS · MANIPULATING · TRAVERSING · MANAGING ATTRIBUTES

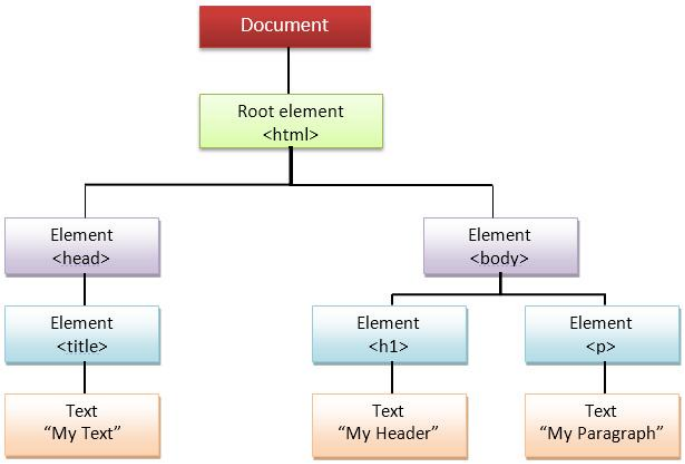
DOM (Document Object Model) na, JavaScript use panni HTML webpage-ah interact pannavum, change pannavum help pannura oru structure.

1. Browser-la webpage load aagumbodhu, HTML code-ah oru Tree-like Structure-ah convert pannum. Idhuku DOM nu peyar.
2. HTML-la irukkura ovvoru element-um DOM-la oru Node-ah represent aagum.
3. JavaScript DOM use panni webpage elements-ah access pannalaam, modify pannalaam.

Think of It Like This (Tree Structure)

Oru webpage-ah tree maathiri imagine pannikkonga.

1. Root: <html> tag.
2. Branches: <head> and <body> tags.
3. Leaves: <p>, , <button> pondra HTML elements.



JavaScript DOM use panni enna pannalaam?

JavaScript DOM use panni webpage-la different changes pannalaam:

1. Changing Text: Webpage-la irukkura text-ah maathalaam.

2. Changing Colors: HTML elements-oda color-ah maathalaam.

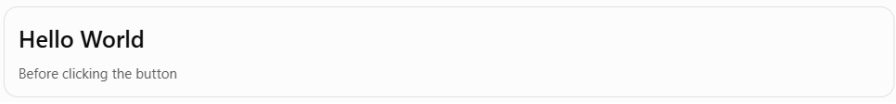
3. Hiding / Showing Elements: HTML elements-ah hide pannalaam illa show pannalaam.

Simple Example

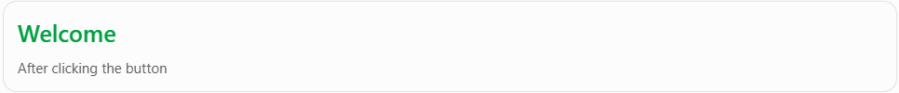
```
<!DOCTYPE html>
<html>
<body>
<h2 id="demo">Hello World</h2>
<button onclick="changeText()">Click Me</button>
<script>
function changeText() {
    document.getElementById("demo").innerHTML = "Welcome";
}
</script>
</body>
</html>
```

Output:

Before button click:



After button click:



Explanation:

- document → Current HTML webpage-ah represent pannum.
- getElementById("demo") → demo ID irukkura element-ah select pannum.
- innerHTML → Element-kulla irukkura content-ah change pannum.
- changeText() → Button click pannumbodhu execute aagura function.

Remember

DOM = HTML + Tree Structure + JavaScript Access

Simple-ah sollanumna, HTML webpage-ah create pannum; DOM andha webpage-ah JavaScript access panna help pannum; JavaScript use panni webpage content, style, elements-ah change pannalaam.

1. Selecting Elements

Selecting Elements na HTML-la irukkura elements-ah JavaScript use panni select pannurathu.

Example: Webpage-la irukkura heading-ah JavaScript-la select panni, adhoda text-ah change pannalaam.

JavaScript-la 5 important methods irukku:

Method	Explanation
getElementById()	ID use panni element-ah select pannum.
getElementsByName()	Name attribute use panni elements-ah select pannum.
getElementsByTagName()	Tag name use panni elements-ah select pannum.
getElementsByClassName()	Class name use panni elements-ah select pannum.
querySelector()	CSS selector use panni first matching element-ah select pannum.

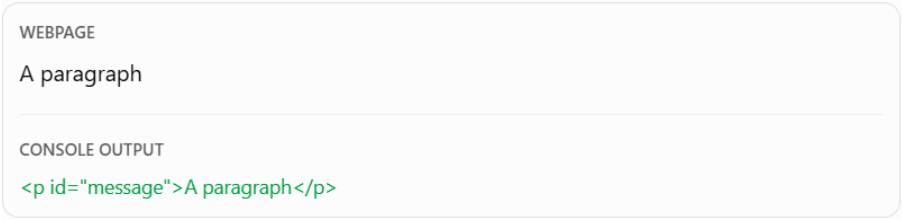
1.1 getElementById()

HTML element-oda id value use panni oru element-ah select pannurathukku getElementById() use pannuvom.

Example:

```
<p id="message">A paragraph</p>
<script>
    const p = document.getElementById("message");
    console.log(p);
</script>
```

Browser screen output:



Explanation:

- document – HTML document-ah represent pannum.
- getElementById("message") – message ID irukkura element-ah select pannum.
- p – selected element-ah store pannum.
- console.log(p) – selected element-ah console-la display pannum.

Note: Same ID multiple elements-ku kudukkurathu valid illa. Appadi irundhaal first matching element-ah return pannum.

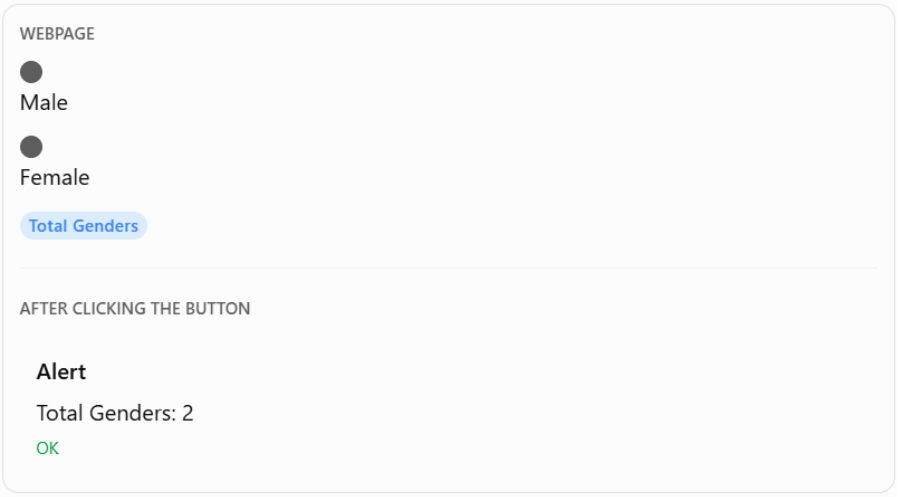
1.2 getElementsByName()

HTML elements-oda name attribute use panni elements-ah select pannurathukku getElementsByName() use pannuvom.

Example:

```
<form>
    Male:
    <input type="radio" name="gender" value="male">
    Female:
    <input type="radio" name="gender" value="female">
    <button type="button" onclick="totalElements()">
        Total Genders
    </button>
</form>
<script>
function totalElements() {
    let allgenders = document.getElementsByName("gender");
    alert("Total Genders: " + allgenders.length);
}
</script>
```

Browser screen output:



Explanation:

- getElementsByName("gender") – gender name irukkura radio buttons-ah select pannum.
- allgenders.length – total elements count-ah return pannum.
- alert() – popup-la result display pannum.

Live NodeList: HTML-la matching elements add/remove aana collection automatic-ah update aagum.

1.3 getElementsByTagName()

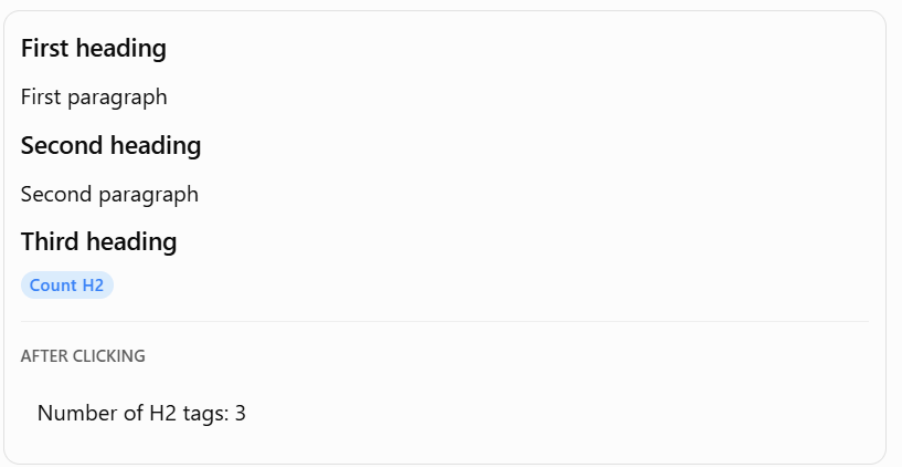
HTML tag name use panni elements-ah select pannurathukku getElementsByTagName() use pannuvom.

Example: Ella h2 tags-ah select pannurathu.

Example:

```
<h2>First heading</h2>
<p>First paragraph</p>
<h2>Second heading</h2>
<p>Second paragraph</p>
<h2>Third heading</h2>
<button id="btnCount">Count H2</button>
<script>
let btn = document.getElementById("btnCount");
btn.addEventListener("click", () => {
    let headings = document.getElementsByTagName("h2");
    alert("Number of H2 tags: " + headings.length);
});
</script>
```

Browser screen output:



Explanation:

- getElementById("btnCount") – button-ah select pannum.
- addEventListener("click", ...) – button click pannumbodhu function execute aagum.
- getElementsByTagName("h2") – ella h2 elements-ah select pannum.
- headings.length – total h2 count-ah return pannum.

1.4 getElementsByClassName()

Class name use panni HTML elements-ah select pannurathukku getElementsByClassName() use pannuvom.

Example:

```
<ul>
    <li class="item">HTML</li>
    <li class="item">CSS</li>
    <li class="item">JavaScript</li>
</ul>
<script>
    let items = document.getElementsByClassName("item");
    console.log(items.length);
</script>
```

Output:

```
3
```

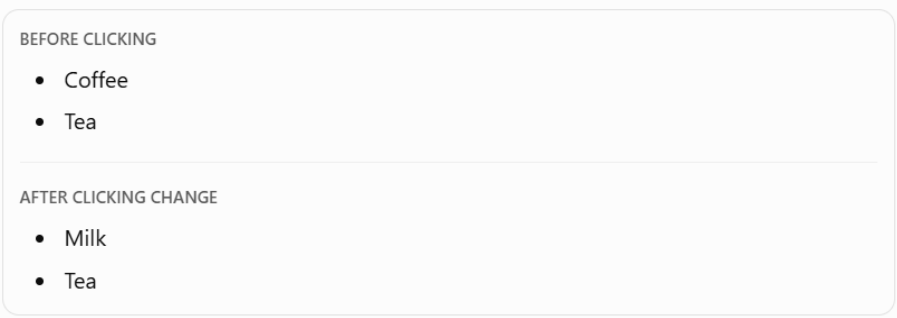
Explanation:

- getElementsByClassName("item") – item class irukkura elements-ah select pannum.
- items.length – total matching elements count pannum.

Text change example:

```
<ul class="example">
    <li class="child">Coffee</li>
    <li class="child">Tea</li>
</ul>
<button onclick="myFunction()">Change</button>
<script>
function myFunction() {
    const list = document.getElementsByClassName("example")[0];
    list.getElementsByClassName("child")[0].innerHTML = "Milk";
}
</script>
```

Browser screen output:



Explanation:

- getElementsByClassName("example")[0] – first example class element-ah select pannum.
- getElementsByClassName("child")[0] – adhukulla first child element-ah select pannum.
- innerHTML = "Milk" – Coffee text-ah Milk-nu change pannum.

1.5 querySelector()

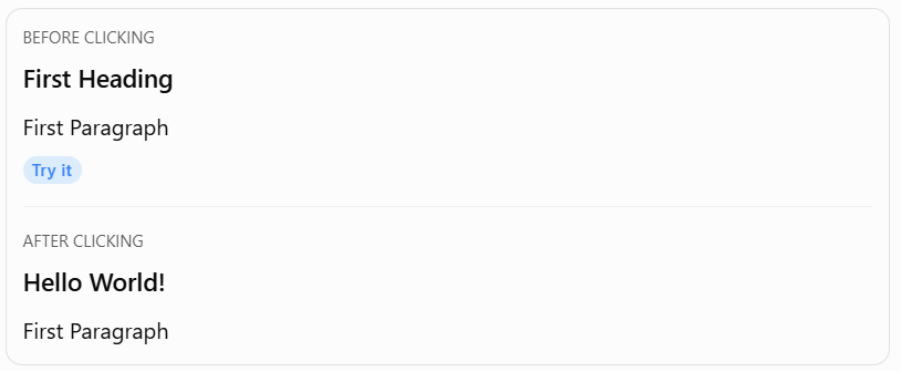
CSS selector use panni HTML element-ah select pannurathukku querySelector() use pannuvom.

Important: Matching elements-la first element mattum return pannum.

Example:

```
<div id="myDIV">
    <h2 class="example">First Heading</h2>
    <p class="example">First Paragraph</p>
</div>
<button onclick="myFunction()">Try it</button>
<script>
function myFunction() {
    let x = document.getElementById("myDIV");
    x.querySelector(".example").innerHTML = "Hello World!";
}
</script>
```

Browser screen output:



Explanation:

- getElementById("myDIV") – myDIV ID irukkura div-ah select pannum.
- x.querySelector(".example") – div-kulla first example class-ah select pannum.
- innerHTML – selected element-oda content-ah change pannum.

Remember:

- .className – class selector.
- #idName – ID selector.
- querySelector() – first matching element.
- querySelectorAll() – matching ella elements.

2. Manipulating Elements

Manipulating Elements na JavaScript use panni HTML elements-ah dynamically change, add, remove pannurathu.

Example: Button click pannumbodhu heading text change aagurathu.

Main operations:

1. Selecting Elements
2. Changing Content
3. Changing Attributes
4. Adding & Removing Elements
5. Handling Classes

2.1 Selecting Elements

Manipulate panna first HTML element-ah select pannanum.

```
// Select by ID
let title = document.getElementById("main-title");
// Select by Class
let boxes = document.getElementsByClassName("box");
// Select by Tag Name
let paragraphs = document.getElementsByTagName("p");
// Select first matching element
let firstBox = document.querySelector(".box");
// Select all matching elements
let allBoxes = document.querySelectorAll(".box");
```

Simple explanation:

- getElementById() – ID base panni select pannum.
- getElementsByClassName() – class base panni select pannum.
- getElementsByTagName() – tag base panni select pannum.
- querySelector() – first matching element select pannum.
- querySelectorAll() – matching ella elements-ah select pannum.

2.2 Changing Text Content

HTML element-kulla irukkura text-ah JavaScript use panni change pannalaam.

Idhuku textContent and innerText use pannuvom.

What is textContent?

textContent na HTML element-kulla irukkura text-ah get or set pannurathukku use pannuvom. HTML tags-ah actual HTML-ah interpret pannaadhu.

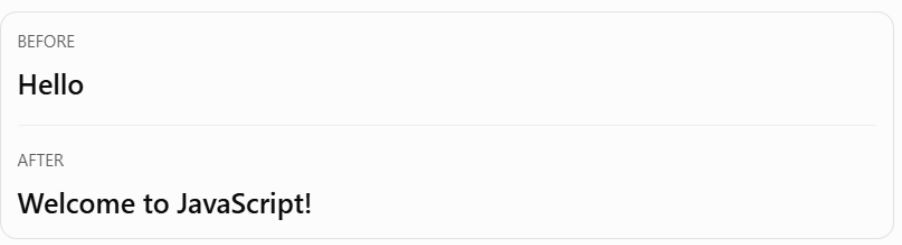
What is innerText?

innerText na webpage-la user-ku visible-ah irukkura text-ah get or set pannurathukku use pannuvom. display: none maadhiri CSS styling-ah consider pannum.

Example:

```
<h1 id="main-title">Hello</h1>
<script>
    document.getElementById("main-title").textContent =
        "Welcome to JavaScript!";
</script>
```

Browser screen output:



Explanation: First heading-ah select pannitu, textContent use panni text-ah change pannirukkom.

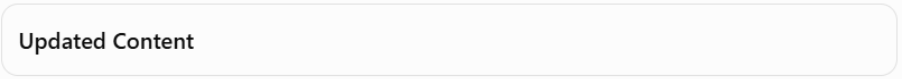
2.3 Changing HTML Content

HTML element-kulla text mattum illaama HTML tags-um add panna innerHTML use pannuvom.

Example:

```
<div id="content"></div>
<script>
    document.getElementById("content").innerHTML =
        "<strong>Updated Content</strong>";
</script>
```

Browser screen output:



Explanation: innerHTML use pannumbodhu tag HTML-ah work aagum. So text bold-ah display aagum.

2.4 Changing Attributes

HTML elements-oda attributes-ah JavaScript use panni change pannalaam.

Examples: src, href, alt.

Example:

```
<img id="image" src="old-image.jpg" alt="Old Image">
<a id="link" href="#">Click Me</a>
<script>
    document.getElementById("image").src = "new-image.jpg";
    document.getElementById("link").href =
        "https://example.com";
</script>
```

Output:

- Image source old-image.jpg lendhu new-image.jpg-ah change aagum.
- Link destination https://example.com-ah change aagum.

Explanation:

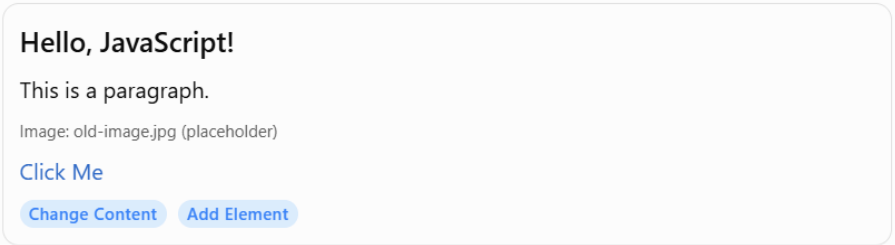
- .src – image source-ah change pannum.
- .href – link destination-ah change pannum.

2.5 Full Code Demonstration

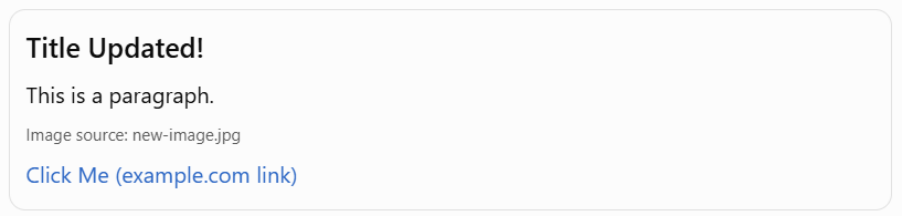
Indha example-la text change, image source change, link change, new element add pannurathu ellame paakkalaam.

```
<!DOCTYPE html>
<html>
<head>
    <title>DOM Manipulation</title>
</head>
<body>
    <h1 id="main-title">Hello, JavaScript!</h1>
    <p id="content">This is a paragraph.</p>
    <img id="image" src="old-image.jpg" alt="Old Image">
    <a id="link" href="#">Click Me</a>
    <div id="container"></div>
    <button onclick="changeContent()">Change Content</button>
    <button onclick="addElement()">Add Element</button>
    <script>
        function changeContent() {
            document.getElementById("main-title").textContent =
                "Title Updated!";
            document.getElementById("image").src =
                "new-image.jpg";
            document.getElementById("link").href =
                "https://example.com";
        }
        function addElement() {
            let newDiv = document.createElement("div");
            newDiv.textContent =
                "New dynamically added div!";
            document.getElementById("container")
                .appendChild(newDiv);
        }
    </script>
</body>
</html>
```

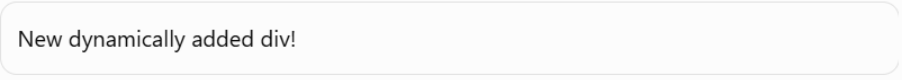
Browser screen output before clicking:



After clicking Change Content:



After clicking Add Element:



Explanation:

1. changeContent() – content and attributes-ah change pannum.
2. createElement("div") – pudhu div element create pannum.
3. textContent – pudhu div-kulla text add pannum.
4. appendChild() – created div-ah container-kulla add pannum.

3. Traversing Elements in JavaScript

Traversing na DOM tree-la irukkura HTML elements-oda relationship use panni oru element-lendhu innoru element-ku navigate pannurathu.

Simple-ah sollanumna, parent, child, sibling elements-ah access pannurathu.

Types of Traversing Elements

1. Parent Node Traversal

Method	Explanation
parentElement	Oru element-oda direct parent-ah select pannum.
closest(selector)	Nearest matching parent/ancestor element-ah find pannum.

2. Child Node Traversal

Method	Explanation
children	Ella child elements-ah select pannum.
firstElementChild	First child element-ah select pannum.
lastElementChild	Last child element-ah select pannum.

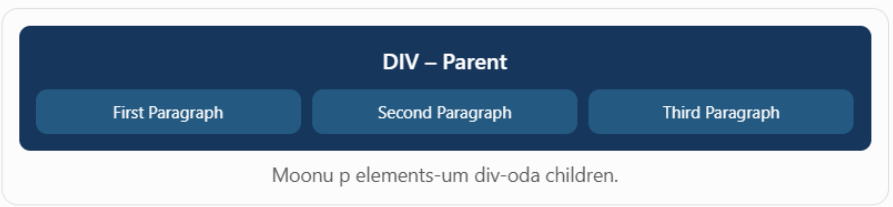
3. Sibling Node Traversal

Method	Explanation
nextElementSibling	Next sibling element-ah select pannum.
previousElementSibling	Previous sibling element-ah select pannum.

Simple Example

```
<div id="parent">
    <p class="child">First Paragraph</p>
    <p class="child">Second Paragraph</p>
    <p class="child">Third Paragraph</p>
</div>
```

DOM screen structure:



JavaScript Example

```
let parentDiv = document.getElementById("parent");
// 1. Parent Traversal
let firstChild = document.querySelector(".child");
console.log(firstChild.parentElement);
// 2. Child Traversal
console.log(parentDiv.children);
console.log(parentDiv.firstElementChild);
console.log(parentDiv.lastElementChild);
// 3. Sibling Traversal
let secondParagraph = firstChild.nextElementSibling;
console.log(secondParagraph);
let firstParagraph =
    secondParagraph.previousElementSibling;
console.log(firstParagraph);
```

Console output (simplified):

```
<div id="parent">...</div>
HTMLCollection(3)
<p class="child">First Paragraph</p>
<p class="child">Third Paragraph</p>
<p class="child">Second Paragraph</p>
<p class="child">First Paragraph</p>
```

Explanation:

- parentElement – First Paragraph-oda parent div-ah return pannum.
- children – div-kulla irukkura 3 p elements-ah return pannum.
- firstElementChild – First Paragraph-ah return pannum.
- lastElementChild – Third Paragraph-ah return pannum.
- nextElementSibling – First Paragraph-ku next-ah irukkura Second Paragraph-ah return pannum.
- previousElementSibling – Second Paragraph-ku previous-ah irukkura First Paragraph-ah return pannum.

4. Managing Attributes in the DOM

Managing Attributes na HTML elements-oda attributes-ah JavaScript use panni access, modify, add, remove pannurathu.

Attributes na HTML elements-ku additional information provide pannura key-value pairs.

Examples:

```
<img id="photo" src="image.jpg" alt="My Photo">
<a href="https://example.com">Visit</a>
```

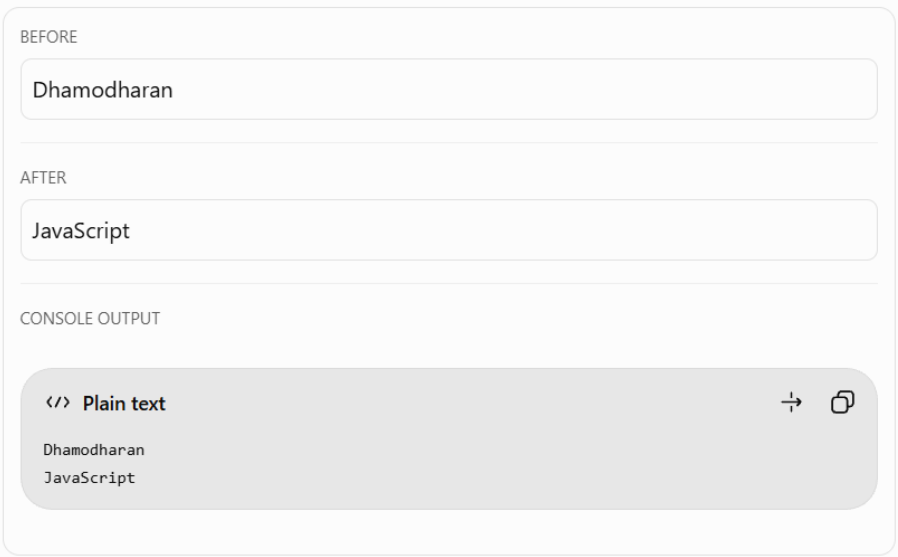
Inga:

- id – attribute.
- src – attribute.
- alt – attribute.
- href – attribute.

Simple Example

```
<input id="username" type="text" value="Dhamodharan">
<script>
    let input = document.getElementById("username");
    console.log(input.value);
    input.value = "JavaScript";
    console.log(input.value);
</script>
```

Browser screen output:



Explanation:

- getElementById() – input element-ah select pannum.
- input.value – current value-ah get pannum.
- input.value = "JavaScript" – input value-ah change pannum.

Quick Revision:

Method / Property	Use
getElementById()	ID base panni select
getElementsByName()	Name base panni select
getElementsByTagName()	Tag base panni select
getElementsByClassName()	Class base panni select
querySelector()	First matching element
querySelectorAll()	All matching elements
textContent	Text change
innerText	Visible text change
innerHTML	HTML content change
createElement()	New element create
appendChild()	Child element add
parentElement	Parent select
children	All child elements
nextElementSibling	Next sibling
previousElementSibling	Previous sibling

> **Practical Task 1:** Create a webpage with a heading, paragraph and image. Select elements using getElementById(), querySelector() and other DOM selection methods, then update their content.

> **Practical Task 2:** Create a dynamic list that allows users to add and remove elements. Also change an element's attributes and traverse its parent and child nodes.

CHAPTER 29

Form

HTML FORMS · GET VS POST · FORMELEMENT · VALIDATION · CHECKBOXES · REVISION

HTML Form na enna? User kitta irundhu information collect panna use panra HTML element dhaan Form.

Examples:

- Login Form
- Registration Form
- Contact Form
- Feedback Form
- Search Form

1. HTML Form Basics

```
<form action="/signup" method="post" id="signup">
</form>
```

Indha code-la 3 important attributes irukku.

Attribute	Explanation
action	Form data enga send aaganum nu specify pannum.
method	Data eppadi send aaganum nu specify pannum.
id	Form-ku unique identification kudukkum.

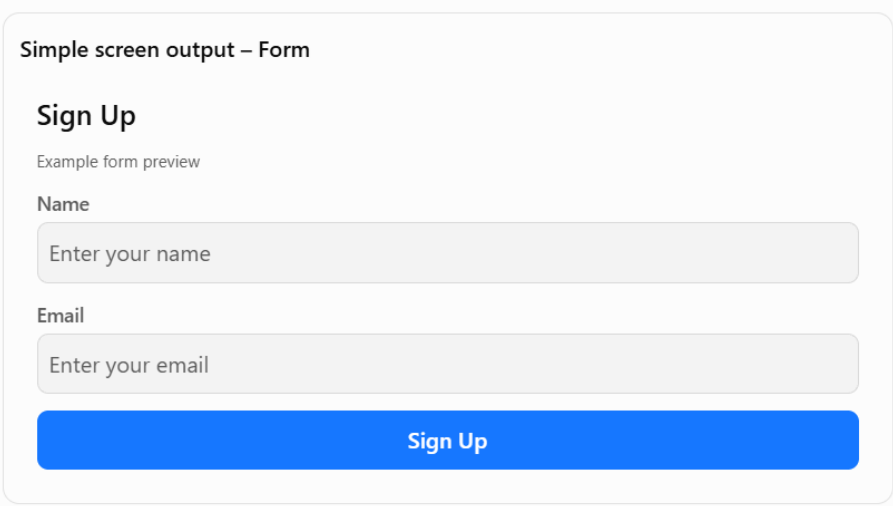
GET vs POST

GET Method:

- Server-la irundhu data retrieve panna use pannuvom.
- URL-la data visible-ah irukkum.
- Example: Search form.

POST Method:

- Server-ku data send panna use pannuvom.
- Data URL-la visible-ah irukkadhu.
- Example: Login, Registration form.



2. HTMLFormElement in JavaScript

JavaScript-la HTML form-ah access panna HTMLFormElement object use aagum.

Idhula important properties and methods irukku.

Properties:

Property	Explanation
action	Form data process aagura URL.
method	GET or POST method.

Methods:

Method	Explanation
submit()	Form-ah submit pannum.
reset()	Form fields-ah clear/reset pannum.

Example:

```
<form id="myForm">
    <input type="text" name="username">
    <button type="reset">Reset</button>
</form>
<script>
let form = document.getElementById("myForm");
console.log(form.action);
console.log(form.method);
</script>
```

Explanation:

- getElementById() → Form-ah select pannum.
- form.action → Form action URL return pannum.
- form.method → Form method return pannum.

3. JavaScript Form Validation

Form Validation na user correct-ah data enter pannirukangala nu check panradhu.

Example:

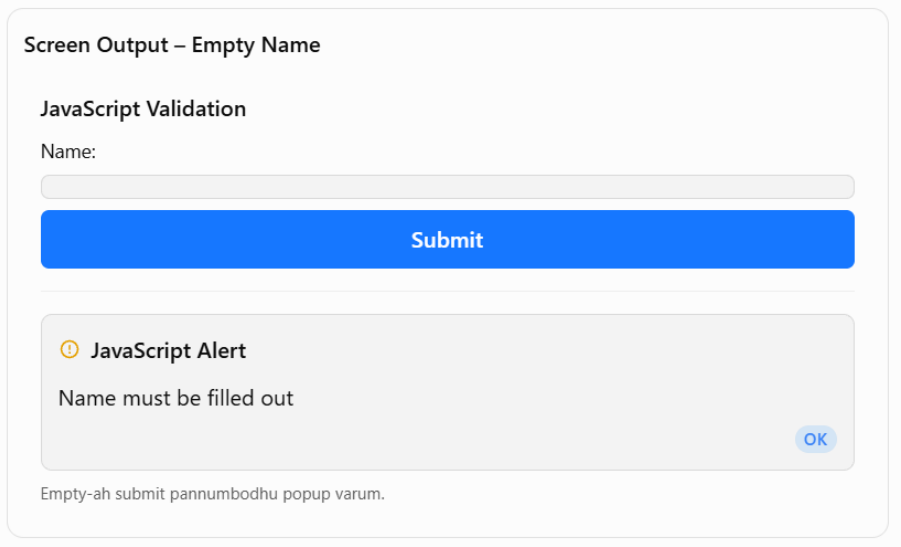
- Name empty-ah irukka?
- Email correct-ah irukka?
- Password enter pannirukangala?

Example 1 – Name Validation

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
    let x = document.forms["myForm"]["fname"].value;
    if (x == "") {
        alert("Name must be filled out");
        return false;
    }
}
</script>
</head>
<body>
<h2>JavaScript Validation</h2>
<form name="myForm"
      onsubmit="return validateForm()">
    Name:
    <input type="text" name="fname">
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

Explanation:

1. document.forms["myForm"] → myForm form-ah select pannum.
2. ["fname"] → fname input-ah select pannum.
3. .value → User enter panna value-ah edukkum.
4. if(x == "") → Input empty-ah irukka nu check pannum.
5. alert() → Popup message display pannum.
6. return false → Form submit aaguradha stop pannum.



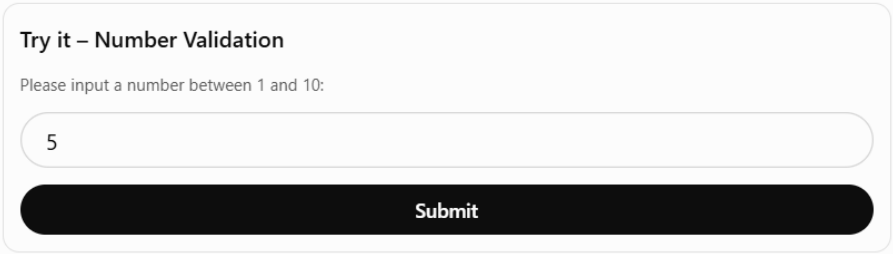
4. Number Validation (1 to 10)

User enter panra number 1 to 10 range-la irukka nu check pannuvom.

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Validation</h2>
<p>Please input a number between 1 and 10:</p>
<input id="numb">
<button type="button" onclick="myFunction()">
Submit
</button>
<p id="demo"></p>
<script>
function myFunction() {
    let x = document.getElementById("numb").value;
    let text;
    if (isNaN(x) || x < 1 || x > 10) {
        text = "Input not valid";
    }
    else {
        text = "Input OK";
    }
    document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

Important concepts:

Code	Meaning
isNaN(x)	x number illaya nu check pannum.
x < 1	1-ku kammiya irukka?
x > 10	10-ku adhigama irukka?
innerHTML	HTML element-kulla output display pannum.



5. Automatic HTML Form Validation – Required

HTML-la JavaScript use pannama kooda basic validation panna mudiyum.

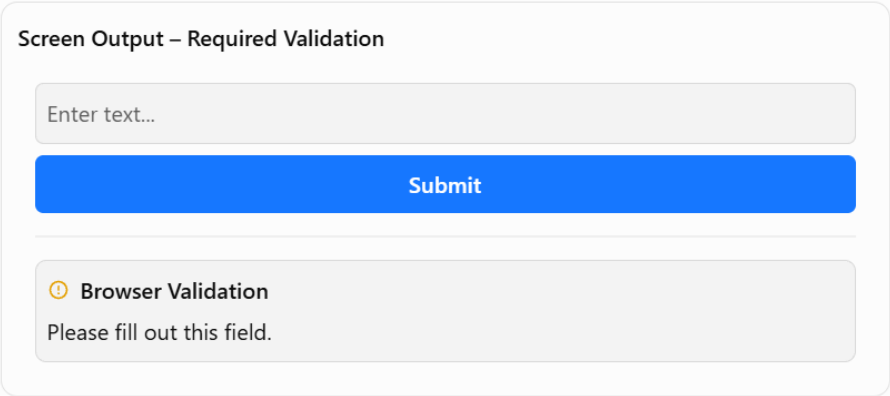
Adhukku required attribute use pannuvom.

```
<form action="/action_page.php" method="post">
    <input type="text" name="fname" required>
    <input type="submit" value="Submit">
</form>
```

Explanation:

required na compulsory-ah fill pannanum.

User input empty-ah irundha browser automatic-ah error message display pannum.



Difference:

- JavaScript Validation → Namale validation logic write pannanum.
- HTML Validation → required, min, max, pattern madhiri attributes use pannalam.

6. JavaScript Checkboxes

Checkbox na multiple options-la namakku pudichadha select panna use pannuvom.

Example:

- Red
- Green
- Blue

Full Code

```
<!DOCTYPE html>
<html>
<body>
<p>Select your favorite colors:</p>
<label>
<input type="checkbox" name="color" value="red">
Red
</label>
<label>
<input type="checkbox" name="color" value="green">
Green
</label>
<label>
<input type="checkbox" name="color" value="blue">
```

```
Blue
</label>
<p>
<button id="btn">Get Selected Colors</button>
</p>
<script>
const btn = document.querySelector('#btn');
btn.addEventListener('click', (event) => {
    let checkboxes = document.querySelectorAll(
        'input[name="color"]:checked'
    );
    let values = [];
    checkboxes.forEach((checkbox) => {
        values.push(checkbox.value);
    });
    alert(values);
});
</script>
</body>
</html>
```

Line-by-line Explanation

1. Checkboxes create panradhu

```
<input type="checkbox" name="color" value="red">
```

- type="checkbox" → Checkbox create pannum.
- name="color" → Checkbox group name.
- value="red" → Checkbox select pannina red value kidaikkum.

2. Button select panradhu

```
const btn = document.querySelector('#btn');
```

querySelector() use panni ID btn irukkura button-ah select pannrom.

3. Click Event

```
btn.addEventListener('click', (event) => {
});
```

Button click pannumbodhu inside irukkura code execute aagum.

4. Checked Checkboxes Select Panradhu

```
let checkboxes = document.querySelectorAll(
    'input[name="color"]:checked'
);
```

Idhu enna pannum?

- querySelectorAll() → Multiple elements select pannum.
- input[name="color"] → Color checkbox group select pannum.
- :checked → User select pannina checkbox mattum select pannum.

5. Empty Array Create Panradhu

```
let values = [];
```

Selected colors store panna empty array create pannrom.

6. forEach Loop

```
checkboxes.forEach((checkbox) => {
    values.push(checkbox.value);
});
```

Selected checkbox ovvondrayum loop pannum.

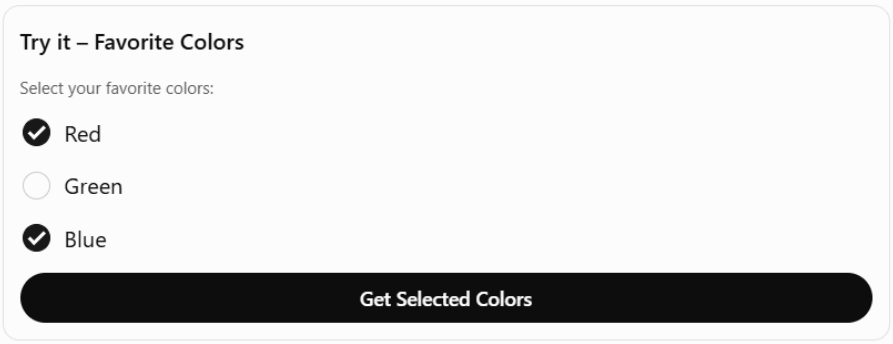
push() use panni selected color-ah array-kulla add pannum.

7. Alert

```
alert(values);
```

Selected colors popup-la display aagum.

Screen Output



7. Important JavaScript Methods – Revision

Method / Property	Use
document.forms	Forms access panna
.value	Input value get panna
.action	Form URL access panna
.method	GET / POST access panna
submit()	Form submit panna
reset()	Form reset panna
alert()	Popup display panna
isNaN()	Number check panna
querySelector()	Single element select panna
querySelectorAll()	Multiple elements select panna
addEventListener()	Event handle panna
forEach()	Array / NodeList loop panna
push()	Array-kulla value add panna
:checked	Selected checkbox identify panna

> **Practical Task 1:** Create a registration form with name, email, age and password fields using suitable HTML input types and validation attributes.

> **Practical Task 2:** Use JavaScript to validate the form, display field-specific error messages and prevent submission when the inputs are invalid.

CHAPTER 30

setInterval() and setTimeout()

TIMERS · SETINTERVAL · CLEARINTERVAL · SETTIMEOUT · CLEARTIMEOUT · REAL-LIFE EXAMPLES

Indha rendu methods-um JavaScript-la Timer Functions nu solluvom.

Simple-ah sonna, oru particular time-ku apram oru function-ah execute panna use pannuvom.

Aana rendu methods-kum difference irukku.

- setInterval() → Repeated-ah execute aagum.
- setTimeout() → One time mattum execute aagum.

1. setInterval() Method

Definition:

setInterval() method na, namma kudukkura time interval-ku oru function-ah repeatedly execute pannum.

Example:

- Every 1 second clock update aaganum.
- Every 3 seconds popup varanum.
- Every 2 seconds background color change aaganum.

Syntax

```
setInterval(function, milliseconds);
```

Parameter	Explanation
function	Execute aaganum function name.
milliseconds	Evlo time interval-la execute aaganum.

Important: 1000 milliseconds = 1 second.

Example 1 – Alert Every 3 Seconds

```
<html>
<body>
<h1>Hello World :)</h1>
<h3>setInterval() Example</h3>
<p>Alert displays every 3 seconds.</p>
<script>
var a = setInterval(fun, 3000);
function fun() {
    alert("Welcome to JavaScript");
}
</script>
</body>
</html>
```

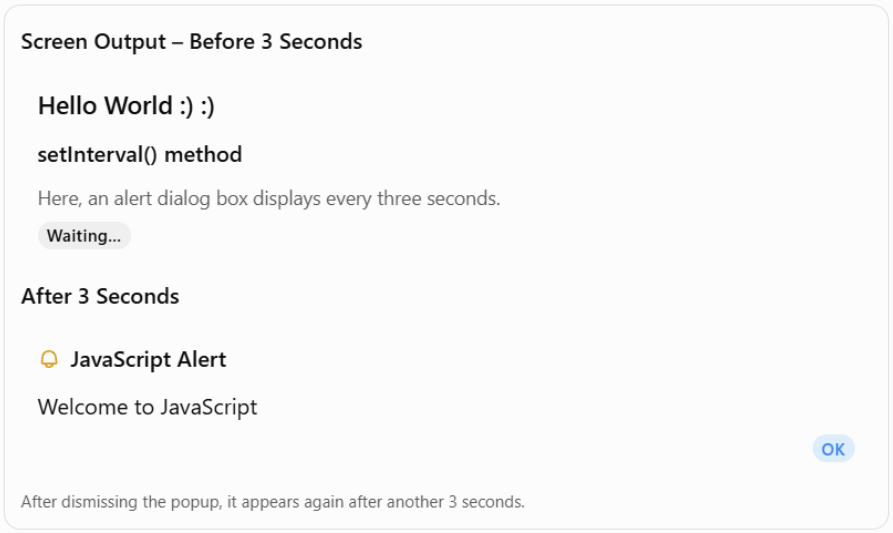
Explanation:

```
var a = setInterval(fun, 3000);
```

- setInterval() → Repeated-ah function execute pannum.
- fun → Execute aaganum function.
- 3000 → 3 seconds.

```
function fun() {
    alert("Welcome to JavaScript");
}
```

Indha function call aagumbodhu alert popup display aagum.



2. clearInterval() Method

setInterval() continuous-ah execute aagite irukkum.

Adha stop panna clearInterval() use pannuvom.

Example:

```
let a = setInterval(fun, 2000);
function fun() {
    console.log("Hello");
}
clearInterval(a);
```

Explanation:

- setInterval() → Timer start pannum.
- a → Timer ID store pannum.
- clearInterval(a) → Timer stop pannum.

Important: clearInterval() call pannumbodhu timer stop aagum. So, indha example-la fun() execute aaga time kidaikkadhu.

3. Background Color Change Using setInterval()

```
<html>
<body>
<h1>Background Color Change</h1>
<script>
var var1 = setInterval(color, 2000);
function color() {
    var var2 = document.body;
    var2.style.backgroundColor =
    var2.style.backgroundColor == "lightblue"
    ? "lightgreen"
    : "lightblue";
}
</script>
</body>
</html>
```

Explanation:

```
var var1 = setInterval(color, 2000);
```

Every 2 seconds color() function execute aagum.

```
var var2 = document.body;
```

Entire webpage body-ah select pannrom.

```
var2.style.backgroundColor
```

Background color-ah change panna use pannrom.

Ternary Operator:

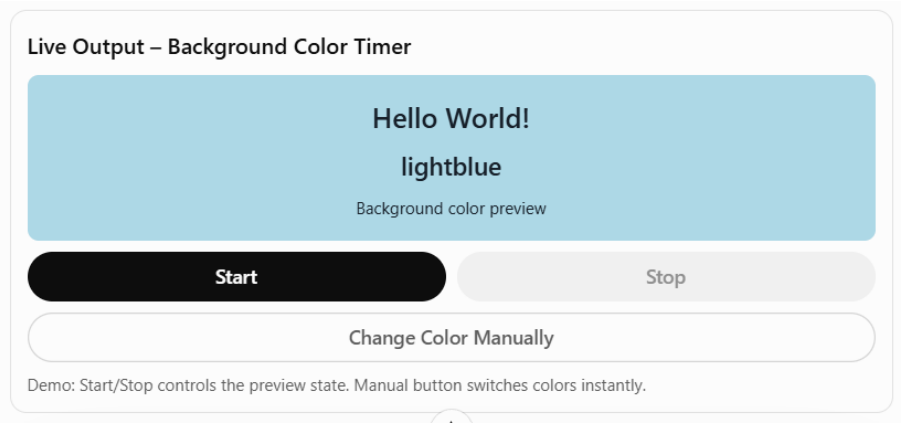
```
condition ? value1 : value2;
```

Indha code-la:

```
var2.style.backgroundColor == "lightblue"
? "lightgreen"
: "lightblue";
```

Meaning:

- Current color lightblue-ah irundha → lightgreen change aagum.
- Illana → lightblue change aagum.



4. setTimeout() Method

Definition:

setTimeout() method na, specified time mudinja piragu oru function-ah execute pannum.

Important: Idhu one time mattum execute aagum.

Syntax

```
setTimeout(function, milliseconds);
```

Parameter	Explanation
function	Execute aaganum function.
milliseconds	Evlo time wait pannanum.

Example

```
function greet() {
    console.log("Hello world");
}
setTimeout(greet, 3000);
console.log("This message is shown first");
```

Explanation:

First:

```
setTimeout(greet, 3000);
```

3 seconds wait pannitu greet() function execute aagum.

Next:

```
console.log("This message is shown first");
```

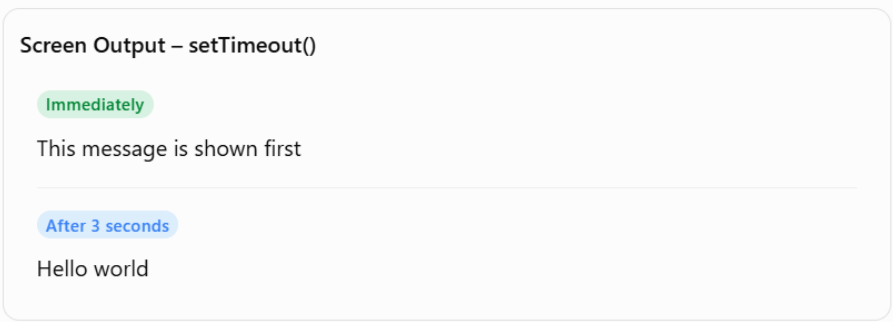
Indha line immediately execute aagum.

Yen?

JavaScript setTimeout() timer-ah schedule pannitu next code-ku poidum. 3 seconds mudinja piragu greet() execute aagum.

Output:

```
This message is shown first
Hello world
```



5. clearTimeout() Method

setTimeout() timer-ah cancel panna clearTimeout() use pannuvom.

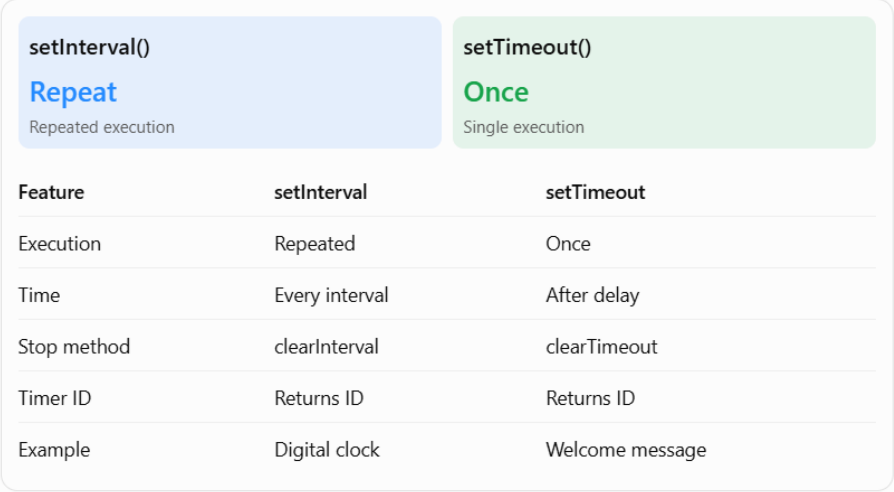
```
let a = setTimeout(greet, 3000);
clearTimeout(a);
function greet() {
    console.log("Hello world");
}
```

Output:

```
// No output
```

Reason: 3 seconds mudiyuradhukku munnadiye timer cancel pannitom.

6. setInterval() vs setTimeout()



Feature	setInterval	setTimeout
Execution	Repeated	Once
Time	Every interval	After delay
Stop method	clearInterval	clearTimeout
Timer ID	Returns ID	Returns ID
Example	Digital clock	Welcome message

7. Real-Life Examples

Requirement	Method
Digital clock update	setInterval()
Countdown timer	setInterval()
Every 5 seconds popup	setInterval()
Show welcome message after 3 seconds	setTimeout()
Show notification after 10 seconds	setTimeout()
Delay an animation	setTimeout()

> **Practical Task 1:** Create a digital clock that updates every second using setInterval() and stops when the user clicks a Stop button.

> **Practical Task 2:** Create a delayed welcome message using setTimeout() and add a Cancel button that uses clearTimeout().

CHAPTER 31

Synchronous vs Asynchronous

SYNCHRONOUS · ASYNCHRONOUS · EXECUTION ORDER · MODERN ASYNC · COMPARISON

JavaScript-la code execute aagura rendu important methods dhaan:

1. Synchronous – Oru task mudinja piragudhaan next task execute aagum.
2. Asynchronous – Oru task wait pannitu irukkumbodhu, JavaScript other tasks-ah execute pannum.

Simple-ah sonna:

- Synchronous = One by One.
- Asynchronous = Wait pannama other work continue pannradhu.

1. Synchronous JavaScript

Definition:

Synchronous na JavaScript code-ah line-by-line execute pannum. Oru line execution complete aana piragudhaan next line execute aagum.

Idhai Blocking Execution nu solluvanga.

Example

```
document.write("Hi");
document.write("<br>");
document.write("Dhamodharan");
document.write("<br>");
document.write("How are you?");
```

Explanation

Step 1:

```
document.write("Hi");
```

First Hi display aagum.

Step 2:

```
document.write("Dhamodharan");
```

Second Dhamodharan display aagum.

Step 3:

```
document.write("How are you?");
```

Third How are you? display aagum.

Screen Output



Real-time Example:

Restaurant-la food order panrom nu vachukonga.

1. First dosa prepare pannuvanga.
2. Dosa ready aana piragu biryani prepare pannuvanga.
3. Biryani ready aana piragu juice prepare pannuvanga.

Oru work mudinja piragudhaan next work start aagudhu.

Idhudhaan Synchronous execution.

2. Asynchronous JavaScript

Definition:

Asynchronous na oru task complete aaguradhukaga JavaScript wait pannama, other tasks-ah continue execute pannum.

Idhai Non-Blocking Execution nu solluvanga.

Example – setTimeout()

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<h2>setTimeout() Example</h2>
<p>Wait 3 seconds for the page to change.</p>
<h1 id="demo"></h1>
<script>
setTimeout(myFunction, 3000);
function myFunction() {
    document.getElementById("demo").innerHTML =
    "Dhamodharan";
}
</script>
</body>
</html>
```

Explanation

Step 1:

```
setTimeout(myFunction, 3000);
```

- setTimeout() → Specified time mudinja piragu function execute pannum.
- myFunction → Execute aaganum function.
- 3000 → 3 seconds.

Step 2:

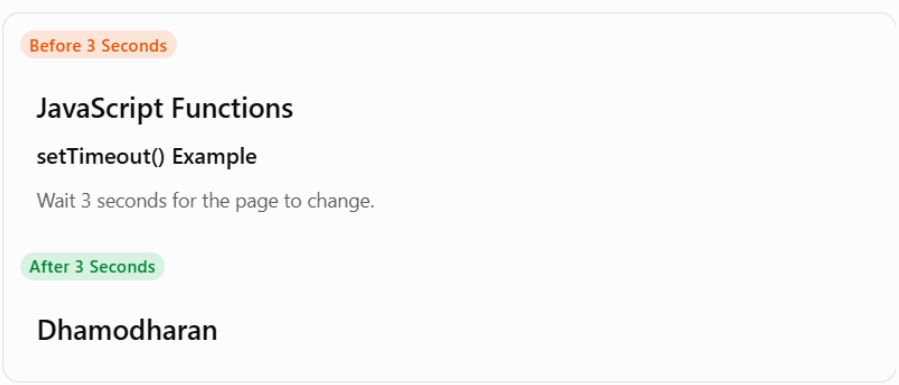
```
function myFunction() {
    document.getElementById("demo").innerHTML =
    "Dhamodharan";
}
```

3 seconds mudinja piragu myFunction() execute aagum.

innerHTML use panni webpage-la Dhamodharan display pannuvom.

Important: 3 seconds wait pannumbodhu JavaScript other code-ah execute pannalaam.

Screen Output



3. Asynchronous – Different Time Example

Ippo ungaloda trainer example-ah paakalaam.

Code

```
// First message
setTimeout(function() {
    document.write("Hi");
    document.write("<br>");
}, 3000);
// Second message
setTimeout(function() {
    document.write("Dhamodharan");
    document.write("<br>");
}, 1000);
// Third message
setTimeout(function() {
    document.write("How are you?");
    document.write("<br>");
}, 2000);
```

Line-by-line Explanation

Function	Delay	Output
First setTimeout	3 seconds	Hi
Second setTimeout	1 second	Dhamodharan
Third setTimeout	2 seconds	How are you?

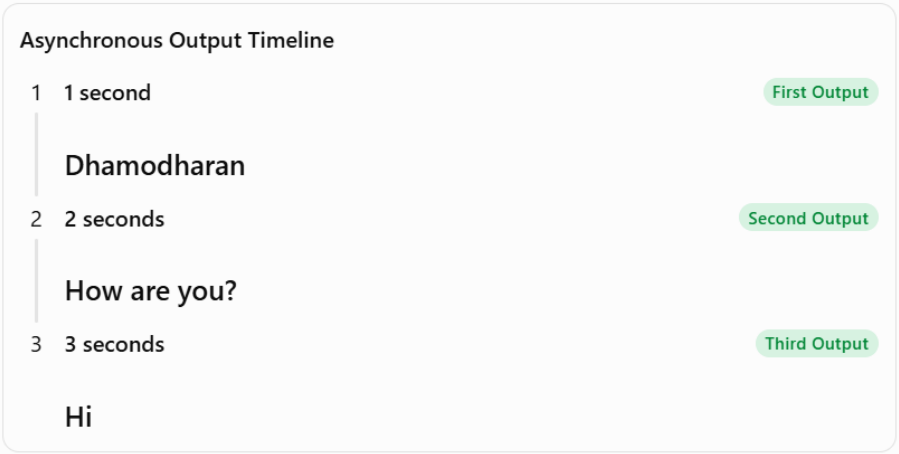
Output order yen change aagudhu?

JavaScript first code-la Hi irukku. Aana adhu 3 seconds delay-la execute aagum.

Second code-la Dhamodharan irukku. Adhu 1 second-la execute aagum.

Third code-la How are you? irukku. Adhu 2 seconds-la execute aagum.

So, execution order:



Output:

```
Dhamodharan
How are you?
Hi
Remember: Code written order vera, execution order vera!
```

4. Modern JavaScript-la Asynchronous Methods

Companies-la commonly use panra important methods:

Method	Purpose
setTimeout()	Delay execution
setInterval()	Repeated execution
Promise	Async operation result handle panna
fetch()	API-la data request panna
async	Async function create panna
await	Promise result wait panna
.then()	Promise success handle panna
.catch()	Promise error handle panna

Modern Company Example – Fetch API

```
async function getUsers() {
    let response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
    );
    let data = await response.json();
    console.log(data);
}
getUsers();
```

1. async → Asynchronous function create pannum.
2. fetch() → API-kitta data request pannum.
3. await → Promise result varum varaikkum andha async function execution wait pannum.
4. response.json() → JSON data-va JavaScript object-ah convert pannum.
5. console.log(data) → Received data display pannum.

await use pannumbodhu entire JavaScript application block aagadhu. Andha async function mattum wait pannum.

6. Synchronous vs Asynchronous – Final Comparison

Synchronous	Asynchronous
Blocking execution	Non-blocking execution
One by one execution	Other tasks continue while waiting
Previous task complete aaganum	Long-running task wait pannumbodhu other work nadakkum
Simple calculations	API calls, file upload
Sequential order	Completion timing based order

One important interview point: JavaScript is single-threaded, but it supports asynchronous programming using the Event Loop, Call Stack, Web APIs and Callback Queue (Task Queue).

> **Practical Task 1:** Write a program with several console.log() statements to demonstrate the order of synchronous execution.

> **Practical Task 2:** Use setTimeout() with different delays to display messages and explain the order in which the callbacks execute.

CHAPTER 32

AJAX

XMLHTTPREQUEST · FETCH · REQUEST LIFECYCLE · READYSTATE · FLOW

AJAX concept-ah beginner level-la irundhu simple examples, code explanation, screen output-oda paakalaam.

1. What is AJAX?

AJAX stands for Asynchronous JavaScript and XML.

AJAX na webpage-ah full-ah reload pannama, background-la server-kitta data send and receive panna use panra technique.

Simple-ah sonna:

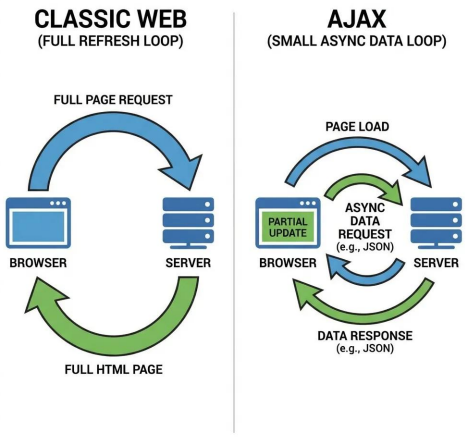
Normal website-la oru button click pannumbodhu entire page reload aagalaam.

Aana AJAX use pannina, webpage-la thevaiyana oru particular portion mattum update aagum. Entire page reload aagadhu.

Example:

Oru website-la user details load panrom nu vachukonga.

1. User button click pannuvanga.
2. JavaScript server-kitta request send pannum.
3. Server user details return pannum.
4. Webpage-la user details display aagum.
5. Entire webpage reload aagadhu.



Important: AJAX originally XML use pannuchu. But modern applications-la mostly JSON format use panranga.

What can AJAX do?

- Webpage load aana piragum server-la irundhu data retrieve pannalaam.
- Entire webpage reload pannama particular section update pannalaam.
- Background-la server-ku data send pannalaam.
- User other webpage interactions continue pannumbodhu data receive pannalaam.
- Website-ah interactive and responsive-ah make pannalaam.

2. AJAX using XMLHttpRequest Object

XMLHttpRequest (XHR) na JavaScript-la server-kitta HTTP request send panni response receive panna use panra object.

Idhu older method. Existing company projects-la innum use aagudhu.

XMLHttpRequest – 4 Steps

Step 1: Create XMLHttpRequest object

```
const xhttp = new XMLHttpRequest();
```

Server-kitta communicate panna oru XMLHttpRequest object create panrom.

Step 2: Define Callback Function

```
xhttp.onload = function() {
    // Response received
};
```

Server-la irundhu response vandha piragu indha function execute aagum.

Idha Callback Function nu solluvom.

Step 3: Open the Request

```
xhttp.open("GET", "ajax_info.txt");
```

- GET → Server-la irundhu data retrieve panna.
- ajax_info.txt → Request panna pora file name.

Step 4: Send the Request

```
xhttp.send();
```

Indha line server-kitta request-ah send pannum.

3. XMLHttpRequest – Complete Example

Rendu files create pannanum.

File 1: index.html

```
<!DOCTYPE html>
<html>
<body>
<h2>AJAX with XMLHttpRequest</h2>
<p id="demo">
    Click the button to load text.
</p>
<button onclick="loadDoc()">
    Load Data
</button>
<script>
function loadDoc() {
    const xhttp = new XMLHttpRequest();
    xhttp.onload = function() {
        document.getElementById("demo").textContent =
            this.responseText;
    };
    xhttp.open("GET", "ajax_info.txt");
    xhttp.send();
}
</script>
</body>
</html>
```

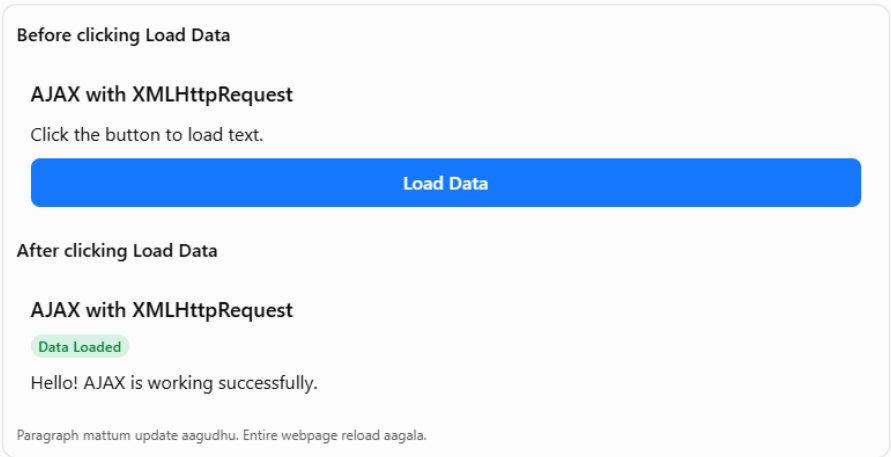
File 2: ajax_info.txt

Hello! AJAX is working successfully.

Explanation

1. onclick="loadDoc()" → Button click pannumbodhu function execute aagum.
2. new XMLHttpRequest() → Server communication object create pannum.
3. xhttp.onload → Response vandha piragu execute aagum.
4. open("GET", "ajax_info.txt") → File request-ah prepare pannum.
5. send() → Request send pannum.
6. this.responseText → Server return panna text-ah edukkum.
7. textContent → Webpage-la text display pannum.

Screen Output



Note: index.html and ajax_info.txt same project folder-la irukkanum. VS Code Live Server use panni open pannanum. Direct-ah HTML file open pannina local file restrictions nala request work aagama irukkalaam.

4. XMLHttpRequest Properties

XMLHttpRequest object-la different properties irukku. Idhu request status and server response details-ah access panna help pannum.

Property	Explanation
onload	Response receive aana piragu function execute aagum.
onreadystatechange	readyState change aagumbodhu execute aagum.
readyState	Request-oda current state-ah indicate pannum.
responseText	Server return panna response-ah text format-la edukkum.
responseXML	XML response-ah XML document format-la return pannum.
status	HTTP status code return pannum.
statusText	HTTP status message return pannum.

readyState Values

readyState na request ippo endha stage-la irukku nu sollum.

Value	Meaning	Explain
0	UNSENT	Request innum initialize aagala.
1	OPENED	Request open panniyachu.
2	HEADERS_RECEIVED	Server response headers received.
3	LOADING	Response data receive aagittu irukku.
4	DONE	Request complete aagiduchu.

Simple Example:

```
const xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    console.log(this.readyState);
};
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```

Possible console output:

```
1
2
3
4
```

Important Difference:

readyState and status rendu different.

- readyState = 4 → Request complete aagiduchu.
- status = 200 → HTTP request successful-ah complete aagiduchu.

Example:

Status Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

5. Modern AJAX Method – fetch()

fetch() na modern JavaScript-la server-kitta HTTP request send panni data receive panna use panra API.

Idhu Promise use pannum.

Old method:

```
XMLHttpRequest
```

Modern method:

```
fetch()
```

fetch() use pannumbodhu code easy-ah understand panna mudiyum.

Basic Syntax

```
fetch(url)
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log(error));
```

Explanation

1. fetch(url)

```
fetch(url)
```

Server-kitta request send pannum.

2. then()

```
.then(response => response.json())
```

Server response receive aana piragu JSON format-la read pannum.

3. then(data)

```
.then(data => console.log(data))
```

Received data-va console-la display pannum.

4. catch()

```
.catch(error => console.log(error));
```

Request-la network error vandha adha handle pannum.

6. Modern AJAX Example Using fetch()

Ippo oru real API-la irundhu data retrieve pannuvom.

API: JSONPlaceholder – Sample Post API (https://jsonplaceholder.typicode.com/posts/1?utm_source=chatgpt.com)

Complete HTML Code

```
<!DOCTYPE html>
<html>
<body>
<h2>AJAX with fetch()</h2>
<button onclick="getData()">Get Data</button>
<h3 id="title">Title appears here</h3>
<p id="body">Content appears here</p>
<script>
async function getData() {
```

```
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/posts/1"
        );
        if (!response.ok) {
            throw new Error("Request failed: " + response.status);
        }
        const data = await response.json();
        document.getElementById("title").textContent =
            data.title;
        document.getElementById("body").textContent =
            data.body;
    }
    catch (error) {
        document.getElementById("title").textContent =
            "Something went wrong: " + error.message;
    }
}
</script>
</body>
</html>
```

Line-by-line Explanation

1. async function

```
async function getData() {
```

async use panni asynchronous function create panrom.

Indha function Promise return pannum.

2. fetch()

```
const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts/1"
);
```

- fetch() → API-kitta request send pannum.
- await → Response varum varaikkum indha async function next line-ku pogama wait pannum.
- response → Server response-ah store pannum.

Important: await use pannalum entire webpage freeze aagadhu. Current async function mattum wait pannum.

3. response.ok

```
if (!response.ok) {
    throw new Error("Request failed: " + response.status);
}
```

Server response successful-ah irukka nu check pannum.

- response.ok true na HTTP status 200–299 range-la irukkum.
- !response.ok na success illa.
- throw new Error() error create panni catch block-ku anuppum.

4. response.json()

```
const data = await response.json();
```

Server return panna JSON data-va JavaScript object-ah convert pannum.

5. data.title

```
document.getElementById("title").textContent =
    data.title;
```

API-la irundhu vandha title-ah webpage-la display pannum.

6. data.body

```
document.getElementById("body").textContent =
    data.body;
```

API-la irundhu vandha body content-ah display pannum.

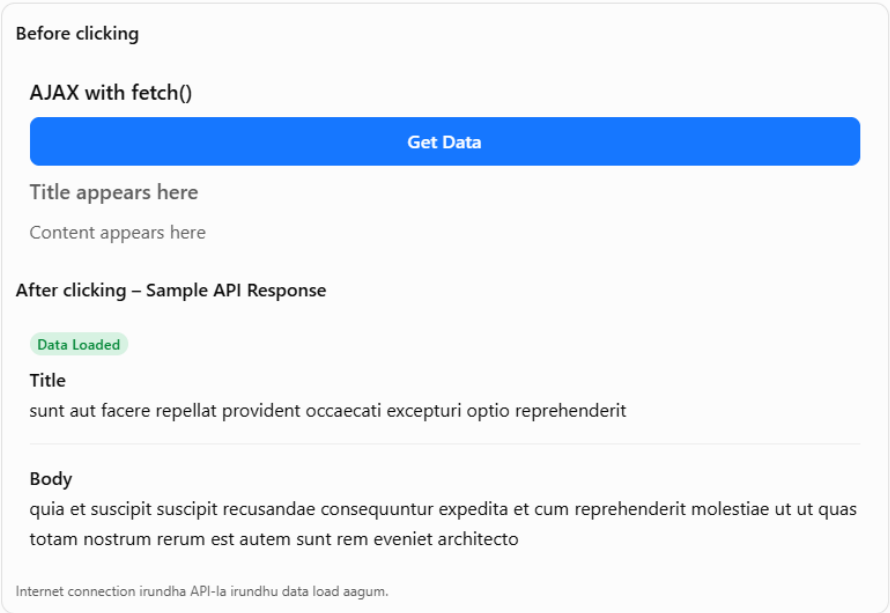
7. try...catch

```
try {
    // Request code
}
catch (error) {
    // Error handling
}
```

try block-la code execute aagum.

Error vandha catch block execute aagum.

Screen Output



7. XMLHttpRequest vs fetch()

Rendu methods-um AJAX request send and receive panna use pannalaam.

XMLHttpRequest	fetch()
Older established API	Modern API
Callback-based events	Promise-based
onload use pannalaam	.then() use pannalaam
open() and send()	fetch(url)
responseText	response.json() / response.text()
onreadystatechange	async / await
More event-based handling	Often easier to read

Example:

XMLHttpRequest:

```
const xhttp = new XMLHttpRequest();
xhttp.onload = function() {
    console.log(this.responseText);
};
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```

fetch():

```
async function getData() {
    const response = await fetch("ajax_info.txt");
    const data = await response.text();
    console.log(data);
}
```

Rendu methods-um full webpage reload pannama server-kitta data exchange pannum.

8. Important fetch() Rules

1. fetch() returns a Promise

```
fetch(url);
```

Request result Promise-ah return aagum.

2. await use panna async function thevai

```
async function getData() {
    const response = await fetch(url);
}
```

3. response.json()

JSON data read panna use pannuvom.

```
const data = await response.json();
```

4. response.text()

Plain text data read panna use pannuvom.

```
const data = await response.text();
```

5. response.ok

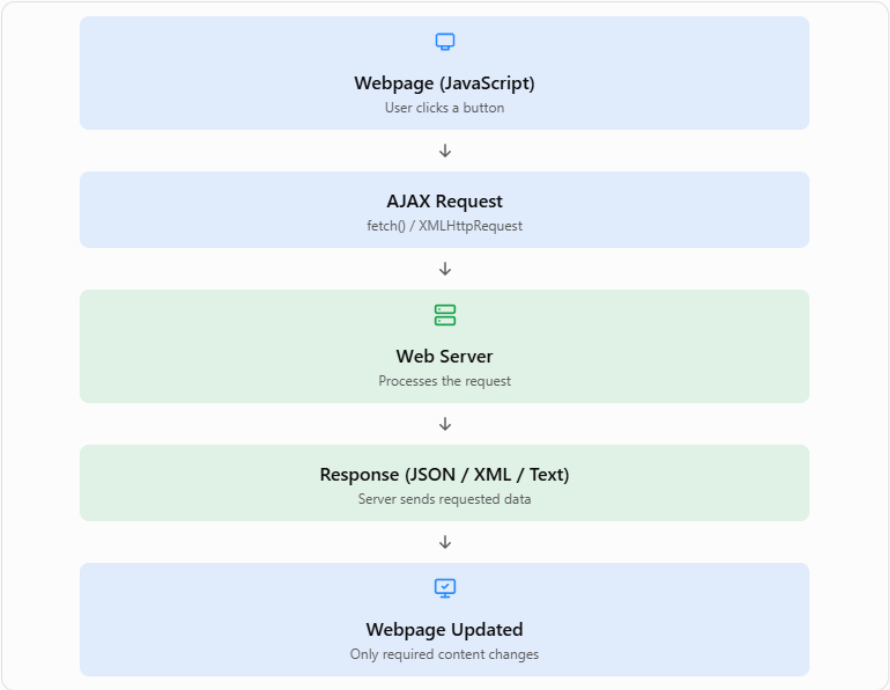
HTTP request successful-ah irukka nu check pannum.

6. try...catch

Errors handle panna use pannuvom.

One important point: fetch() HTTP 404 or 500 response vandhaalum automatically reject aagadhu. Adhanala response.ok check pannanum. Network failure madhiri errors-ku catch use pannuvom.

9. AJAX – Simple Flow Diagram



Overall concept:

- AJAX = Background-la server communication.
- XMLHttpRequest = Older AJAX method.
- fetch() = Modern AJAX method.
- Promise = Asynchronous operation result.
- async = Asynchronous function.
- await = Promise result-ku wait pannum.
- response.json() = JSON data read pannum.
- try...catch = Error handle pannum.

> **Practical Task 1:** Load data from a local text or JSON file using XMLHttpRequest and display the response on a webpage.

> **Practical Task 2:** Fetch user data from a public API and display the user's name, email and address. Handle loading and error states.

CHAPTER 33

Cookies

CREATE · READ · DELETE · ENCODE / DECODE · COOKIE PROPERTIES

1. What are Cookies?

Cookie na browser-la small text file format-la store aagura data.

Website-ku oru user already visit pannirukangala, avanga username enna, etc. nu remember panna Cookies use pannuvanga.

Simple Example:

Neenga oru website-ku first time visit panreenga.

- First time: Username Dhamodharan enter panreenga.
- Browser: Andha username-ah cookie-la save pannum.
- Next time: Same website open pannumbodhu cookie-la irukkura username-ah read pannalam.

Ippadi browser user information-ah remember pannum.

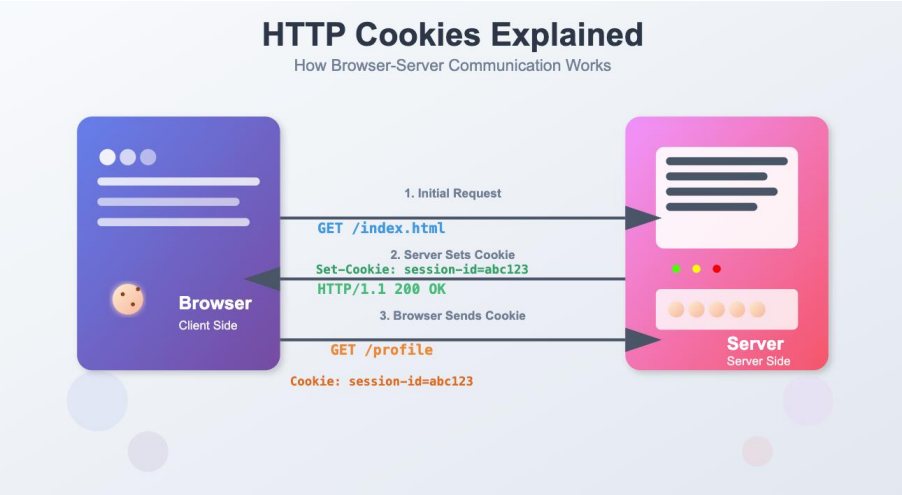
2. Why Cookies are Needed?

HTTP request-la ovvoru request-um separate request-ah treat pannapadum. Server automatically previous request information-ah remember pannaadhu.

Indha problem-ah solve panna Cookies use pannuvanga.

Example:

1. User website-ku request send panrar.
2. Server response-oda cookie-ah browser-ku send pannum.
3. Browser cookie-ah store pannum.
4. Next time user request send pannumbodhu, browser automatically matching cookie-ah request-oda send pannum.
5. Server cookie use panni user-ah identify pannalam.



3. How to Create a Cookie in JavaScript?

Syntax:

```
document.cookie = "name=value";
```

Explanation:

- document.cookie – Browser-la cookie create/read panna use pannuvom.
- name – Cookie name.
- value – Cookie-la store panna vendiya data.

Example:

```
document.cookie = "username=Dhamodharan";
```

Ippo browser-la username nu oru cookie create aagum.

4. encodeURIComponent() and decodeURIComponent()

Cookie values-la special characters irundha, encode pannuradhu useful.

encodeURIComponent()

Special characters-ah safe format-ku convert pannum.

```
let name = "Dhamodharan Kumar";
let result = encodeURIComponent(name);
console.log(result);
```

Output:

```
Dhamodharan%20Kumar
```

decodeURIComponent()

Encoded value-ah original value-ku convert pannum.

```
let result = "Dhamodharan%20Kumar";
console.log(decodeURIComponent(result));
```

Output:

```
Dhamodharan Kumar
```

5. Understanding Your Trainer's Example

```
<html>
<body>
    <form name="myform">
        Enter name:
        <input type="text" name="customer">
        <input
            type="button"
            value="Set Cookie"
            onclick="WriteCookie()"
        >
    </form>
    <script>
        function WriteCookie() {
            if (document.myform.customer.value == "") {
                alert("Enter some value!");
                return;
            }
            let value = encodeURIComponent(
                document.myform.customer.value
            );
            document.cookie = "name=" + value;
            document.write(
                "Setting Cookies: name=" +
                decodeURIComponent(value)
            );
        }
    </script>
</body>
</html>
```

Line-by-line Explain:

1. form name="myform" – Form-ku myform nu name kudukkurom.
2. input type="text" – User name enter panna input box.
3. onclick="WriteCookie()" – Button click pannumbodhu function call aagum.
4. function WriteCookie() – Cookie create panna function.
5. document.myform.customer.value – Input box-la user enter panna value-ah get pannum.
6. if (... == "") – Input empty-ah irukka nu check pannum.
7. alert() – Empty-ah irundha warning message kaattum.
8. encodeURIComponent() – Enter panna value-ah encode pannum.

9. document.cookie – Browser-la cookie save pannum.
10. decodeURIComponent() – Encoded value-ah original value-ku convert pannum.
11. document.write() – Webpage-la output display pannum.

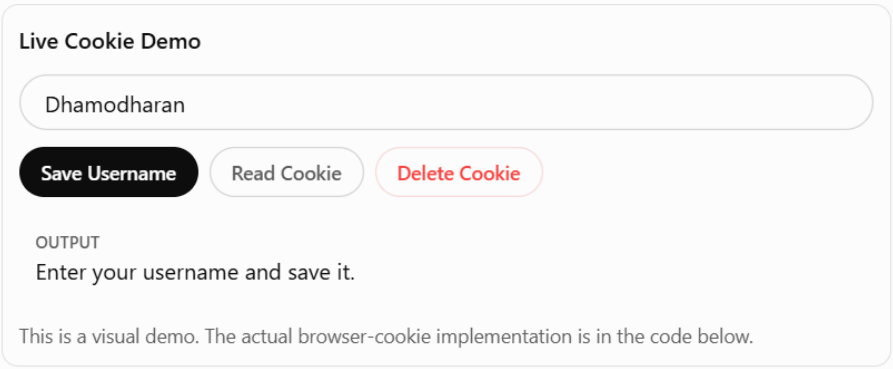
Important correction

document.write() button click event-kulla use pannina, existing webpage content overwrite aagalam. Real projects-la innerHTML use pannuradhu better.

6. Exercise – Complete Solution

Indha exercise-la 3 operations irukku:

1. Save Username
2. Read Cookie
3. Delete Cookie



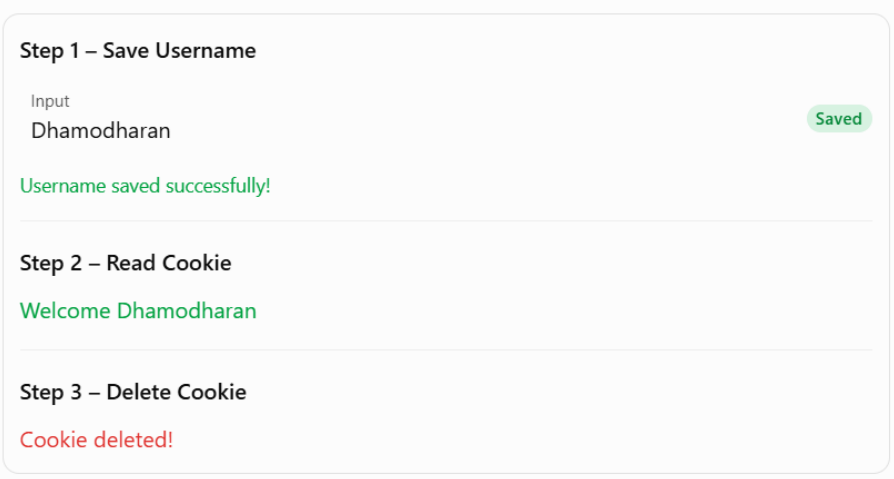
Full HTML + JavaScript Code

Copy this code into index.html and run it in your browser.

```
<!DOCTYPE html>
<html>
<head>
    <title>Cookie Example</title>
</head>
<body>
    <h2>JavaScript Cookies</h2>
    <input
        type="text"
        id="username"
        placeholder="Enter username"
    >
    <button onclick="saveCookie()">Save Username</button>
    <button onclick="readCookie()">Read Cookie</button>
    <button onclick="deleteCookie()">Delete Cookie</button>
    <h3 id="output"></h3>
    <script>
        // 1. Save Username
        function saveCookie() {
            let name = document.getElementById("username").value;
            if (name.trim() === "") {
```

```
                document.getElementById("output").innerHTML =
                    "Please enter username!";
                return;
            }
            document.cookie =
                "username=" + encodeURIComponent(name) +
                "; max-age=86400; path=/";
            document.getElementById("output").innerHTML =
                "Username saved successfully!";
        }
        // 2. Read Cookie
        function readCookie() {
            let cookies = document.cookie.split("; ");
            let username = cookies.find(
                cookie => cookie.startsWith("username=")
            );
            if (username) {
                let value = username.substring("username=".length);
                document.getElementById("output").innerHTML =
                    "Welcome " + decodeURIComponent(value);
            } else {
                document.getElementById("output").innerHTML =
                    "No username cookie found!";
            }
        }
        // 3. Delete Cookie
        function deleteCookie() {
            document.cookie =
                "username=; max-age=0; path=/";
            document.getElementById("output").innerHTML =
                "Cookie deleted!";
        }
    </script>
</body>
</html>
```

Expected Output



Important Cookie Properties

Property	Meaning
document.cookie	Read or create cookies
encodeURIComponent()	Encode special characters
decodeURIComponent()	Decode encoded values
max-age=86400	Cookie expires after 24 hours
max-age=0	Delete cookie
path=/	Cookie available across the website

Remember: Cookies are stored in the browser. Don't store passwords or sensitive information directly in cookies. For authentication, use secure server-managed cookies with appropriate settings such as HttpOnly, Secure, and SameSite.

> **Practical Task 1:** Create a form that stores a username in a browser cookie and displays the saved username when the page is reopened.

> **Practical Task 2:** Create a Delete Cookie button that removes the username cookie by setting its expiry date in the past.

CHAPTER 34

Session Storage & Local Storage

SESSION STORAGE · LOCAL STORAGE · METHODS · EXAMPLES · KEY DIFFERENCES

Session Storage and Local Storage rendu-me browser-la data save panna use aagum. Aana rendu perukkum important difference irukku: data evvalavu neram store aagum enbadhudhaan.

1. What is Session Storage?

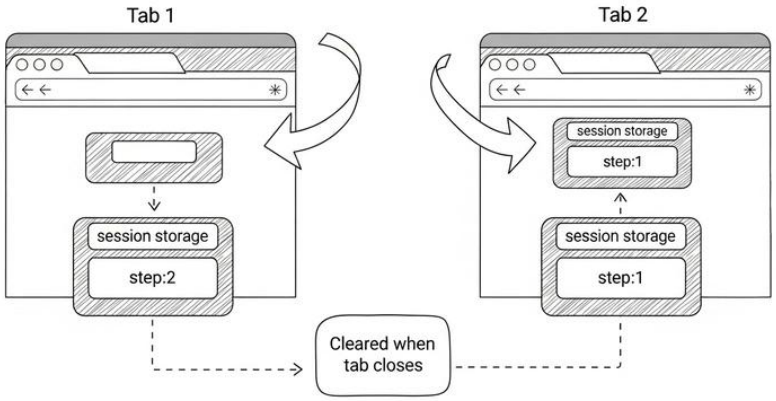
Session Storage na browser-la temporary-ah data store panna use panra method.

Example:

Neenga oru website-la username enter panreenga. Andha username-ah sessionStorage-la save pannina:

1. Page refresh pannalum data irukkum.
2. Same tab-la website use pannumbodhu data available-ah irukkum.
3. Andha tab session close aana data clear aagum.

Real-time example: Online exam website-la temporary exam progress-ah save pannuradhu.



2. Session Storage Syntax

```
window.sessionStorage
```

Or simply:

```
sessionStorage
```

Rendu-me same.

3. Session Storage Methods

Session Storage-la mainly 5 methods irukku.

1. setItem() – Save Data

```
sessionStorage.setItem("name", "Dhamodharan");
```

- setItem() – Data save panna use pannuvom.
- "name" – Key name.
- "Dhamodharan" – Store panna value.

2. getItem() – Read Data

```
let username = sessionStorage.getItem("name");
console.log(username);
```

Output:

```
Dhamodharan
```

Already save panna data-va thirumba retrieve panna getItem() use pannuvom.

3. removeItem() – Remove One Data

```
sessionStorage.removeItem("name");
```

name nu irukkura particular key and value mattum delete aagum.

4. clear() – Remove All Data

```
sessionStorage.clear();
```

Current tab-oda sessionStorage-la irukkura ella data-vum delete aagum.

5. key() – Get Key Name

```
let keyName = sessionStorage.key(0);
console.log(keyName);
```

Output:

```
name
```

Storage-la irukkura key-oda index use panni key name-ah get pannalam. Index 0 na first key.

4. Complete Session Storage Example

```
<!DOCTYPE html>
<html>
<body>
<h2>Session Storage Example</h2>
<p>Counter: <span id="demo">0</span></p>
<button onclick="countClick()">Count +1</button>
<button onclick="showValue()">Get Value</button>
<button onclick="removeCount()">Remove Count</button>
<button onclick="showKey()">Show Key</button>
<button onclick="clearAll()">Clear All</button>
<script>
// 1. Save data
function countClick() {
    let count = Number(
        sessionStorage.getItem("clickcount") || 0
    );
    count++;
    sessionStorage.setItem("clickcount", count);
    document.getElementById("demo").textContent = count;
}
// 2. Read data
function showValue() {
    let value = sessionStorage.getItem("clickcount");
    alert("Saved count: " + value);
}
// 3. Remove one item
function removeCount() {
    sessionStorage.removeItem("clickcount");
    document.getElementById("demo").textContent = 0;
```

```
    alert("Count removed!");
}
// 4. Get key name
function showKey() {
    let keyName = sessionStorage.key(0);
    alert("First key: " + keyName);
}
// 5. Clear all data
function clearAll() {
    sessionStorage.clear();
    document.getElementById("demo").textContent = 0;
    alert("All session storage cleared!");
}
</script>
</body>
</html>
```

Code Explanation

```
countClick()
let count = Number(sessionStorage.getItem("clickcount") || 0);
```

- getItem() – Already save panna count-ah get pannum.
- || 0 – Data illana default-ah 0 edukkum.
- Number() – String-ah number-ah convert pannum.

```
count++;
```

Count-ah 1 increase pannum.

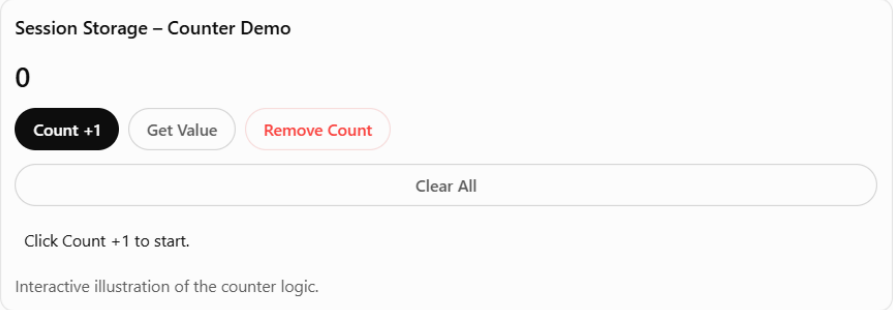
```
sessionStorage.setItem("clickcount", count);
```

Updated count-ah sessionStorage-la save pannum.

```
document.getElementById("demo").textContent = count;
```

Webpage-la updated count display pannum.

Expected Output



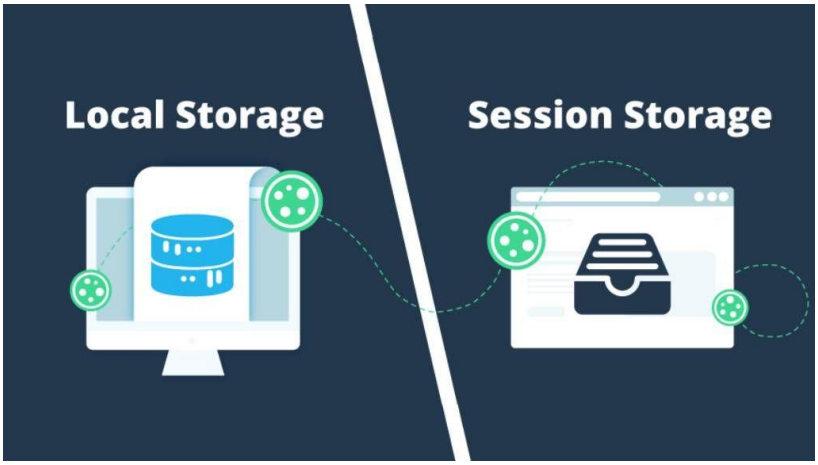
5. What is Local Storage?

Local Storage na browser-la data-va long-term-ah store panna use panra method.

Example:

Neenga oru website-la Dark Mode select panreenga.

- Dark Mode preference-ah localStorage-la save pannuvanga.
- Page refresh pannalum Dark Mode irukkum.
- Browser close pannitu reopen pannalum Dark Mode irukkum.
- Application or user remove pannura varaikkum data remain aagum (browser storage policies apply).



6. Local Storage Syntax

```
window.localStorage
```

Or:

```
localStorage
```

Rendum same.

7. Local Storage Methods

Session Storage-la use panna same 5 methods-thaan Local Storage-layum use pannuvom.

1. setItem() – Save Data

```
localStorage.setItem("name", "Dhamodharan");
```

Key = name

Value = Dhamodharan

2. getItem() – Read Data

```
let username = localStorage.getItem("name");
console.log(username);
```

Output:

```
Dhamodharan
```

3. removeItem() – Remove One Item

```
localStorage.removeItem("name");
```

name key mattum delete aagum.

4. clear() – Remove All Items

```
localStorage.clear();
```

Current website origin-oda localStorage data ellam delete aagum.

5. key() – Get Key Name

```
let keyName = localStorage.key(0);
console.log(keyName);
```

Output:

```
name
```

Index 0-la irukkura key name return pannum. Key illana null return pannum.

8. Complete Local Storage Example

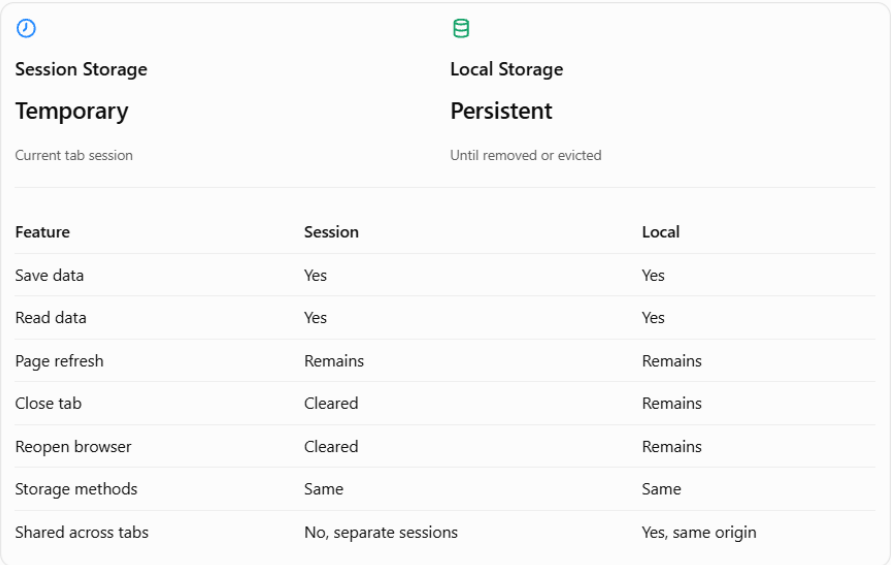
```
<!DOCTYPE html>
<html>
<body>
<h2>Local Storage Example</h2>
<input type="text" id="username" placeholder="Enter your name">
<button onclick="saveData()">Save Name</button>
<button onclick="getData()">Get Name</button>
<button onclick="removeData()">Remove Name</button>
<button onclick="showKey()">Show Key</button>
<button onclick="clearData()">Clear All</button>
<p id="demo"></p>
<script>
// 1. Save data
function saveData() {
    let name = document.getElementById("username").value;
    localStorage.setItem("name", name);
    document.getElementById("demo").textContent =
        "Name saved!";
}
```

```
// 2. Read data
function getData() {
    let name = localStorage.getItem("name");
    document.getElementById("demo").textContent =
        "Saved name: " + (name ?? "No name found");
}
// 3. Remove one item
function removeData() {
    localStorage.removeItem("name");
    document.getElementById("demo").textContent =
        "Name removed!";
}
// 4. Get key name
function showKey() {
    let keyName = localStorage.key(0);
    document.getElementById("demo").textContent =
        "Key: " + (keyName ?? "No key found");
}
// 5. Clear all data
function clearData() {
    localStorage.clear();
    document.getElementById("demo").textContent =
        "All local storage data cleared!";
}
</script>
</body>
</html>
```

Expected Output

1. Enter Dhamodharan → Click Save Name → Name saved!
2. Click Get Name → Saved name: Dhamodharan
3. Click Show Key → Key: name (if it is the first key).
4. Click Remove Name → Name removed!
5. Click Clear All → All local storage data cleared!

9. Session Storage vs Local Storage



Session Storage

Temporary

Current tab session

Local Storage

Persistent

Until removed or evicted

Feature	Session	Local
Save data	Yes	Yes
Read data	Yes	Yes
Page refresh	Remains	Remains
Close tab	Cleared	Remains
Reopen browser	Cleared	Remains
Storage methods	Same	Same
Shared across tabs	No, separate sessions	Yes, same origin

Easy Example:

- Session Storage = Temporary exam progress.
- Local Storage = Website Dark Mode preference.

10. Important Rules

1. Data is stored as Strings

```
localStorage.setItem("age", 25);
console.log(typeof localStorage.getItem("age"));
```

Output:

```
string
Number save pannalum string-ah store aagum.
2. Store Objects using JSON.stringify()
```

```
let user = {
    name: "Dhamodharan",
    age: 23
};
localStorage.setItem("user", JSON.stringify(user));
```

3. Read Objects using JSON.parse()

```
let data = JSON.parse(localStorage.getItem("user"));
console.log(data.name);
```

Output:

```
Dhamodharan
4. getItem() returns null if key doesn't exist.
```

```
console.log(localStorage.getItem("unknown"));
```

Output:

```
null
```

5. Never store passwords or sensitive information in localStorage or sessionStorage. JavaScript running on your webpage can access them.

Remember this:

- setItem() → Save
- getItem() → Read
- removeItem() → Delete one
- clear() → Delete all
- key() → Get key name
- sessionStorage → Temporary
- localStorage → Persistent

> **Practical Task 1:** Create a page that saves a user's name and selected theme in Local Storage and restores them after the browser is reopened.

> **Practical Task 2:** Create a session-based click counter using Session Storage. Add buttons to remove one stored item and clear all session data.

CHAPTER 35

Error Handling

TRY-CATCH · THROW · FINALLY · ERROR OBJECT · EXAMPLES

Error Handling na JavaScript code execute aagumbodhu unexpected errors vandha, adha detect panni handle panra process.

Simple-ah sollanumna, namma code-la error vandhaalum entire program stop aagama, andha error-ah handle panni application smooth-ah work aaga help panradhu dhaan Error Handling.

Why Error Handling is Important?

1. Application crash aagama prevent pannalam.
2. Errors-ah identify pannalam.
3. User-ku meaningful error messages show pannalam.
4. Application stability and reliability improve aagum.

1. Try-Catch Block

try-catch block runtime errors-ah handle panna use aagum.

- try – Error varalaam nu expect panra code-ah write pannuvom.
- catch – Try block-la error vandha adha catch panni handle pannum.

Syntax:

```
try {
    // Code that may cause an error
}
catch (error) {
    // Handle the error
}
```

Example: Handling an Undefined Variable

```
try {
    let result = x * 10;
}
catch (error) {
    console.log("An error occurred:", error.message);
}
```

Screen Output:

```
An error occurred: x is not defined
```

Explanation:

- x variable declare pannala.
- try block execute aagumbodhu ReferenceError varum.
- catch block andha error-ah catch pannum.
- error.message error-oda message-ah display pannum.

Important: Try block-la error vandhaalum catch execute aagum. Adhukku apram program continue aagalam.

2. Throw Statement

```
throw statement use panni nammale manually custom error generate pannalam.
```

Simple-ah sollanumna, particular condition satisfy aagala na nammale error create pannalam.

Example: Divide by Zero

```
function divide(a, b) {
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}
try {
    console.log(divide(10, 0));
}
catch (error) {
    console.log("Error:", error.message);
}
```

Screen Output:

```
Error: Cannot divide by zero
```

Explanation:

- divide(a, b) – Two numbers receive panra function.
- if (b === 0) – Second number zero-va check panrom.
- throw new Error() – Custom error create panrom.
- catch – Throw panna error-ah catch panrom.
- error.message – Error message display panrom.

One more example:

```
let age = 15;
try {
    if (age < 18) {
        throw new Error("You are not eligible");
    }
    console.log("Eligible");
}
catch (error) {
    console.log(error.message);
}
```

Screen Output:

```
You are not eligible
```

3. Finally Block

finally block na error vandhaalum varalainaalum execute aagura block.

Simple-ah sollanumna, try block success aanaalum, catch block execute aanaalum finally block kandippa execute aagum.

Syntax:

```
try {
    // Code
}
catch (error) {
    // Error handling
}
```

finally {

```
    // Always executes
}
```

Example:

```
try {
    let num = 10;
    console.log(num.toUpperCase());
}
catch (error) {
    console.log("Error caught:", error.message);
}
```

finally {

```
    console.log("Finally block executed");
}
```

Screen Output:

```
Error caught: num.toUpperCase is not a function
Finally block executed
```

Explanation:

- num is a number.
- toUpperCase() is a string method.
- Number-la toUpperCase() use panna mudiyadhu.
- So TypeError generate aagum.
- Catch block error-ah handle pannum.
- Finally block execute aagum.

Remember: Finally block error irundhaalum illainaalum execute aagum (normal control flow-la).

4. Error Object

JavaScript-la errors-ah represent panna built-in Error object irukku.

Error object-la mainly two important properties irukku:

Property	Purpose
error.name	Error type/name return pannum
error.message	Error description return pannum

JavaScript-la common-ah use aagura error types:

1. ReferenceError
2. TypeError
3. SyntaxError
4. RangeError

A. ReferenceError

Declare pannadha variable-ah access panna ReferenceError varum.

```
try {
    console.log(username);
}
catch (error) {
    console.log(error.name);
    console.log(error.message);
}
```

Screen Output:

```
ReferenceError
username is not defined
```

B. TypeError

Wrong data type or invalid operation perform pannumbodhu TypeError varum.

```
try {
    let num = 5;
    num.toUpperCase();
}
catch (error) {
    console.log("Error Type:", error.name);
    console.log("Error Message:", error.message);
}
```

Screen Output:

```
Error Type: TypeError
Error Message: num.toUpperCase is not a function
```

C. SyntaxError

JavaScript syntax rules-ah follow pannama code write pannumbodhu SyntaxError varum.

```
try {
    eval("let a = ;");
}
catch (error) {
    console.log(error.name);
}
```

Screen Output:

```
SyntaxError
```

Inga eval() string-ah JavaScript code madhiri execute panna try pannum. Invalid syntax irukkaradhunaala SyntaxError varum.

D. RangeError

Allowed range-ku veliya value use pannumbodhu RangeError varum.

```
try {
    let num = 10;
    num.toFixed(101);
}
catch (error) {
    console.log(error.name);
    console.log(error.message);
}
```

Screen Output:

```
RangeError
toFixed() digits argument must be between 0 and 100
```

5. Complete Error Handling Example

Ippo try, catch, throw, finally ellathayum combine panni oru example paakalam.

```
function checkAge(age) {
    try {
        if (age < 18) {
            throw new Error("Not eligible");
        }
        console.log("You are eligible");
    }
    catch (error) {
        console.log("Error:", error.message);
    }
    finally {
        console.log("Age verification completed");
    }
}
checkAge(15);
```

Screen Output:

```
Error: Not eligible
Age verification completed
```

6. Quick Revision

Concept	Usage
try	Error varalaam nu expect panra code
catch	Error-ah handle pannum
throw	Custom error generate pannum
finally	Always execute aagum
Error	Error object create pannum
error.name	Error type
error.message	Error description

> **Practical Task 1:** Use try...catch...finally to handle an invalid operation and display a meaningful error message.

> **Practical Task 2:** Create a function that checks a user's age and uses throw to generate an error when the age is below the required limit.

CHAPTER 36

Rest Parameters

SYNTAX · EXAMPLES · RULES · VS ARGUMENTS

Rest Parameters na function-ku ethana arguments kuduthalum, andha arguments ellathayum oru array-la collect pannurathukku use pannuvom.

Rest parameter-ku ... (three dots) symbol use pannuvom.

1. Syntax

```
function myFunction(...args) {
    // Code
}
```

- ... – Rest operator symbol.
- args – Arguments-ah collect pannura array.

2. Basic Example

```
function add(...numbers) {
    console.log(numbers);
}
add(10, 20, 30, 40);
```

Output:

```
  [ 10, 20, 30, 40 ]
```

Explanation:

add() function-ku 4 arguments pass pannirukkom. Rest parameter adhellathayum numbers array-la store pannuthu.

3. Rest Parameters with Addition

```
function add(...numbers) {
    let total = 0;
    for (let num of numbers) {
        total = total + num;
    }
    console.log(total);
}
add(10, 20, 30, 40);
```

Output:

```
  100
```

Explanation:

total = 0

0 + 10 = 10

10 + 20 = 30

30 + 30 = 60

60 + 40 = 100

Final output 100.

4. Normal Parameter with Rest Parameter

Normal parameters-um rest parameter-um same function-la use pannalam.

```
function student(name, ...marks) {
    console.log(name);
    console.log(marks);
}
student("Arun", 80, 90, 85, 95);
```

Output:

```
Arun
[ 80, 90, 85, 95 ]
```

Explanation:

- name → First argument (Arun).
- ...marks → Remaining arguments-ah array-la collect pannum.

5. Important Rules

1. Rest parameter ... symbol use pannum.
2. Multiple arguments-ah single array-la collect pannum.
3. Function-la only one rest parameter mattum use pannalam.
4. Rest parameter eppovum last parameter-ah irukkanum.

Wrong Example:

```
function add(...numbers, a) {
    console.log(numbers);
}
```

Output:

```
SyntaxError
Reason: Rest parameter last-la irukkanum.
```

6. Rest Parameters vs Arguments

Rest Parameters	Arguments
...args use pannuvom	arguments use pannuvom
Real array	Array-like object
Array methods use pannalam	Direct-ah ella array methods-um use panna mudiyadhu
Arrow function-la work aagum	Arrow function-ku own arguments illa

Easy-ah remember pannunga:

Rest Parameters = Function-ku pass pannura remaining arguments-ah array-la collect pannurathu.

> **Practical Task 1:** Create a function that accepts any number of numeric arguments using rest parameters and returns their sum.

> **Practical Task 2:** Create a student marks function that accepts a student name followed by multiple marks and returns the highest mark and average.

CHAPTER 37

Closures

BASIC · COUNTER · MULTIPLE FUNCTIONS · USES · POINTS

Closure na oru function, adhu create aana outer function-oda variables-ah remember panni access pannuradhu.

Simple-ah sonna, Outer function execute aagi mudinjirundhalum, inner function outer function-oda variables-ah access panna mudiyum.

1. Basic Example

```
function outer() {
    let name = "Arun";
    function inner() {
        console.log(name);
    }
    inner();
}
outer();
```

Output:

```
Arun
```

Explanation:

1. outer() function create pannirukkom.
2. Adhukkulla name variable irukku.
3. inner() function, outer function-oda name variable-ah access pannuthu.
4. outer() call pannumbodhu inner() execute aaguthu.
5. So output Arun.

2. Closure Example

Inga important concept paarunga.

```
function outer() {
    let count = 10;
    function inner() {
        console.log(count);
    }
    return inner;
}
let result = outer();
result();
```

Output:

```
10
```

Explanation:

- outer() execute aagumbodhu inner function return aaguthu.
- outer() execute aagi mudinjiruchu.
- Aana inner() function count variable-ah remember pannirukku.
- result() call pannumbodhu count value 10 print aaguthu.

Idhudhaan Closure.

3. Closure with Counter

```
function counter() {
    let count = 0;
    return function() {
        count++;
        console.log(count);
    };
}
let result = counter();
result();
result();
result();
```

Output:

```
1
2
3
```

Explanation:

- Starting count = 0.
- First call → count = 1.
- Second call → count = 2.
- Third call → count = 3.

Inner function previous count value-ah remember pannuthu.

4. Closure with Multiple Functions

```
function student() {
    let name = "Arun";
    function displayName() {
        console.log(name);
    }
    function changeName() {
        name = "Kumar";
    }
    return {
        displayName,
        changeName
    };
}
let result = student();
result.displayName();
result.changeName();
result.displayName();
```

Output:

```
Arun
Kumar
```

Explanation:

1. First displayName() call → Arun print aagum.
2. changeName() call → name value Kumar-ah change aagum.
3. Again displayName() call → Kumar print aagum.

Rendu inner functions-um same outer variable-ah access pannuthu.

5. Uses of Closures

1. Data Privacy – Variables-ah direct-ah outside access panna mudiyama protect pannalam.
2. Counter – Previous value-ah remember pannalam.
3. State Management – Values-ah maintain pannalam.
4. Function Factories – Different values use panni functions create pannalam.

6. Important Points

- Closure-la inner function outer function variables-ah access pannum.
- Outer function execute aagi mudinja pinnadiyum required variables accessible-ah irukkum.
- Inner function outer variables-ah remember pannum.
- JavaScript-la closures commonly used in counters, callbacks and data privacy.

> **Practical Task 1:** Create a counter function using a closure. Add separate methods to increment, decrement and read the private counter value.

> **Practical Task 2:** Create a bank account function that stores a private balance and returns deposit, withdraw and getBalance methods.

CHAPTER 38

Microtask vs Macrotask

MACROTASK · MICROTASK · EVENT LOOP · COMPARISON

JavaScript-la Microtask and Macrotask na asynchronous operations-ah execute panna use aagura task types.

Simple-ah sonna, JavaScript-la synchronous code first execute aagum. Adhukku apram Microtask queue-la irukkura tasks execute aagum. Adhukku apram Macrotask queue-la irukkura tasks execute aagum.

1. What is a Macrotask?

Macrotask na JavaScript event loop-la execute aaga wait pannura tasks.

Examples:

- setTimeout()
- setInterval()
- DOM events (click, keypress)
- I/O operations

Example:

```
console.log("Start");
setTimeout(() => {
    console.log("Macrotask");
}, 0);
console.log("End");
```

Output:

```
Start
End
Macrotask
```

Explanation:

1. Start print aagum.
2. setTimeout() callback Macrotask queue-ku pogum.
3. End print aagum.
4. Synchronous code complete aana piragu Macrotask execute aagum.

2. What is a Microtask?

Microtask na asynchronous task-la higher priority-la execute aagura task.

Examples:

- Promise.then()
- Promise.catch()
- Promise.finally()
- queueMicrotask()
- MutationObserver (browser)

Example:

```
console.log("Start");
Promise.resolve().then(() => {
    console.log("Microtask");
});
console.log("End");
```

Output:

```
Start
End
Microtask
```

Explanation:

1. Start print aagum.
2. Promise callback Microtask queue-ku pogum.
3. End print aagum.
4. Synchronous code mudinja udane Microtask execute aagum.

3. Microtask vs Macrotask

```
console.log("Start");
setTimeout(() => {
    console.log("Macrotask");
}, 0);
Promise.resolve().then(() => {
    console.log("Microtask");
});
console.log("End");
```

Output

```
Start
End
Microtask
Macrotask
```

Step-by-step Explanation:

Step	Code	Result
1	console.log("Start")	Start
2	setTimeout()	Macrotask queue
3	Promise.then()	Microtask queue
4	console.log("End")	End
5	Microtask executes	Microtask
6	Macrotask executes	Macrotask

Important: Synchronous code first execute aagum. Adhukku apram Microtask queue empty aagura varaikkum execute pannum. Appuram next Macrotask execute aagum.

4. Real-Time Example

```
console.log("1");
setTimeout(() => {
    console.log("2");
}, 0);
Promise.resolve().then(() => {
    console.log("3");
});
setTimeout(() => {
    console.log("4");
}, 0);
Promise.resolve().then(() => {
    console.log("5");
});
console.log("6");
```

Output:

```
1
6
3
5
2
4
```

Explanation:

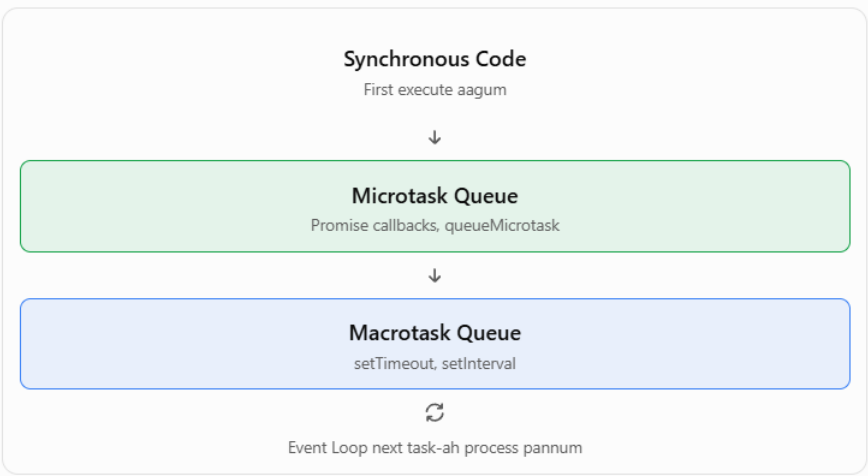
- 1 – Synchronous.
- 6 – Synchronous.
- 3 – First Microtask.
- 5 – Second Microtask.
- 2 – First Macrotask.
- 4 – Second Macrotask.

Microtasks first complete aagum. Adhukku apram Macrotasks execute aagum.

5. Difference Between Microtask and Macrotask

Microtask	Macrotask
Higher priority	Microtasks-ku apram execute aagum
Promise.then()	setTimeout()
Promise.catch()	setInterval()
queueMicrotask()	DOM events
MutationObserver	I/O callbacks

6. Event Loop Execution Order



Note: Idhu beginner-friendly simplified flow. Browser-la rendering and other event-loop details-um irukkum. Ovvoru Macrotask-ku apram Microtask queue again empty aagura varaikkum process pannappadum.

> **Practical Task 1:** Write a program using console.log(), Promise.then() and setTimeout() to observe the execution order.

> **Practical Task 2:** Create multiple Promise callbacks and setTimeout callbacks. Record their output order and identify the microtasks and macrotasks.

CHAPTER 39

Promises

PROMISES · CHAINING · ASYNC / AWAIT · ALL · RACE

1. Promises and Promise Chaining

Promise na enna?

JavaScript-la oru asynchronous operation complete aagurathukku wait pannumbothu use panrathu Promise.

Promise-ku 3 states irukkum:

1. Pending – Operation innum complete aagala.
2. Fulfilled – Operation successful-a complete aagiduchu.
3. Rejected – Operation fail aagiduchu.

Simple Example

```
let myPromise = new Promise((resolve, reject) => {
    resolve("Task Completed!");
});
myPromise.then((result) => {
    console.log(result);
});
```

Output:

```
Task Completed!
```

- resolve() – Promise success aana call pannuvom.
- reject() – Promise fail aana call pannuvom.
- .then() – Promise success aana execute aagum.
- .catch() – Promise fail aana execute aagum.

Promise Chaining

Oru Promise mudinja piragu next operation execute aaganum na .then() chain pannalam.

```
Promise.resolve(10)
    .then((num) => {
        console.log(num);
        return num + 10;
    })
    .then((num) => {
        console.log(num);
        return num + 10;
    })
    .then((num) => {
        console.log(num);
    });
```

Output:

```
10
20
30
```

Simple-a purinjukonga: First 10 print aagum. Adhukku 10 add panni 20 varum. Again 10 add panni 30 varum.

Error Handling – .catch()

```
let myPromise = new Promise((resolve, reject) => {
    reject("Something went wrong!");
});
```

myPromise

```
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });
```

Output:

```
Something went wrong!
```

2. async / await

async na enna?

async keyword use panni oru function create pannumbothu, andha function always oru Promise return pannum.

Example

```
async function greet() {
    return "Hello JavaScript!";
}
greet().then((result) => {
    console.log(result);
});
```

Output:

```
Hello JavaScript!
```

await na enna?

await keyword Promise result varum varaikkum andha async function-oda execution-a wait panna vaikkum.

Example:

```
function getData() {
    return new Promise((resolve) => {
        resolve("Data Received!");
    });
}
async function display() {
    let result = await getData();
    console.log(result);
}
display();
```

Output:

```
Data Received!
```

async / await with setTimeout

```
function getData() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Data Loaded!");
        }, 2000);
    });
}
async function display() {
    console.log("Loading...");
    let result = await getData();
    console.log(result);
}
display();
```

Output:

```
Loading...
Data Loaded!
2 seconds delay-kku piragu Data Loaded! print aagum.
```

Error Handling using try...catch

```
async function getData() {
    try {
        let result = await Promise.reject("Error Occurred!");
        console.log(result);
    }
    catch (error) {
        console.log(error);
    }
}
getData();
```

Output:

```
Error Occurred!
```

Remember:

- async – Function Promise return pannum.
- await – Promise result-kaga wait pannum.
- try – Code execute pannum.
- catch – Error handle pannum.

3. Promise.all() and Promise.race()

A. Promise.all()

Multiple Promises irundha, ellame successful-a complete aana piragu result return pannum.

```
let promise1 = Promise.resolve("Apple");
let promise2 = Promise.resolve("Banana");
let promise3 = Promise.resolve("Orange");
Promise.all([promise1, promise2, promise3])
    .then((result) => {
        console.log(result);
    });
```

Output:

```
  [ 'Apple', 'Banana', 'Orange' ]
```

Important:

- All Promises success aaganum.
- Oru Promise fail aana Promise.all() reject aagum.
- Results original array order-la varum; completion order-la illa.

B. Promise.race()

Multiple Promises-la first-a settle aagura Promise result-a return pannum.

Settle na success (resolve) illa failure (reject) rendu-me include aagum.

```
let promise1 = new Promise((resolve) => {
    setTimeout(() => {
        resolve("Apple");
    }, 2000);
});
let promise2 = new Promise((resolve) => {
    setTimeout(() => {
        resolve("Banana");
    }, 1000);
});
Promise.race([promise1, promise2])
    .then((result) => {
        console.log(result);
    });
```

Output:

```
Banana
```

Banana Promise 1 second-la complete aaguthu. Apple Promise 2 seconds edukuthu.

Difference between Promise.all() and Promise.race()

Promise.all()	Promise.race()
All Promises settle aaganum	First Promise settle aana podhum
All success aana results return pannum	First success or error return pannum
Results array-a return pannum	Single result return pannum
One rejection-na reject aagum	First settled Promise-a follow pannum

Practical Task 1: Create a Promise that resolves after two seconds. Handle the result using then(), catch() and finally().

Practical Task 2: Create an asynchronous product-order simulation using async/await and try...catch. Demonstrate Promise.all() or Promise.race().

CHAPTER 40

Shallow Copy vs Deep Copy & Prototypes and Prototype Chain

SHALLOW · DEEP · PROTOTYPE · CHAIN · COMPARISON

1. Shallow Copy vs Deep Copy

JavaScript-la oru object-a copy pannumbothu rendu methods irukku: Shallow Copy and Deep Copy.

A. Shallow Copy

Shallow Copy na object-oda first-level properties mattum copy aagum. Nested objects irundha, original object-oda reference share pannum.

Example:

```
let person = {
    name: "Dhamo",
    age: 23,
    address: {
        city: "Chennai"
    }
};
let copy = { ...person };
copy.name = "Dharan";
copy.address.city = "Madurai";
console.log(person.name);
console.log(person.address.city);
```

Output:

```
Dhamo
Madurai
```

Explanation:

- copy.name change pannalum original person.name change aagadhu.
- Aana copy.address.city change pannumbothu original-um change aagum.
- Reason: Nested object reference share pannuthu.

Shallow Copy create panna methods:

```
let copy1 = { ...person };
let copy2 = Object.assign({}, person);
```

B. Deep Copy

Deep Copy na object-oda nested objects-um separate-a copy aagum. Original object and copied object independent-a irukkum.

Example:

```
let person = {
    name: "Dhamo",
    address: {
        city: "Chennai"
    }
};
let copy = structuredClone(person);
copy.name = "Dharan";
copy.address.city = "Madurai";
console.log(person.name);
console.log(person.address.city);
console.log(copy.name);
console.log(copy.address.city);
```

Output:

```
Dhamo
Chennai
Dharan
Madurai
```

Explanation:

- Original object change aagadhu.
- Copied object thaniya change aagum.
- Nested objects-um separate-a copy aagum.

Difference

Shallow Copy	Deep Copy
First-level properties copy aagum	Nested objects-um copy aagum
Nested reference share pannum	Nested reference share pannaadhu
Nested changes original-la affect aagalam	Original independent-a irukkum
Faster and simple	More processing thevaipadalam

Important

Modern JavaScript-la deep copy-ku structuredClone() use pannalam. Functions and some special values support aagadhu.

2. Prototypes and Prototype Chain

A. Prototype na enna?

JavaScript-la every object-kum innoru object-a reference panna mudiyum. Andha referenced object-thaan Prototype.

Prototype moolama oru object innoru object-oda properties and methods-a access pannalam.

Example:

```
let person = {
    name: "Dhamo"
};
console.log(person.name);
console.log(person.toString);
```

Output:

```
Dhamo
ƒ toString() { [native code] }
```

Explanation:

- name property namma create pannathu.
- toString() method JavaScript object prototype-la irundhu kidaikkuthu.
- Athanala namma direct-a use panna mudiyuthu.

B. Prototype using Object.create()

Object.create() use panni oru object-ku innoru object-a prototype-ah set pannalam.

Example:

```
let person = {
    greet: function() {
        console.log("Hello!");
    }
};
let student = Object.create(person);
student.name = "Dhamo";
console.log(student.name);
student.greet();
```

Output:

```
Dhamo
Hello!
```

Explanation:

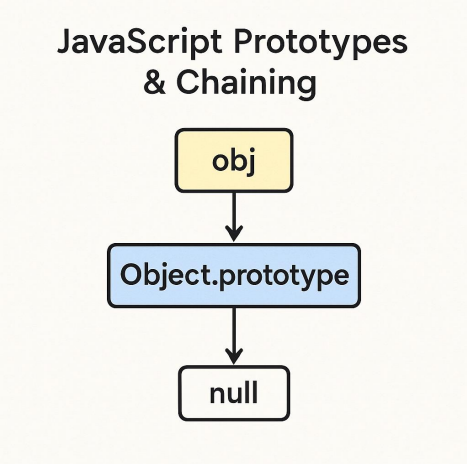
- person object-la greet() method irukku.
- student object create pannumbothu person-a prototype-ah set pannirukkom.
- So student-ku direct-a greet() illa naalum, person prototype moolama access pannuthu.

C. Prototype Chain na enna?

Oru object-ku thevaiyana property adhu kitta illana, JavaScript adhan prototype-la search pannum.

Angayum illana, adutha prototype-la search pannum.

Indha searching process-thaan Prototype Chain.



Example:

```
let animal = {
    eat: function() {
        console.log("Eating...");
    }
};
let dog = Object.create(animal);
dog.bark = function() {
    console.log("Barking...");
};
dog.bark();
dog.eat();
console.log(dog.toString);
```

Output:

```
Barking...
Eating...
ƒ toString() { [native code] }
```

How it works:

1. dog.bark() – dog object-la method irukku.
2. dog.eat() – dog-la illa; animal prototype-la irukku.
3. dog.toString – animal-la illa; next Object.prototype-la irukku.

Prototype chain mudivil null varum.

D. prototype vs __proto__

prototype	__proto__
Constructor function-la irukkum	Object-oda prototype-a access pannum
Shared methods define panna use aagum	Prototype reference-a access panna use aagum

Example:

```
function Person(name) {
    this.name = name;
}
Person.prototype.greet = function() {
    console.log("Hello " + this.name);
};
let user = new Person("Dhamo");
user.greet();
```

Output:

```
Hello Dhamo
```

Inga greet() method ovvoru object-kum separate-a create aagadhu. Person.prototype-la share aagum.

> **Practical Task 1:** Create an object with a nested object and demonstrate how changing a nested property affects a shallow copy and a deep copy.

> **Practical Task 2:** Create a Person object and another object that inherits from it using Object.create(). Demonstrate property lookup through the prototype chain.

CHAPTER 41

ES6 Modules (import / export)

NAMED · DEFAULT · ALIAS · IMPORT ALL · HTML MODULE

ES6 Modules na enna?

JavaScript-la oru file-la irukkura functions, variables, classes-a innoru file-la use pannurathukku ES6 Modules use pannuvom.

Idhula two important keywords irukku:

1. export – Oru file-la irukkura code-a another file-ku share pannum.
2. import – Another file-la irundhu code-a receive pannum.

Example: Oru project-la math.js and app.js nu two files irukku.

1. Named Export

Oru file-la multiple functions or variables export pannalam.

```
math.js
```

export let a = 10;

export let b = 20;

export function add() {

```
    console.log(a + b);
}
app.js
```

import { a, b, add } from "./math.js";

```
console.log(a);
console.log(b);
add();
```

Output:

```
10
20
30
```

Explanation:

- export use panni variables and function share pannirukkom.
- import { } use panni required items receive pannirukkom.
- Named export-la same name use pannanum.

2. Default Export

Oru file-la one main value or function-a default-ah export pannalam.

```
greet.js
function greet() {
    console.log("Welcome to JavaScript!");
}
```

export default greet;

```
app.js
```

import greet from "./greet.js";

```
greet();
```

Output:

```
Welcome to JavaScript!
```

Important:

- Oru file-la one default export mattum irukkalam.
- Default import-ku { } thevai illa.
- Import pannumbothu vera name kooda use pannalam.

3. Named Export vs Default Export

Named Export	Default Export
Multiple exports possible	One default export per module
{ } use pannanum	{ } thevai illa
Same exported name use pannanum (alias use pannalam)	Any import name use pannalam
export	export default

4. Import with Alias

Export pannina name-a different name-la import pannalam.

```
math.js
```

export function add() {

```
    console.log(10 + 20);
}
app.js
```

import { add as sum } from "./math.js";

```
sum();
```

Output:

```
30
Inga add function-a sum nu rename pannirukkom.
```

5. Import Everything

Oru module-la irukkura named exports ellathayum import panna * use pannalam.

```
math.js
```

export let a = 10;

export let b = 20;

export function add() {

```
    console.log(a + b);
}
app.js
```

import * as math from "./math.js";

```
console.log(math.a);
console.log(math.b);
math.add();
```

Output:

```
10
20
30
```

6. HTML-la JavaScript Module Use Pannurathu

Browser-la ES6 Modules use panna HTML file-la type="module" add pannanum.

```
index.html
<script type="module" src="app.js"></script>
```

Ippo app.js-la import and export use pannalam.

Remember: ES6 Modules use pannumbothu browser-la type="module" important. Modules automatically strict mode-la execute aagum.

> **Practical Task 1:** Create a math.js module that exports addition and subtraction functions, then import and use them in a main JavaScript file.

> **Practical Task 2:** Create separate modules for product data and display functions. Use both named exports and a default export.

CHAPTER 42

REST API Integration

API · HTTP METHODS · FETCH · GET · POST · PUT · PATCH · DELETE

REST API Integration na enna?

JavaScript use panni external server or backend-la irundhu data-va send and receive pannurathuthaan REST API Integration.

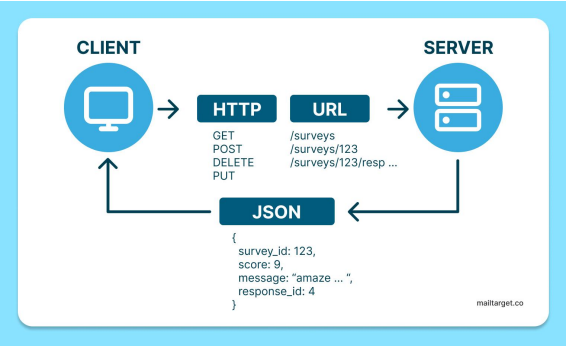
Example:

- Weather application-la weather details get pannurathu.
- E-commerce website-la products display pannurathu.
- Login form-la user details backend-ku send pannurathu.
- Social media-la posts fetch pannurathu.

1. What is API?

API (Application Programming Interface) na two applications communicate pannikkirathukku help pannura interface.

Example:



1. JavaScript frontend server-ku request send pannum.
2. Server request process pannum.
3. Server response send pannum.
4. JavaScript response data-va display pannum.

2. HTTP Methods

REST API-la commonly use pannura HTTP methods:

Method	Purpose
GET	Data receive pannum
POST	New data create pannum
PUT	Existing data full-a update pannum
PATCH	Specific data update pannum
DELETE	Data delete pannum

3. Fetch API

JavaScript-la server-kitta request send panna fetch() use pannuvom.

Syntax:

```
fetch("API_URL")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

4. GET Request

Server-la irundhu data-va receive panna GET request use pannuvom.

Example:

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => {
        console.log(data);
    })
    .catch(error => {
        console.log(error);
    });
```

Output (example):

[

```
  {
    id: 1,
    name: "Leanne Graham",
    username: "Bret",
    email: "Sincere@april.biz"
  },
  {
    id: 2,
    name: "Ervin Howell",
    username: "Antonette",
    email: "Shanna@melissa.tv"
  }
]
```

API-la return aagura full user data console-la display aagum.

5. POST Request

Server-ku new data send panna POST request use pannuvom.

Example:

fetch("https://jsonplaceholder.typicode.com/posts", {

method: "POST",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        title: "JavaScript",
        body: "Learning REST API",
        userId: 1
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
});
```

Output:

```
{
  title: "JavaScript",
  body: "Learning REST API",
  userId: 1,
  id: 101
}
```

- method – Request type.
- headers – Data format information.
- JSON.stringify() – JavaScript object-a JSON string-ah convert pannum.
- body – Server-ku send pannura data.

6. PUT Request

Existing data-va full-a update panna PUT use pannuvom.

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "PUT",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        id: 1,
        title: "Updated Title",
        body: "Updated Content",
        userId: 1
    })
})
.then(response => response.json())
.then(data => console.log(data));
```

Output:

```
{
  id: 1,
  title: "Updated Title",
  body: "Updated Content",
  userId: 1
}
```

7. PATCH Request

Existing data-la specific property mattum update panna PATCH use pannuvom.

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "PATCH",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        title: "New Title"
    })
})
.then(response => response.json())
.then(data => console.log(data));
```

Output:

```
{
  userId: 1,
  id: 1,
  title: "New Title",
  body: "..."
}
```

8. DELETE Request

Server-la irukkura data-va delete panna DELETE method use pannuvom.

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "DELETE"

```
})
.then(response => response.json())
.then(data => console.log(data));
```

Output:

```
{}
```

9. REST API using Async / Await

async and await use panni API request-a simple-ah handle pannalam.

```
async function getUsers() {
    try {
        let response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );
        let data = await response.json();
        console.log(data);
    }
    catch (error) {
        console.log("Error:", error);
    }
}
getUsers();
```

Output:

```
[
  { id: 1, name: "Leanne Graham", ... },
  { id: 2, name: "Ervin Howell", ... },
  ...
]
```

10. HTTP Status Codes

API request success or failure status-a identify panna HTTP status codes use pannuvom.

Status Code	Meaning
200	OK – Success
201	Created – New data created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Important: fetch() 404 or 500 response vandhalum automatically catch() execute aagadhu. response.ok check pannanum.

Code:

```
async function getData() {
    try {
        let response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );
        if (!response.ok) {
            throw new Error("API Request Failed");
        }
        let data = await response.json();
        console.log(data);
    }
    catch (error) {
        console.log(error.message);
    }
}
getData();
```

> **Practical Task 1:** Use Fetch API to send a GET request to a sample REST API and display the returned user details.

> **Practical Task 2:** Build a small interface that performs POST, PUT, PATCH and DELETE requests against a sample REST API and displays each response.

CHAPTER 43

HTTP Methods and Status Codes

GET · POST · PUT · PATCH · DELETE · STATUS CODES

1. HTTP Methods

HTTP Methods na frontend and backend-ku idaiyila enna type of operation perform pannanum nu specify pannurathu.

A. GET – Get Data

Server-la irundhu data-va receive panna GET method use pannuvom.

JavaScript Code:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(response => {
        console.log("Status:", response.status);
        return response.json();
    })
    .then(data => {
        console.log(data);
    });
```

Screen Output (Browser Console):

Status: 200

```
{
  id: 1,
  name: "Leanne Graham",
  username: "Bret",
  email: "Sincere@april.biz",
  ...
}
```

B. POST – Create Data

Server-ku new data send panni create panna POST use pannuvom.

JavaScript Code:

fetch("https://jsonplaceholder.typicode.com/posts", {

method: "POST",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        title: "JavaScript",
        body: "Learning REST API",
        userId: 1
    })
})
.then(response => {
    console.log("Status:", response.status);
    return response.json();
})
.then(data => {
    console.log(data);
});
```

Screen Output:

```
Status: 201
```

```
{
  title: "JavaScript",
  body: "Learning REST API",
  userId: 1,
  id: 101
}
```

C. PUT – Full Update

Existing data-va full-ah update panna PUT method use pannuvom.

JavaScript Code:

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "PUT",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        id: 1,
        title: "Updated Title",
        body: "Updated Content",
        userId: 1
    })
})
.then(response => {
    console.log("Status:", response.status);
    return response.json();
})
.then(data => console.log(data));
```

Screen Output:

```
Status: 200
```

```
{
  id: 1,
  title: "Updated Title",
  body: "Updated Content",
  userId: 1
}
```

D. PATCH – Partial Update

Existing data-la specific property mattum update panna PATCH use pannuvom.

JavaScript Code:

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "PATCH",

headers: {

"Content-Type": "application/json"

```
    },
    body: JSON.stringify({
        title: "New Title"
    })
})
.then(response => {
    console.log("Status:", response.status);
    return response.json();
})
.then(data => console.log(data));
```

Screen Output:

```
Status: 200
```

```
{
  userId: 1,
  id: 1,
  title: "New Title",
  body: "..."
}
```

E. DELETE – Delete Data

Server-la irukkura data-va delete panna DELETE method use pannuvom.

JavaScript Code:

fetch("https://jsonplaceholder.typicode.com/posts/1", {

method: "DELETE"

```
})
.then(response => {
    console.log("Status:", response.status);
    return response.json();
})
.then(data => console.log(data));
```

Screen Output:

```
Status: 200
{}
```

2. HTTP Status Codes

HTTP Status Codes na server namma request-ku enna response kuduthirukku nu indicate pannum.

Status Code	Name	Description
100	Continue	Request continue pannalam.
101	Switching Protocols	Protocol change aaguthu.
200	OK	Request successful.
201	Created	New resource create aagiduchu.
202	Accepted	Request accept pannirukanga.
204	No Content	Success, response body illa.
301	Moved Permanently	URL permanently change aagiduchu.
302	Found	Temporary redirection.
304	Not Modified	Cached data use pannalam.
400	Bad Request	Invalid request.
401	Unauthorized	Authentication missing or invalid.
403	Forbidden	Access panna permission illa.
404	Not Found	Requested resource kidaikkala.
405	Method Not Allowed	HTTP method allowed illa.
408	Request Timeout	Request time out aagiduchu.
409	Conflict	Existing data-oda conflict.
410	Gone	Resource permanently unavailable.
413	Content Too Large	Request data romba perusa irukku.
415	Unsupported Media Type	Unsupported data format.
422	Unprocessable Content	Validation error.
429	Too Many Requests	Too many requests send pannirukkom.
500	Internal Server Error	Server-la unexpected error.
501	Not Implemented	Server-ku indha functionality support illa.
502	Bad Gateway	Upstream server invalid response.
503	Service Unavailable	Server temporarily unavailable.
504	Gateway Timeout	Upstream server response late aaguthu.

Status Code Categories

Category	Range	Meaning
1xx	100–199	Informational
2xx	200–299	Success
3xx	300–399	Redirection
4xx	400–499	Client Errors
5xx	500–599	Server Errors

IT Companies-ku romba important: 200, 201, 204, 301, 400, 401, 403, 404, 405, 409, 422, 429, 500, 502, 503, 504.

Note: Ivai common examples mattum. Actual status code API implementation and operation-a poruthu maaralam.

Important: HTTP Method enna operation perform pannanum nu sollum. HTTP Status Code andha request-ku server enna response kuduthuchu nu sollum.

> **Practical Task 1:** Create a Fetch API program that retrieves a resource using GET and displays the response status code.

> **Practical Task 2:** Send sample data using POST and demonstrate how your program handles successful and unsuccessful HTTP status codes.

CHAPTER 44

CORS (Cross-Origin Resource Sharing)

ORIGIN · ERROR · FIX · HEADERS · PREFLIGHT

CORS na enna?

CORS na Cross-Origin Resource Sharing.

Oru website or frontend application, different origin-la irukkura backend server/API-kitta request send pannumbothu browser security mechanism use pannum. Andha server permission kuduthirundha mattum browser response-a frontend JavaScript access panna allow pannum.

Simple Example:

- Frontend: http://localhost:3000
- Backend: http://localhost:8080

Rendu different ports-la irukkurathala different origins.



1. What is Origin?

Origin na 3 parts combination:

1. Protocol – http or https
2. Domain – example.com
3. Port – 3000, 8080

Example:

URL	Origin
http://localhost:3000	localhost:3000
http://localhost:8080	localhost:8080
https://localhost:3000	https + localhost:3000

Protocol, domain, or port different-ah irundha different origin.

2. CORS Error Example

Frontend JavaScript:

```
fetch("http://localhost:8080/api/users")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

Browser Console Output (example):

Access to fetch at 'http://localhost:8080/api/users'

from origin 'http://localhost:3000'

has been blocked by CORS policy.

Explanation:

- Frontend backend-ku request send pannuthu.
- Backend correct CORS permission provide pannala.
- Browser response-a JavaScript access panna block pannidum.

3. How to Fix CORS?

CORS problem-a backend server-la correct-ah configure pannanum.

Example – Spring Boot:

@CrossOrigin(origins = "http://localhost:3000")

@RestController

public class UserController {

@GetMapping("/api/users")

public String getUsers() {

```
        return "Users Data";
    }
}
```

Ippo localhost:3000 frontend-kku access permission kuduthirukkom.

Expected Browser Console Output:

Users Data

4. CORS Headers

Server response-la commonly use pannura CORS headers:

Header	Purpose
Access-Control-Allow-Origin	Which origin access pannalam
Access-Control-Allow-Methods	Allowed HTTP methods
Access-Control-Allow-Headers	Allowed request headers
Access-Control-Allow-Credentials	Credentials allow pannalama

Example Response Headers:

Access-Control-Allow-Origin: http://localhost:3000

Access-Control-Allow-Methods: GET, POST, PUT, DELETE

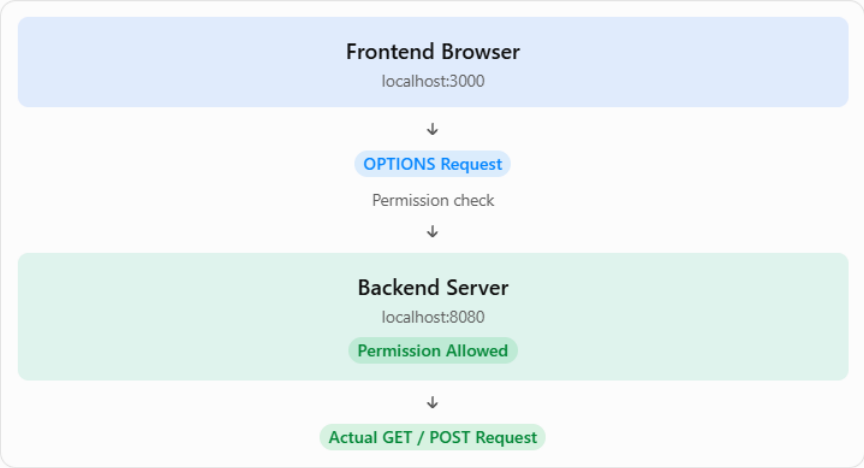
Access-Control-Allow-Headers: Content-Type, Authorization

5. Preflight Request (OPTIONS)

Sila cross-origin requests-ku browser actual request send pannurathukku munadi OPTIONS request send pannum.

Idhuthaan Preflight Request.

Example Flow:



Example OPTIONS Request Headers:

OPTIONS /api/users HTTP/1.1

Origin: http://localhost:3000

Access-Control-Request-Method: POST

Access-Control-Request-Headers: Content-Type

6. Important CORS Points

- CORS is a browser security mechanism.
- Backend server CORS headers moolama permission provide pannum.
- Different ports also different origins.
- OPTIONS request preflight check-ku use aagum.
- Access-Control-Allow-Origin: * use pannina any origin allow aagalam (credentials use pannumbothu * allowed illa).
- Frontend-la no-cors mode use pannurathu CORS problem-ku proper solution illa; response opaque-ah irukkum.

Remember: CORS problem frontend JavaScript-la mattum fix panna mudiyadhu. Usually backend server-la correct CORS configuration set pannanum.

CHAPTER 45

Debouncing and Throttling in JavaScript

DEBOUNCING · THROTTLING · COMPARISON · APPLICATIONS

JavaScript-la frequent-ah events trigger aagumbothu unnecessary function executions-a control panna Debouncing and Throttling use pannuvom.

Example:

- Search bar typing
- Button clicking
- Window resizing
- Scroll events
- API requests

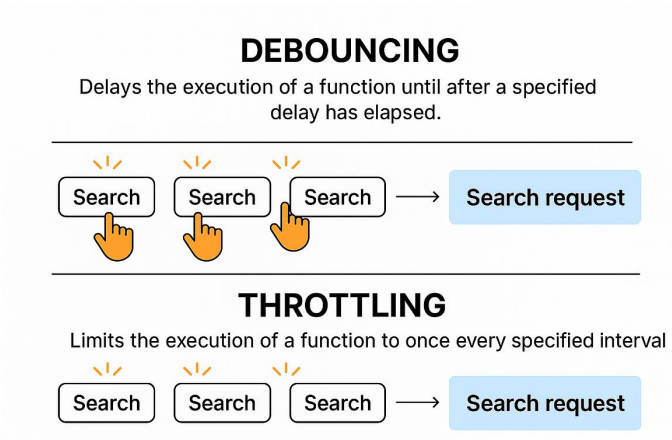
1. Debouncing

Debouncing na enna?

User continuous-ah oru action perform pannumbothu, action stop pannitu specific time delay-ku piragu mattum function execute aagurathuthaan Debouncing.

Real-time Example: Search bar.

User JavaScript nu type pannumbothu every letter-kum API call pogama, typing stop pannina 500ms apram API call pogum.



Example Code

```
function debounce(callback, delay) {
    let timer;
    return function() {
        clearTimeout(timer);
        timer = setTimeout(() => {
            callback();
        }, delay);
    };
}
function search() {
    console.log("API Called!");
}
let optimizedSearch = debounce(search, 1000);
optimizedSearch();
optimizedSearch();
optimizedSearch();
```

Output:

```
API Called!
```

Last function call-kku 1 second delay aana piragu mattum output varum.

Explanation:

1. setTimeout() delay set pannum.
2. clearTimeout() previous timer-a cancel pannum.
3. User action stop pannina delay complete aagum.
4. Appuram function execute aagum.

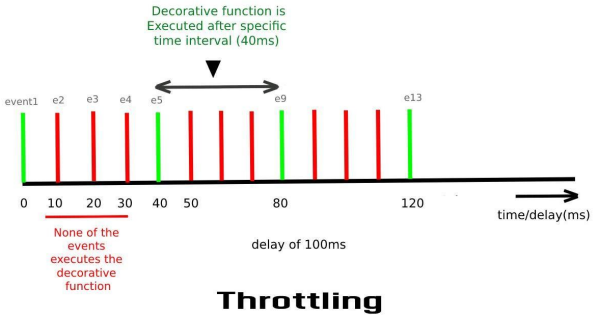
2. Throttling

Throttling na enna?

Oru function frequent-ah call aagumbothu, specific time interval-ku oru murai mattum execute aagura maathiri control pannurathuthaan Throttling.

Real-time Example: Scroll event.

User continuous-ah page scroll pannalum, every 1 second-ku oru murai mattum function execute aagum.



Example Code

```
function throttle(callback, delay) {
    let lastTime = 0;
    return function() {
        let currentTime = Date.now();
        if (currentTime - lastTime >= delay) {
            callback();
            lastTime = currentTime;
        }
    };
}
function handleScroll() {
    console.log("Scroll Event!");
}
let optimizedScroll = throttle(handleScroll, 1000);
optimizedScroll();
optimizedScroll();
optimizedScroll();
```

Output:

```
Scroll Event!
```

Continuous calls vandhalum, 1 second interval condition satisfy aagumbothu mattum execute aagum.

Explanation:

1. Date.now() current time-a return pannum.
2. lastTime previous execution time-a store pannum.
3. 1 second interval complete aagirundha function execute aagum.
4. Illana execution skip aagum.

3. Debouncing vs Throttling

Debouncing	Throttling
Action stop aanathukku piragu execute aagum	Fixed interval-la execute aagum
Previous timer cancel aagum	Calls interval base-la control aagum
Search input-ku useful	Scroll event-ku useful
API calls reduce pannum	Frequent executions control pannum
Example: 500ms after typing stops	Example: Every 1 second

4. Real-Time Applications

Debouncing

- Search suggestions

- Username availability check
- Form validation
- API search requests

Throttling

- Window scroll
- Window resize
- Mouse movement tracking
- Repeated button actions

> **Practical Task 1:** Make a frontend request to a different-origin public API and observe the browser's CORS response in the Console and Network panels.

> **Practical Task 2:** Create a simple frontend and backend example. Configure the backend to allow the frontend origin and test the cross-origin request.

CHAPTER 46

JWT Authentication

STRUCTURE · FLOW · GENERATE · VERIFY · SECURITY

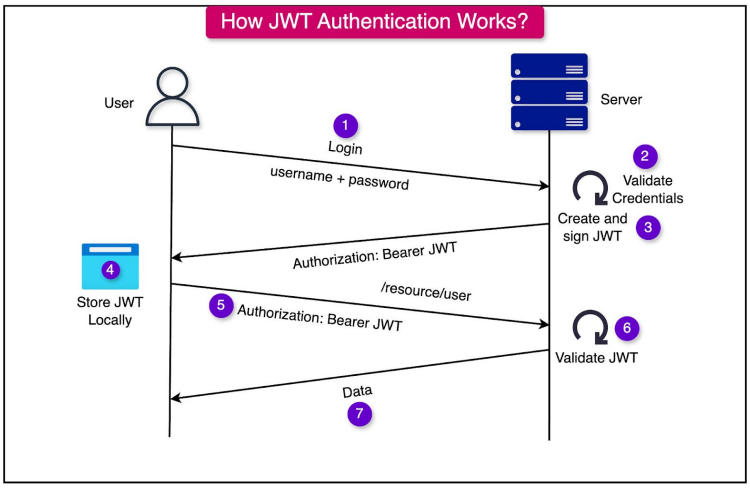
JWT na enna?

JWT stands for JSON Web Token.

User login pannina piragu, server oru token generate panni frontend-ku send pannum. Andha token use panni user authenticated-ah irukkarangala nu verify pannalam.

Real-time Example:

- User login pannuranga.
- Backend username and password verify pannum.
- Correct-ah irundha JWT token generate pannum.
- Frontend token receive pannum.
- Next API requests-la token send pannum.
- Backend token verify panni access provide pannum.



1. JWT Structure

JWT-la 3 parts irukkum:

```
Header.Payload.Signature
```

A. Header

Token type and signing algorithm details irukkum.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

B. Payload

User related information and claims irukkum.

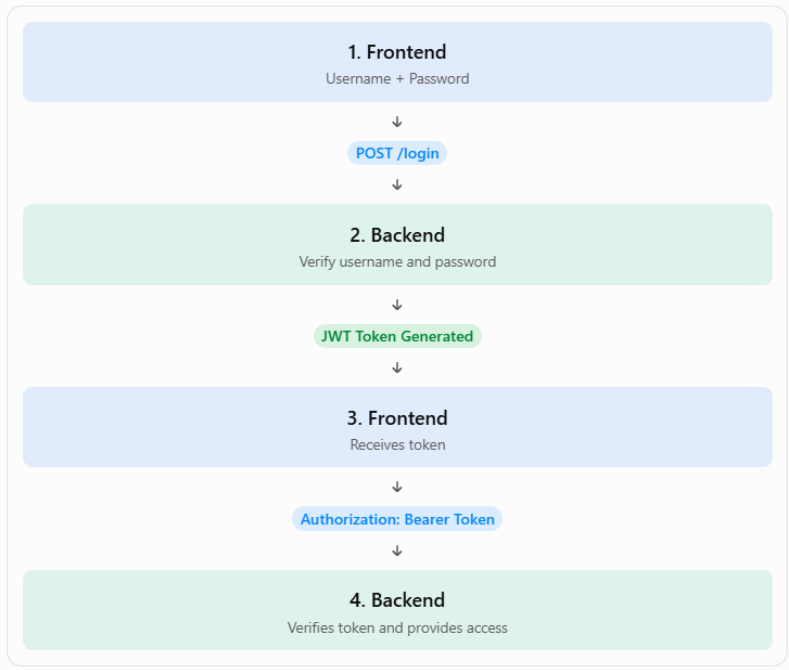
```
{
  "id": 101,
  "name": "Dhamo",
  "role": "user"
}
```

C. Signature

Token original-ah irukka, modify pannirukangala nu verify panna signature use pannuvom.

Remember: JWT payload normally Base64URL encoded mattum; encrypted illa. So password or sensitive information payload-la store panna koodadhu.

2. JWT Authentication Flow



3. JWT Token Generate Pannurathu

JavaScript-la jsonwebtoken package use panni JWT generate pannalam.

Install:

npm install jsonwebtoken

```
server.js
const jwt = require("jsonwebtoken");
const user = {
    id: 101,
    name: "Dhamo"
};
const secretKey = process.env.JWT_SECRET;
const token = jwt.sign(user, secretKey, {
    expiresIn: "1h"
});
console.log(token);
```

Screen Output:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Actual output long JWT token-ah irukkum. Ovvoru generated token-um different-ah irukkalam.

4. JWT Token Verify Pannurathu

Backend-ku vandha token valid-ah irukka nu verify pannuvom.

```
const decoded = jwt.verify(token, secretKey);
console.log(decoded);
```

Output:

```
{
  id: 101,
  name: 'Dhamo',
  iat: 1794050000,
  exp: 1794053600
}
```

- iat – Token issued time.
- exp – Token expiration time.

Invalid or expired token-na verification error throw pannum.

5. JWT Token API Request-la Send Pannurathu

Frontend-la API request send pannumbothu Authorization header-la token attach pannuvom.

fetch("http://localhost:8080/api/profile", {

method: "GET",

headers: {

"Authorization": "Bearer YOUR_JWT_TOKEN"

```
    }
})
.then(response => response.json())
.then(data => console.log(data));
```

Request Header:

GET /api/profile HTTP/1.1

Authorization: Bearer YOUR_JWT_TOKEN

Backend token verify pannitu user profile data return pannum.

6. JWT Authentication vs Authorization

Authentication	Authorization
User yaaru nu verify pannum	User-ku enna permission irukku nu check pannum
Login process	Access control
Username and password	User role and permissions
Example: Login success	Example: Admin page access

7. Important JWT Terms

Term	Meaning
JWT	JSON Web Token
Header	Token algorithm information
Payload	User claims/data
Signature	Token verification
jwt.sign()	Token generate pannum
jwt.verify()	Token verify pannum
Bearer Token	API request-la token send pannura format
expiresIn	Token validity time
iat	Issued At
exp	Expiration Time

8. Security Best Practices

1. Secret key-a frontend-la expose panna koodadhu.
2. JWT payload-la password store panna koodadhu.
3. HTTPS use pannanum.
4. Short-lived access tokens use pannalam.
5. Token storage carefully handle pannanum; XSS risk consider pannanum.
6. Secure, HttpOnly cookies use pannurathu many browser applications-ku useful option.
7. Logout and token revocation strategy implement pannanum.

Real-time IT Project Example: E-commerce website-la JWT Authentication use panni user login, profile access, order history, admin authorization ellam implement pannalam.

> **Practical Task 1:** Create a demonstration that splits a sample JWT into header, payload and signature sections and displays the decoded header and payload.

> **Practical Task 2:** Create a frontend demo that sends a sample JWT in an Authorization header when requesting a protected API route. Do not store passwords in the token.

CHAPTER 47

JavaScript Debugging (Chrome DevTools)

CONSOLE · SOURCES · BREAKPOINTS · PANELS · ERRORS

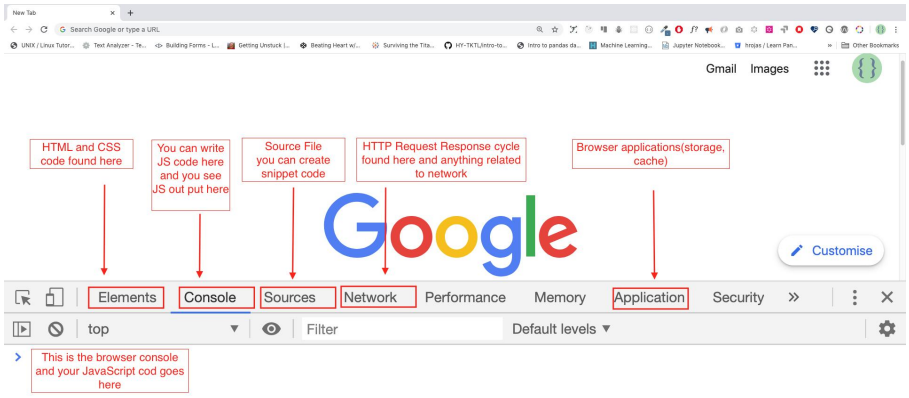
Debugging na enna?

JavaScript code-la irukkura errors-a identify panni, reason find panni, correct pannurathuthaan Debugging.

Chrome DevTools use panni JavaScript code, variables, errors, API requests, HTML and CSS inspect pannalam.

Chrome DevTools open panna:

- F12
- Ctrl + Shift + I
- Right-click → Inspect



1. Console Panel

JavaScript output and errors display aagura place.

Example:

```
console.log("Hello JavaScript");
console.warn("Warning Message");
console.error("Something went wrong");
```

Console Screen Output:

Hello JavaScript

Warning: Warning Message

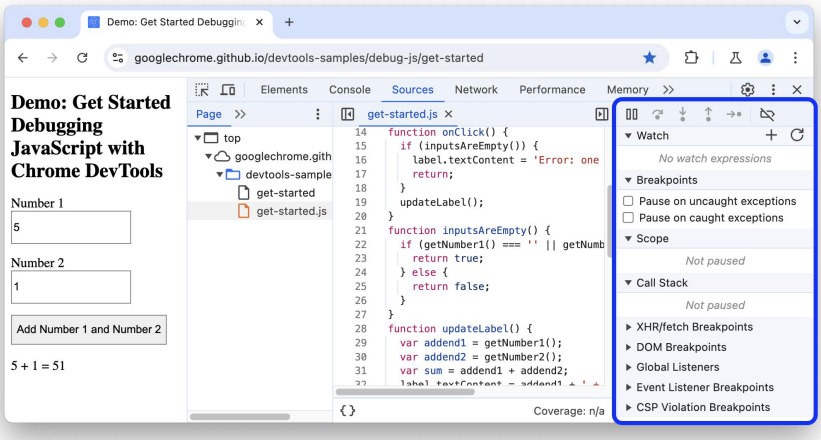
Error: Something went wrong

Common console methods:

Method	Purpose
console.log()	Normal output
console.error()	Error display
console.warn()	Warning display
console.table()	Data table format
console.clear()	Console clear
console.info()	Information display
console.count()	Function call count
console.time()	Execution time start
console.timeEnd()	Execution time end

2. Sources Panel

JavaScript files inspect panni, breakpoints set panni, code execution-a debug pannalam.



Example:

```
let a = 10;
let b = 20;
let result = a + b;
console.log(result);
```

Output:

```
30
```

Sources panel-la line number click panni breakpoint set pannalam. Andha line execute aagumbothu JavaScript pause aagum.

3. Breakpoints

Breakpoint na JavaScript code particular line-la execution-a temporarily stop pannurathu.

Example:

```
let a = 10;
let b = 20;
let result = a + b; // Breakpoint
console.log(result);
```

Screen Output (Paused State):

Paused on breakpoint

a = 10

b = 20

result = 30

Execution pause aagumbothu current variables-a inspect pannalam.

4. Debugger Statement

debugger keyword use panni code-la direct-ah breakpoint set pannalam.

Example:

```
let a = 10;
let b = 20;
```

debugger;

```
let result = a + b;
console.log(result);
```

Screen Output:

```
Paused in debugger
```

Debugger continue pannina:

30

5. Watch

Watch use panni debugging time-la variables and expressions values-a monitor pannalam.

Example:

```
let a = 10;
let b = 20;
let result = a + b;
console.log(result);
```

Watch Panel Output:

a 10

b 20

result 30

6. Call Stack

Call Stack na current function and adha call panna previous functions-oda execution order-a display pannum.

Example:

```
function first() {
    second();
}
function second() {
    third();
}
function third() {
    debugger;
    console.log("Hello");
}
first();
```

Call Stack Screen Output:

```
third()
second()
first()
```

Global

7. Scope

Scope section-la current execution context-la available-ah irukkura variables-a inspect pannalam.

Example:

```
let name = "Dhamo";
function greet() {
    let age = 23;
    debugger;
}
greet();
```

Scope Panel Output:

Local:

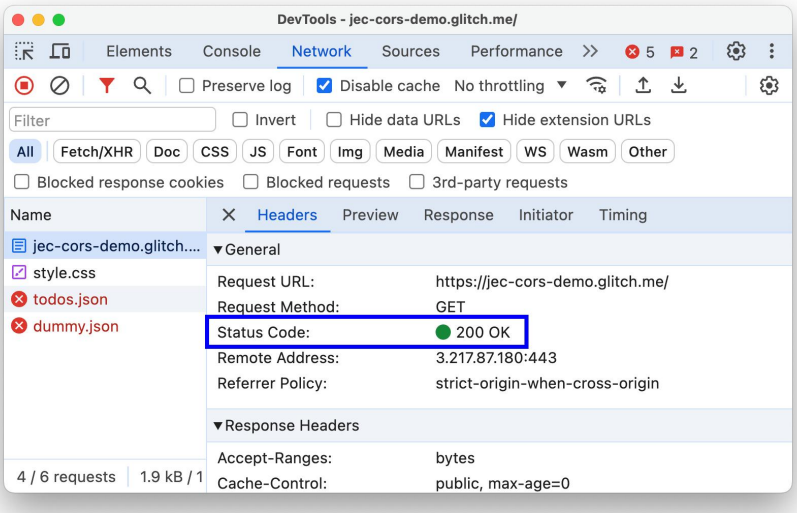
age: 23

Script:

name: "Dhamo"

8. Network Panel

Frontend-la irundhu server-ku pogura API requests and server responses-a inspect pannalam.



Example:

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => console.log(data));
```

Network Screen Output:

Name Status Type

users 200 fetch

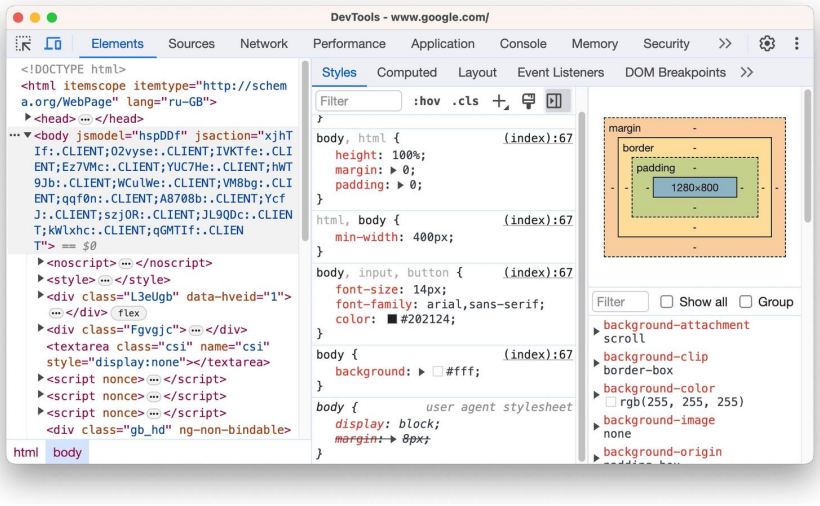
Request click pannina:

- Headers
- Payload
- Preview
- Response
- Timing

Details inspect pannalam.

9. Elements Panel

HTML structure and CSS styles inspect and modify pannalam.



HTML Example:

```
<h1 id="title">Hello World</h1>
```

Elements Panel Output:

```
<h1 id="title">Hello World</h1>
```

Element select panni CSS styles temporary-ah change pannalam.

10. Application Panel

Browser storage and application-related data inspect pannalam.

Example:

```
localStorage.setItem("username", "Dhamo");
sessionStorage.setItem("role", "Developer");
```

Application Panel Output:

Local Storage:

username = Dhamo

Session Storage:

role = Developer

Common storage:

- Local Storage
- Session Storage
- Cookies
- IndexedDB

- Cache Storage

11. Performance Panel

Website performance and JavaScript execution time analyze pannalam.

Example:

```
console.time("Loop");
for (let i = 0; i < 100000; i++) {
    // Loop execution
}
console.timeEnd("Loop");
```

Console Output (example):

Loop: 3.5 ms

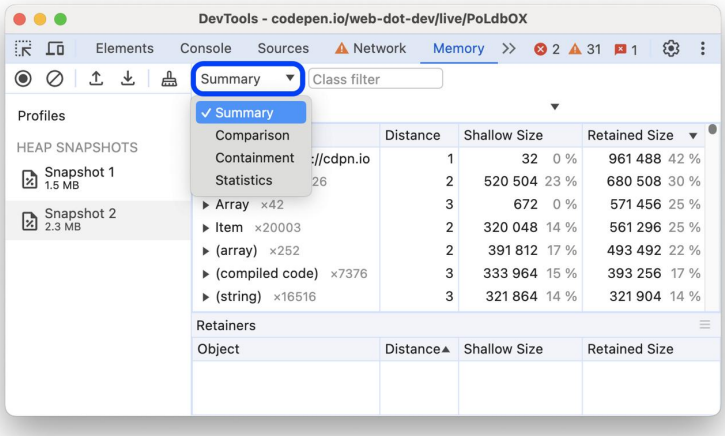
Execution time system and browser-a poruthu maarum.

12. Memory Panel

Website-la JavaScript memory usage analyze pannalam.

Common uses:

- Memory usage inspect pannurathu.
- Heap snapshots edukkirathu.
- Memory leaks identify pannurathu.
- Unnecessary objects memory-la irukka nu check pannurathu.



13. Error Types

JavaScript-la common-ah encounter pannura errors:

Error	Meaning
SyntaxError	Code syntax mistake
ReferenceError	Undefined variable access
TypeError	Invalid operation on a value
RangeError	Value allowed range-ku veliya irukku
Network Error	Request/network problem

Example:

```
console.log(username);
```

Console Output:

```
Uncaught ReferenceError: username is not defined
```

CHAPTER 48

Important Topics

DEBUGGING · BREAKPOINTS · WATCH · CALL STACK · PERFORMANCE

Chrome DevTools use panni JavaScript code-la irukkura errors, API problems, performance issues ellam identify panni fix pannalam.

1. Console Debugging

Console debugging na, JavaScript code execute aagumbothu values and errors check pannurathu.

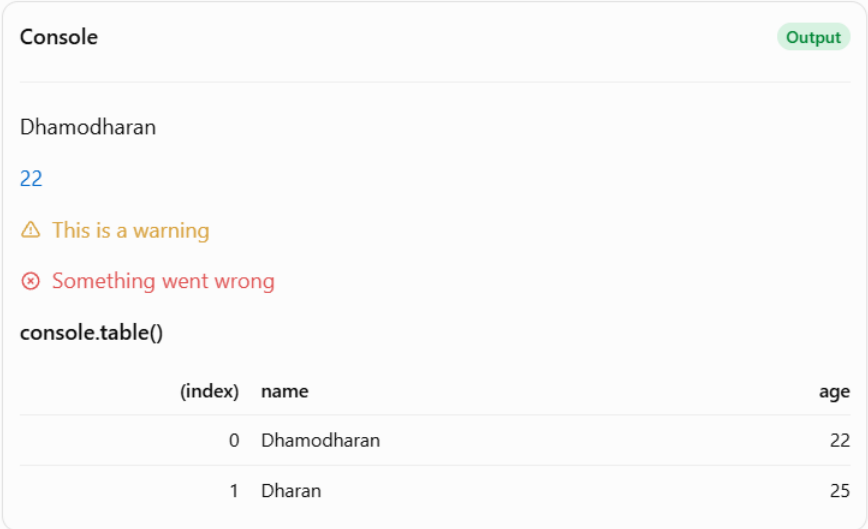
Important Console Methods:

Method	Use
console.log()	Normal output display
console.info()	Information display
console.warn()	Warning display
console.error()	Error display
console.table()	Data table format-la display
console.clear()	Console clear pannum
console.count()	Function execution count
console.time()	Execution time start
console.timeEnd()	Execution time end

Example:

```
let name = "Dhamodharan";
let age = 22;
console.log(name);
console.info(age);
console.warn("This is a warning");
console.error("Something went wrong");
console.table([
    { name: "Dhamodharan", age: 22 },
    { name: "Dharan", age: 25 }
]);
```

Console Output:



2. Breakpoints

Breakpoint na, JavaScript code execute aagumbothu oru particular line-la execution-ah temporarily stop pannurathu.

Uses:

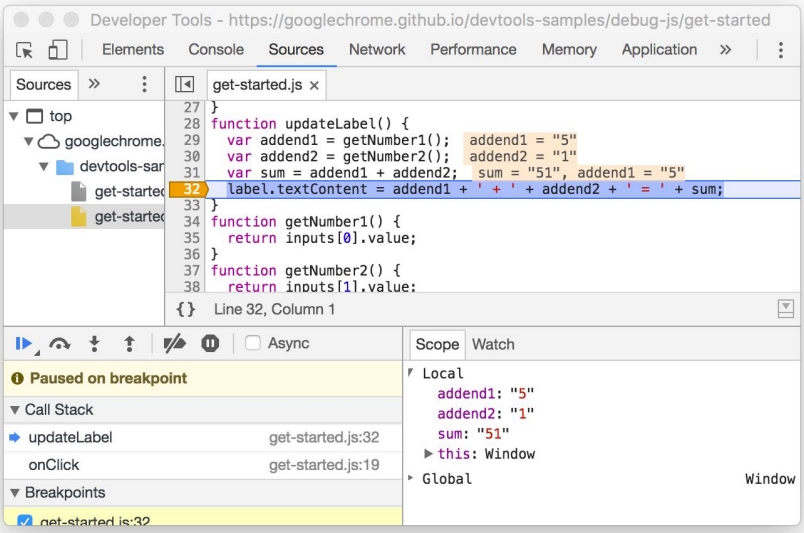
1. Variable values check pannalam.
2. Code execution step-by-step-ah analyze pannalam.
3. Error enga nadakkuthunu identify pannalam.

Example:

```
let a = 10;
let b = 20;
let result = a + b;
console.log(result);
```

Chrome DevTools-la:

1. F12 press pannunga.
2. Sources tab open pannunga.
3. JavaScript file select pannunga.
4. Line number pakkathula click pannunga.
5. Breakpoint set aagum.



Screen Output:

```
Debugger Paused
a = 10
b = 20
result = Not yet executed
```

3. debugger Statement

debugger statement use pannina, browser DevTools open irukkumbothu code execution andha line-la pause aagum.

Example:

```
let a = 10;
let b = 20;
```

debugger;

```
let result = a + b;
console.log(result);
```

Screen Output:

```
Debugger Paused
```

```
let a = 10;
let b = 20;
```

debugger; <-- Execution stopped here

```
let result = a + b;
```

DevTools-la Resume (▶) click pannina, remaining code execute aagum.

Use: Code execution-ah manually inspect panna useful.

4. Watch and Scope

Ithu rendu Sources tab-la debugging pannumbothu use aagum.

Watch:

Watch use panni specific variables or expressions-oda values-ah continuously monitor pannalam.

Example:

```
let a = 10;
let b = 20;
let result = a + b;
```

debugger;

```
console.log(result);
```

Watch Panel Output:

Watch

a 10

b 20

result 30

Scope:

Scope panel-la current execution context-la available-a irukkura variables-ah check pannalam.

- Local: Current function variables.
- Script: Script-level variables.
- Global: Global scope variables.

Example:

```
let name = "Dhamodharan";
function display() {
    let age = 22;
    debugger;
    console.log(name, age);
}
display();
```

Scope Panel Output:

Local:

age = 22

Script:

```
name = "Dhamodharan"
```

5. Call Stack

Call Stack na, functions endha order-la call aaguthu and current function enga irunthu call aachu nu show pannum.

JavaScript-la function call aagumbothu Call Stack-la add aagum. Function complete aana remove aagum.

Example:

```
function first() {
    second();
}
function second() {
    third();
}
function third() {
    debugger;
    console.log("Hello");
}
first();
```

Call Stack Output:

Call Stack

```
third()
second()
first()
```

Global

Use: Nested functions-la error enga start aachu nu identify panna useful.

6. Network API Debugging

Network debugging na, frontend-la irunthu backend-ku send pannura API requests and responses-ah inspect pannurathu.

Chrome DevTools:

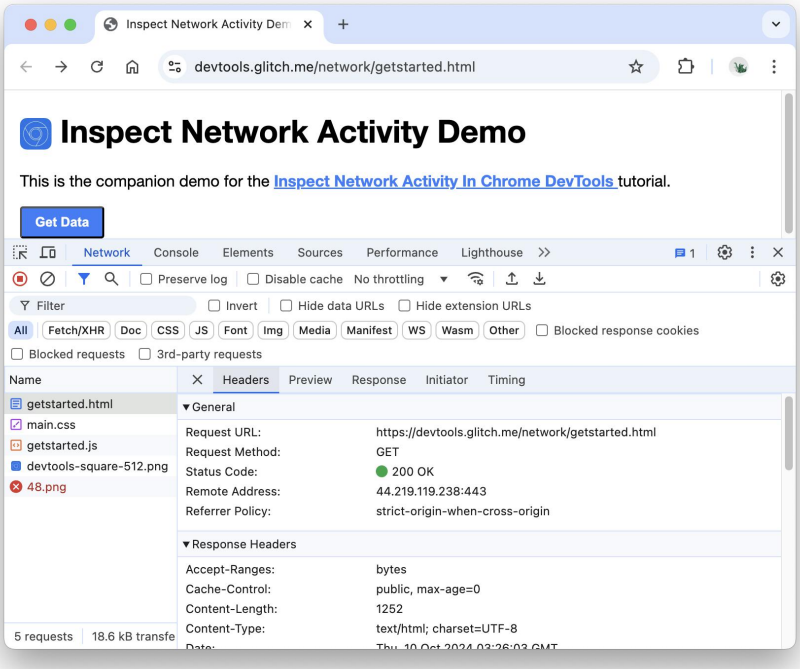
1. F12 press pannunga.
2. Network tab open pannunga.
3. Fetch/XHR select pannunga.
4. API request click pannunga.

Example:

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error(error));
```

Network Tab-la Check Pannurathu:

Section	Purpose
Headers	Request URL, method, status code
Payload	Send pannura data
Preview	Response preview
Response	Complete response data
Timing	Request execution time



Screen Output:

Request URL:

https://jsonplaceholder.typicode.com/users

Request Method:

GET

Status Code:

200 OK

Response:

[

```
  {
    "id": 1,
    "name": "Leanne Graham"
  },
  {
    "id": 2,
    "name": "Ervin Howell"
  }
]
```

7. JavaScript Error Handling

Error handling na, JavaScript code-la error varumbothu application crash aagama handle pannurathu.

Important Keywords:

- try – Error varalaam nu expect pannura code.
- catch – Error-ah handle pannum.
- finally – Error vanthalum varalenaalum execute aagum.
- throw – Custom error create pannum.

Example:

```
try {
    let result = 10 / 0;
    if (!isFinite(result)) {
        throw new Error("Invalid result");
    }
    console.log(result);
}
catch (error) {
    console.log(error.message);
}
```

finally {

```
    console.log("Program completed");
}
```

Output:

```
Invalid result
Program completed
```

Important Error Types:

Error	Meaning
SyntaxError	Syntax mistake
ReferenceError	Undefined variable access
TypeError	Invalid operation on a value
RangeError	Value outside allowed range

8. Local Storage and Session Storage Inspection

Chrome DevTools-la browser storage data-ah inspect pannalam.

Local Storage:

- Data browser close pannalum remain aagum.
- Manually clear pannum varai store aagum.

Session Storage:

- Current browser tab session-ku mattum data store aagum.
- Tab close pannumbothu data clear aagum.

Example:

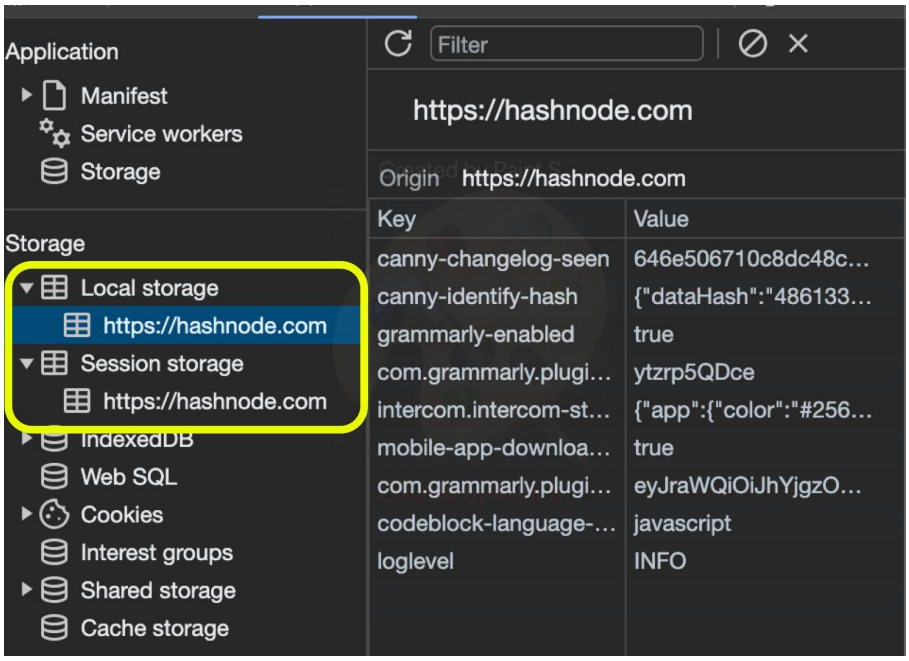
```
localStorage.setItem("username", "Dhamodharan");
sessionStorage.setItem("course", "JavaScript");
console.log(localStorage.getItem("username"));
console.log(sessionStorage.getItem("course"));
```

Output:

```
Dhamodharan
JavaScript
```

Chrome DevTools-la:

1. F12 open pannunga.
2. Application tab select pannunga.
3. Storage section open pannunga.
4. Local Storage or Session Storage select pannunga.



9. Performance Debugging

Performance debugging na, website slow-ah load aagutha, JavaScript execution time adhigama irukka nu analyze pannurathu.

Important Tools:

- Performance tab – Website performance analyze pannum.
- CPU profiling – JavaScript execution time check pannum.
- Network timing – API and resource loading time check pannum.
- Lighthouse – Website performance and other quality metrics analyze pannum.

Example:

```
console.time("Calculation");
let sum = 0;
for (let i = 0; i < 1000000; i++) {
    sum += i;
}
console.timeEnd("Calculation");
console.log(sum);
```

Output (approximate):

Calculation: 5.2 ms

499999500000

Execution time computer and browser based-ah change aagum.

10. Memory Leak Detection

Memory leak na, application-ku thevai illatha memory properly release aagama continuously occupy pannurathu.

Common Causes:

1. Unnecessary global variables.
2. setInterval() clear pannama vidurathu.
3. Event listeners remove pannama vidurathu.
4. Unused references maintain pannurathu.

Example – Memory Leak:

```
let numbers = [];
setInterval(() => {
    numbers.push(new Array(10000).fill("Hello"));
}, 1000);
```

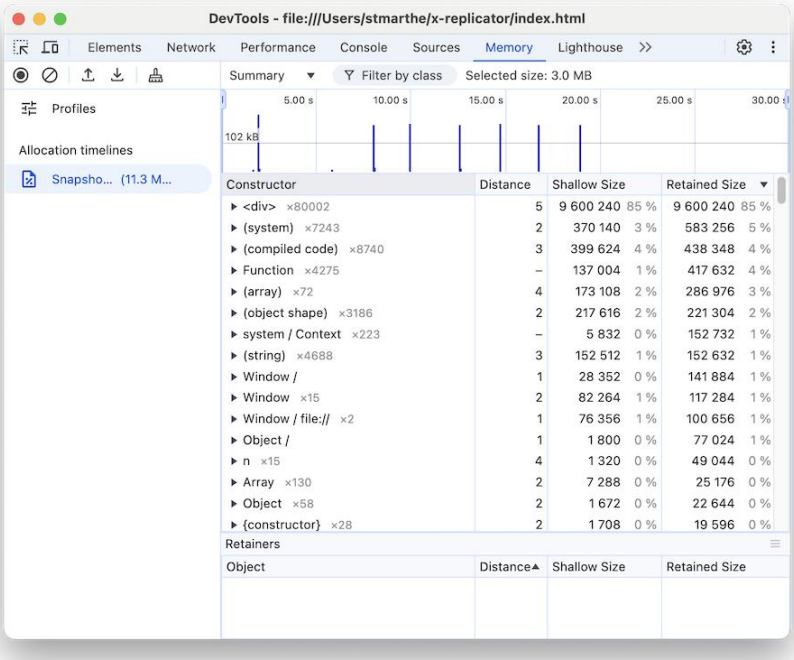
Intha code every second new data add pannite irukkum. numbers array grow aagite irukkum.

Memory Leak Fix Example:

```
let numbers = [];
let timer = setInterval(() => {
    numbers.push(new Array(10000).fill("Hello"));
    if (numbers.length >= 5) {
        clearInterval(timer);
        numbers = [];
    }
}, 1000);
```

Chrome DevTools-la:

1. F12 open pannunga.
2. Memory tab select pannunga.
3. Heap Snapshot edunga.
4. Application memory usage analyze pannunga.
5. Repeated snapshots compare panni unnecessary memory growth identify pannunga.



Quick Revision

IT companies-ku important: Console Debugging, Breakpoints, Error Handling, Network API Debugging, Call Stack, Performance Debugging and Memory Leak Detection practical-ah therinjirukkanum. React and JavaScript application development-la ivai frequently useful-a irukkum.

> **Practical Task 1:** Create a JavaScript program with an intentional error. Use the Console and Sources panels to locate and fix the error.

> **Practical Task 2:** Debug a student-mark calculation program using breakpoints, the Watch panel, Scope and Call Stack. Inspect an API request in the Network panel.

CHAPTER 49

Interview Questions and Answers

60 QUESTIONS · CONCEPTS · EXAMPLES · QUICK REVISION

1. What is JavaScript?

Answer: JavaScript is a programming language used to make web pages interactive and dynamic. It can handle user actions, validate forms, update HTML content, and communicate with servers.

2. What are the main features of JavaScript?

Answer:

- Lightweight and interpreted.
- Dynamically typed.
- Supports object-oriented programming.
- Supports asynchronous programming.
- Runs in web browsers.
- Supports event handling.

3. What are JavaScript comments?

Answer: Comments are used to explain code. They are ignored during execution.

```
// Single-line comment

/*
Multi-line
comment
*/
```

4. What is the difference between var, let, and const?

Answer:

Keyword	Scope	Reassignment
var	Function	Allowed
let	Block	Allowed
const	Block	Not allowed

5. What are JavaScript data types?

Answer: Data types define the kind of value stored in a variable.

Main primitive types:

- String

- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Objects and arrays are non-primitive values.

6. What is the difference between null and undefined?

Answer:

- undefined: A variable has been declared but has not been assigned a value.
- null: Represents an intentional absence of a value.

7. What is the typeof operator?

Answer: The typeof operator returns the data type of a value.

```
let name = "Dhamodharan";
console.log(typeof name);
```

Output:

```
string
```

8. What are string methods in JavaScript?

Answer: String methods are built-in methods used to work with text.

Examples:

- toUpperCase()
- toLowerCase()
- trim()
- slice()
- includes()
- replace()

9. What are JavaScript operators?

Answer: Operators are symbols used to perform operations on values.

Types include:

- Arithmetic operators (+, -, *, /)
- Assignment operators (=, +=)
- Comparison operators (==, ===, >)

- Logical operators (&&, ||, !)
- Ternary operator (? :)

10. What is the difference between == and ===?

Answer:

- == compares values after type conversion when necessary.
- === compares both value and data type without type conversion.

```
console.log(10 == "10");  // true
console.log(10 === "10"); // false
```

Part 2: Window Methods, Arrays and Functions

11. What is the window object?

Answer: The window object represents the browser window. It provides methods and properties to interact with the browser.

Examples:

```
window.alert("Welcome");
window.confirm("Continue?");
window.prompt("Enter your name");
```

12. What is the difference between alert(), prompt(), and confirm()?

Answer:

- alert() displays a message.
- prompt() accepts user input.
- confirm() asks the user to confirm an action and returns true or false.

13. What is the Date object in JavaScript?

Answer: The Date object is used to work with dates and times.

```
let today = new Date();

console.log(today);
console.log(today.getFullYear());
console.log(today.getMonth());
console.log(today.getDate());
```

14. What is an array in JavaScript?

Answer: An array is a data structure used to store multiple values in a single variable.

```
let fruits = ["Apple", "Banana", "Orange"];

console.log(fruits[0]);
```

Output: Apple

15. What is the difference between push() and pop()?

Answer:

- push() adds an element to the end of an array.
- pop() removes the last element from an array.

16. What is the difference between shift() and unshift()?

Answer:

- shift() removes the first element.
- unshift() adds an element to the beginning.

17. What is the difference between slice() and splice()?

Answer:

- slice() extracts a portion of an array without modifying the original array.
- splice() adds, removes, or replaces elements and modifies the original array.

18. What is the difference between forEach(), map(), and filter()?

Answer:

- forEach() executes a function for every array element.
- map() creates a new array containing transformed elements.
- filter() creates a new array containing elements that satisfy a condition.

19. What is a Set in JavaScript?

Answer: A Set is a collection of unique values. It does not store duplicate values.

```
let numbers = new Set([10, 20, 10, 30]);

console.log(numbers);
```

Output: Set(3) {10, 20, 30}

20. What is a function in JavaScript?

Answer: A function is a reusable block of code designed to perform a specific task.

```
function add(a, b) {
    return a + b;
}

console.log(add(10, 20));
```

Output: 30

Part 3: Objects and Object-Oriented Programming

21. What is an object in JavaScript?

Answer: An object is a collection of key-value pairs used to store related data and methods.

```
let car = {
    brand: "BMW",
    price: 15
};
```

22. What is the this keyword in JavaScript?

Answer: The this keyword refers to the current execution context. In a regular object method call, it usually refers to the object that called the method.

```
let car = {
    price: 15,

    showPrice() {
        console.log(this.price);
    }
};

car.showPrice();
```

23. What is object destructuring?

Answer: Object destructuring is a feature used to extract properties from an object into variables.

```
let student = {
    name: "Dhamodharan",
    age: 22
};

let { name, age } = student;
```

24. What is the spread operator?

Answer: The spread operator (...) expands elements of an array or properties of an object.

```
let a = [10, 20];
let b = [...a, 30];

console.log(b);
```

Output: [10, 20, 30]

25. What is a class in JavaScript?

Answer: A class is a template used to create objects with properties and methods.

```
class Student {
    constructor(name) {
        this.name = name;
    }
}

let s1 = new Student("Dhamodharan");
```

26. What is a constructor in JavaScript?

Answer: A constructor is a special method inside a class that automatically executes when an object is created using the new keyword.

27. What is OOP in JavaScript?

Answer: OOP stands for Object-Oriented Programming. It is a programming approach that organizes code using objects and classes.

Its four main concepts are:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

28. What is encapsulation?

Answer: Encapsulation means combining data and methods inside a single unit, such as a class, and controlling access to that data.

29. What is inheritance?

Answer: Inheritance allows one class to acquire properties and methods from another class using the extends keyword.

```
class Animal {
    eat() {
        console.log("Eating");
    }
}

class Dog extends Animal {
    bark() {
        console.log("Barking");
    }
}
```

30. What is the DOM in JavaScript?

Answer: DOM stands for Document Object Model. It represents an HTML document as a tree of objects, allowing JavaScript to access and modify HTML elements.

```
document.getElementById("title").innerHTML = "Hello";
```

Part 4: Events, Storage and Advanced Functions

31. What are JavaScript events?

Answer: Events are actions or occurrences detected by the browser.

Examples:

- onclick
- onmouseover
- onmouseout
- onkeydown
- onkeyup
- onsubmit

32. What is event handling?

Answer: Event handling is the process of executing JavaScript code when a particular event occurs.

```
button.addEventListener("click", function() {
    alert("Button clicked");
});
```

33. What is form validation?

Answer: Form validation checks whether the information entered by a user meets the required conditions before submitting the form.

Examples include checking:

- Empty fields.
- Valid email format.
- Password length.
- Required input values.

34. What is the difference between Cookies, Local Storage, and Session Storage?

Answer:

Feature	Cookies	Local Storage	Session Storage
Storage	Browser	Browser	Browser tab session
Expiration	Configurable	No automatic expiry	When the tab session ends
Server sent automatically	Yes, when applicable	No	No

35. What is error handling in JavaScript?

Answer: Error handling prevents unexpected errors from stopping the normal execution of a program.

Important keywords:

- try
- catch
- finally
- throw

36. What are rest parameters?

Answer: Rest parameters allow a function to accept multiple arguments and collect them into an array.

```
function add(...numbers) {
    return numbers;
}

console.log(add(10, 20, 30));
```

37. What is a closure in JavaScript?

Answer: A closure is created when an inner function remembers and accesses variables from its outer function, even after the outer function has finished executing.

```
function counter() {
    let count = 0;

    return function() {
        count++;
        console.log(count);
    };
}

let result = counter();
result();
result();
```

Output:

```
1
2
```

38. What is a higher-order function?

Answer: A higher-order function is a function that accepts another function as an argument or returns another function.

Examples: map(), filter(), and reduce().

Part 5: Asynchronous JavaScript and APIs

39. What is the difference between synchronous and asynchronous JavaScript?

Answer:

- Synchronous JavaScript executes statements one after another.
- Asynchronous JavaScript allows certain operations to complete later without blocking the rest of the program.

40. What is setTimeout()?

Answer: setTimeout() executes a function once after a specified delay.

```
setTimeout(() => {
    console.log("Hello");
}, 2000);
```

41. What is AJAX?

Answer: AJAX stands for Asynchronous JavaScript and XML. It allows web pages to communicate with servers and update data without reloading the entire page.

42. What is the Fetch API?

Answer: The Fetch API is used to make HTTP requests and retrieve data from servers.

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

43. What is a Promise in JavaScript?

Answer: A Promise is an object representing the eventual success or failure of an asynchronous operation.

It has three states:

- Pending
- Fulfilled
- Rejected

44. What are then(), catch(), and finally() in Promises?

Answer:

- then() handles successful completion.
- catch() handles rejection or errors.
- finally() executes after the Promise settles, whether fulfilled or rejected.

45. What is the difference between microtasks and macrotasks?

Answer:

- Microtasks include Promise callbacks such as .then().
- Macrotasks include callbacks from setTimeout() and similar browser APIs.

After the current synchronous code finishes, the event loop processes queued microtasks before moving to the next macrotask.

46. What is the difference between shallow copy and deep copy?

Answer:

- Shallow copy copies the top-level properties, but nested objects may still share references.
- Deep copy creates independent copies of nested data as well.

47. What is the prototype chain in JavaScript?

Answer: The prototype chain is JavaScript's mechanism for inheritance. When a property is not found on an object, JavaScript searches its prototype and continues up the chain.

48. What are ES6 modules?

Answer: ES6 modules allow developers to organize JavaScript code into separate files and share functionality using export and import.

```
// math.js
export function add(a, b) {
    return a + b;
}

// app.js
import { add } from "./math.js";
```

Part 6: Web Technologies, Security and Debugging

49. What is a REST API?

Answer: REST API is an architectural approach that allows applications to communicate with each other using HTTP requests and resources.

50. What are the common HTTP methods?

Answer:

Method	Purpose
GET	Retrieve data
POST	Create or submit data
PUT	Replace existing data
PATCH	Partially update data
DELETE	Delete data

51. What are HTTP status codes?

Answer: HTTP status codes indicate the result of a server request.

Examples:

- 200 – OK
- 201 – Created
- 400 – Bad Request
- 401 – Unauthorized
- 403 – Forbidden
- 404 – Not Found
- 500 – Internal Server Error

52. What is CORS?

Answer: CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page can access resources from another origin.

53. What is debouncing in JavaScript?

Answer: Debouncing delays function execution until a specified time has passed after the last event.

Example: A search box that waits until the user stops typing before sending an API request.

54. What is throttling in JavaScript?

Answer: Throttling limits how frequently a function can execute within a specified time period.

Example: Limiting the execution of a scroll event handler.

55. What is authentication?

Answer: Authentication is the process of verifying a user's identity, usually through login credentials or another verification method.

56. What is authorization?

Answer: Authorization determines what resources or actions an authenticated user is allowed to access.

57. What is JWT?

Answer: JWT stands for JSON Web Token. It is a compact token format commonly used to transfer claims between parties and support authentication systems.

A JWT generally contains three parts:

- Header
- Payload
- Signature

58. What is Chrome DevTools?

Answer: Chrome DevTools is a set of browser development tools used to inspect, debug, test, and analyze web applications.

Important panels:

- Elements
- Console
- Sources
- Network
- Application
- Performance
- Memory

59. What is a breakpoint in JavaScript debugging?

Answer: A breakpoint is a point where JavaScript execution pauses so developers can inspect variables, understand program flow, and identify errors.

60. What is the difference between console.log() and console.table()?

Answer:

- console.log() displays values in the browser console.
- console.table() displays arrays or objects in a table format, making structured data easier to read.

CHAPTER 50

Mini Projects

CALCULATOR · TO-DO · CLOCK · QUIZ · WEATHER · CART

1. **Simple Calculator** (Beginner)

Features:

- Addition, subtraction, multiplication and division.
- Clear and delete buttons.
- Display calculation results.
- Handle division by zero.

Concepts: Variables, operators, functions, DOM, events.

2. Student Grade Calculator (Beginner)

Features:

- Enter student name and marks.
- Calculate total and average.
- Display grade based on marks.
- Show pass or fail status.

Concepts: Variables, arrays, conditions, functions, operators.

3. To-Do List Application (Beginner)

Features:

- Add new tasks.
- Delete tasks.
- Mark tasks as completed.
- Save tasks using Local Storage.

Concepts: Arrays, objects, DOM, events, Local Storage.

4. Digital Clock & Stopwatch (Intermediate)

Features:

- Display current time.
- Show current date.
- Start and stop stopwatch.
- Reset stopwatch.

Concepts: Date object, functions, setInterval(), events.

5. Quiz Application (Intermediate)

Features:

- Display multiple-choice questions.
- Select answers.
- Calculate scores.

- Display final results.
- Restart the quiz.

Concepts: Arrays, objects, loops, conditions, DOM, events.

6. Weather Application (Intermediate)

Features:

- Search weather by city.
- Display temperature.
- Show humidity and wind speed.
- Display weather conditions.
- Handle API errors.

Concepts: Fetch API, Promises, async/await, DOM, error handling.

7. Expense Tracker (Advanced)

Features:

- Add income and expenses.
- Categorize transactions.
- Calculate total balance.
- Delete transactions.
- Save records using Local Storage.

Concepts: Objects, arrays, array methods, functions, DOM, Local Storage.

8. E-Commerce Shopping Cart (Advanced)

Features:

- Display product list.
- Add products to cart.
- Increase or decrease quantity.
- Remove products.
- Calculate subtotal and total price.
- Save cart information.

Concepts: Objects, classes, array methods, spread operator, DOM, events, Local Storage.